

UNIVERSIDAD NACIONAL SAN CRISTÓBAL DE HUAMANGA

Facultad de Ingeniería de Minas, Geología y Civil

Escuela Profesional de Ingeniería de Sistemas



INFORME FINAL DE PROYECTO

Democra: Plataforma Web SaaS Multi-Tenant para la Gestión Integral de Organizaciones de Voluntariado (ONG)

Asignatura: Pruebas y Aseguramiento de la Calidad de Software

Presentado por:

N.º	Apellidos y Nombres	Código
1	Paipay Vega, Eduardo Sebastián	27222136

Docente:

Mg. Ing. Richard Zapata Casaverde

Dedicatoria

*A quienes dedican su tiempo al servicio de los demás a través del voluntariado,
y a los docentes de la Escuela Profesional de Ingeniería de Sistemas,
cuya formación hizo posible este trabajo.*

Agradecimiento

Expreso mi agradecimiento a la Universidad Nacional de San Cristóbal de Huamanga y, en particular, al docente de la asignatura de Pruebas y Aseguramiento de la Calidad de Software, por la orientación metodológica brindada durante el desarrollo de este informe. Agradezco asimismo a quienes contribuyeron con la construcción y documentación de la plataforma Democra, cuyo repositorio constituye el objeto de estudio del presente trabajo.

Resumen

El presente informe documenta y evalúa, mediante un enfoque de ingeniería inversa documental sobre el repositorio, el diseño y la implementación de **Democra**, una plataforma web de tipo *Software as a Service* (SaaS) multi-tenant para la gestión integral de organizaciones de voluntariado (ONG). El sistema se estructura en un cliente *Single Page Application* (SPA) construido con React, TypeScript y Vite, y una interfaz de programación de aplicaciones (API) REST implementada con Node.js y Express sobre la plataforma de datos Supabase (PostgreSQL 16). A nivel de datos, adopta un modelo multi-esquema (public, ong, rrhh, finanzas, clinico, academico, comunicaciones) con aislamiento por inquilino reforzado por *Row Level Security* (RLS) y funciones de base de datos. La seguridad se sustenta en autenticación Supabase, control de acceso basado en roles y permisos (RBAC) con negación por defecto, un motor de evaluación de riesgo con verificación en dos pasos (OTP), auditoría forense y borrado lógico con políticas de retención. Metodológicamente, el desarrollo se gobierna con la metodología DDS (*Design Documentation System* asistido por IA), organizada en ocho fases del ciclo de vida. La investigación es de tipo aplicada-tecnológica, nivel descriptivo y diseño no experimental transversal. Las variables de estudio son el Funcionamiento y la Usabilidad; sus indicadores se validan mediante análisis documental del código, pruebas automatizadas (Jest, Vitest, supertest, pgTAP) y cobertura de código y pruebas automatizadas. Los resultados cuantitativos de cobertura y rendimiento se estructuran y se reservan para su ejecución en un entorno controlado, sin reportar valores no verificados.

Palabras clave: SaaS multi-tenant, ONG, voluntariado, React, Supabase, PostgreSQL, RBAC, RLS, aseguramiento de la calidad, metodología DDS.

Abstract

This report documents and evaluates, through a documentary reverse-engineering approach over the repository, the design and implementation of **Democra**, a multi-tenant Software as a Service (SaaS) web platform for the comprehensive management of volunteer organizations (NGOs). The system comprises a Single Page Application (SPA) client built with React, TypeScript and Vite, and a REST API implemented with Node.js and Express on top of the Supabase data platform (PostgreSQL 16). At the data level, it adopts a multi-schema model with tenant isolation enforced through Row Level Security (RLS) and database functions. Security relies on Supabase authentication, Role-Based Access Control (RBAC) with a deny-by-default policy, a risk evaluation engine with two-step verification (OTP), forensic auditing, and soft deletion with retention policies. Methodologically, the development is governed by the DDS methodology (Design Documentation System assisted by AI), organized into eight lifecycle phases. The research is applied-technological and descriptive, with a non-experimental cross-sectional design. The study variables are Functioning and Usability; their indicators are validated through documentary code analysis, automated testing (Jest, Vitest, supertest, pgTAP) and code coverage and automated testing. Quantitative coverage and performance results are structured and reserved for execution in a controlled environment, without reporting unverified values.

Keywords: multi-tenant SaaS, NGO, volunteering, React, Supabase, PostgreSQL, RBAC, RLS, quality assurance, DDS methodology.

Contents

Dedicatoria	i
Agradecimiento	ii
Resumen	iii
Abstract	iv
Introducción	1
1 Planteamiento del Problema	2
1.1 Diagnóstico y Enunciado del Problema	2
1.2 Formulación del Problema	3
1.2.1 Problema General	3
1.2.2 Problemas Específicos	3
1.3 Hipótesis	3
1.4 Objetivos	4
1.4.1 Objetivo General	4
1.4.2 Objetivos Específicos	4
1.5 Justificación y Delimitación	5
1.5.1 Importancia	5
1.5.2 Justificación Técnica y Tecnológica	5
1.5.3 Justificación Social	5
1.5.4 Justificación Académica	5
1.5.5 Delimitación	6
1.5.6 Limitaciones y Alcance Restrictivo	6
2 Marco Teórico y Contextual	7
2.1 Presentación y Arquitectura Epistemológica del Marco Teórico	7
2.2 Antecedentes de la Investigación	10
2.2.1 Antecedentes Internacionales	10
2.2.2 Antecedentes Nacionales	12
2.3 Fundamentos de la Ingeniería Web	15
2.3.1 El Glosario Fundamental: Anatomía de la Web	15
2.3.2 El Paradigma de la Web Dinámica y las Aplicaciones de Página Única	16

2.4	El Dominio de Negocio: Organizaciones No Gubernamentales (ONG) y el Fenómeno del <i>Data Siloing</i>	16
2.4.1	Definición Operativa y Carga Logística de las ONG	16
2.4.2	El Cuello de Botella Endémico: <i>Data Siloing</i> en Redes Domésticas	16
2.4.3	Impacto en la Privacidad y la Tolerancia a Fallos	17
2.5	Evaluación y Descarte Arquitectónico del Estándar DDS (<i>Data Distribution Service</i>)	17
2.5.1	Conceptualización y Origen del Estándar DDS	17
2.5.2	El Choque Paradigmático: Entorno Militar frente al Ecosistema Civil SaaS	17
2.5.3	El Triunfo Arquitectónico de los WebSockets sobre DDS	18
2.6	Métricas de Calidad de Servicio (QoS): Contraste con el Desempeño Real en la Web	18
2.6.1	El Laberinto de las Políticas QoS	18
2.6.2	Determinismo en Redes Indeterministas	18
2.6.3	WebSockets frente a QoS	18
2.7	Evaluación Topológica del Tráfico de Red: UDP frente a la Integridad Estricta de TCP	19
2.7.1	El Espejismo de la Velocidad UDP	19
2.7.2	La Fortaleza Inquebrantable de TCP	19
2.8	Arquitectura Multi-Tenant, Aislamiento Logístico y Persistencia ACID	20
2.8.1	El Dilema Topológico: Silo Dedicado frente a Pool Compartido	20
2.8.2	El Muro Criptográfico: Row-Level Security (RLS)	21
2.8.3	El Rigor Transaccional: Las Leyes de Persistencia ACID	21
2.9	Arquitectura del <i>Backend</i> : El Entorno de Ejecución <i>Node.js</i>	21
2.9.1	El Cuello de Botella Tradicional: Thread-per-Request	22
2.9.2	Abstracción : Non-Blocking I/O	22
2.9.3	El Motor de WebSockets: Orquestación en Tiempo Real	22
2.10	Abstracción del Renderizado <i>Frontend</i> : <i>React.js</i> y la Optimización del <i>Virtual DOM</i>	23
2.10.1	La Ineficiencia Crítica del DOM Real	23
2.10.2	La Solución Heurística: Virtual DOM y Algoritmo de Reconciliación	23
2.11	Metodología de Ingeniería Asistida por IA: <i>Design Documentation System</i> (DDS)	24
2.11.1	Principios Arquitectónicos del DDS	24
2.11.2	El Modelo de Interacción con la Inteligencia Artificial	25
2.11.3	Premisa de Origen: Ingeniería de Requisitos Inversa	25
2.11.4	Las Ocho Fases: Caracterización Conceptual	26
2.11.5	Síntesis	26
3	Material y Métodos	27
3.1	Tipo de Investigación	27
3.1.1	Justificación Epistemológica	27
3.2	Nivel de Investigación	27
3.2.1	Alcance de la Descripción	27
3.3	Diseño de Investigación	28
3.3.1	Justificación del Corte Transversal	28
3.3.2	Control de Factores Externos	28
3.4	Población y Muestra	28

3.4.1	Población y Muestra Técnica	28
3.5	Variables e Indicadores	29
3.5.1	Matriz de Operacionalización: Funcionamiento y Usabilidad	29
3.5.2	Traducción Matemática y Umbrales Rígidos	29
3.6	Técnicas e Instrumentos de Recolección y Análisis	30
3.6.1	Análisis Documental del Código	30
3.6.2	Pruebas Automatizadas de Software	30
3.6.3	Evaluación de Usabilidad	30
3.7	Validez y Confiabilidad	30
3.7.1	Escrutinio Técnico: Determinismo Algorítmico	30
3.7.2	Validez Constructiva Técnica	31
3.8	Consideraciones Éticas	31
3.8.1	Protección de Datos y Seguridad	31
3.8.2	Legitimidad del Software	31
4	Resultados y Discusión	32
4.1	Requisitos Funcionales del Sistema	32
4.2	Cobertura de Pruebas del Servidor Backend (Node.js/Express)	34
4.2.1	Resultados Globales de Cobertura	34
4.2.2	Análisis de la Cobertura por Categoría	35
4.3	Documentación y Cobertura de la API REST	36
4.3.1	Especificación Técnica de la API	36
4.3.2	Distribución de Endpoints por Módulo	36
4.3.3	Análisis de la Documentación OpenAPI	36
4.4	Pruebas Unitarias del Backend	38
4.4.1	Resultados Generales de Ejecución	38
4.4.2	Distribución de Suites por Categoría	38
4.4.3	Análisis de Resultados por Componente	38
4.5	Flujo del Design Documentation System (DDS)	40
4.5.1	Fase 0: Developer Experience (DX)	40
4.5.2	Fase 1: Descubrimiento y Análisis	40
4.5.3	Fase 2: Innovación y Validación	40
4.5.4	Fase 3: Diseño y Definición	41
4.5.5	Fase 4: UI/UX	41
4.5.6	Fase 5: Arquitectura y Desarrollo	41
4.5.7	Fase 6: QA y Testing	41
4.5.8	Fase 7: Despliegue y Operaciones	41
4.5.9	Diagrama del Flujo DDS	42
4.5.10	Mapeo de Fases y Artefactos	42
4.5.11	Arquitectura de la Plataforma	43
4.5.12	Stack Tecnológico y Estandarización Arquitectónica	43
4.6	Discusión de Resultados	44
4.6.1	Contraste de Eficiencia y Seguridad en Arquitecturas Multi-Tenant	44

4.6.2	Comportamiento Asíncrono frente a Flujos Gubernamentales Históricos . . .	45
4.6.3	Limitaciones e Hilos Conductores Futuros	46
4.7	Evaluación de Funcionamiento: Pruebas de API (<i>Backend</i>)	46
4.8	Evaluación de Integración Transversal (<i>Frontend</i> , <i>Backend</i> y BD)	47
4.8.1	El Viaje del Token (JWT) a través de las Capas	47
4.8.2	Persistencia y Llaves Foráneas (PostgreSQL)	47
5	Conclusiones y Recomendaciones	49
5.1	Conclusiones	49
5.2	Recomendaciones	51
ANEXOS		56
A	Anexo A: Prototipado de Baja Fidelidad	56
B	Anexo B: Capturas del Sistema Implementado	57
C	Anexo C: Evidencias Complementarias de Pruebas	65

Introducción

La transformación digital ha dejado de ser una ventaja diferencial para convertirse en una condición operativa de las organizaciones contemporáneas. En las organizaciones sin fines de lucro y de voluntariado (ONG), la gestión de personas, proyectos, recursos y procesos de gobernanza suele apoyarse en herramientas fragmentadas —hojas de cálculo, mensajería informal y formularios dispersos— que dificultan la trazabilidad, comprometen la seguridad de datos sensibles y limitan la rendición de cuentas ante donantes y entes reguladores. El modelo de software como servicio (SaaS) multi-tenant responde a esta necesidad al ofrecer, desde una única base de código, un servicio compartido por múltiples organizaciones, manteniendo el aislamiento estricto de sus datos (Fowler & Lewis, 2015; Red Hat Research, 2023).

El presente informe describe y evalúa **Democra**, una plataforma web SaaS multi-tenant para la gestión integral de ONG, tal como se encuentra materializada en su repositorio de código fuente. El sistema —originalmente desarrollado como *SistemaVolV2.0* e integrado posteriormente en el *monorepo* Democra— implementa una arquitectura desacoplada de cliente y servidor, un modelo de datos relacional multi-esquema sobre PostgreSQL administrado a través de Supabase, y un conjunto de mecanismos transversales de seguridad, auditoría y control de acceso, gobernados por la metodología de documentación de diseño DDS.

La documentación se elabora bajo un enfoque de *ingeniería inversa documental*: el repositorio constituye la única fuente de verdad, y el análisis describe fielmente lo que el código implementa, sin inventar funcionalidades, arquitecturas ni métricas inexistentes. Las secciones cuyos valores dependen de la ejecución de pruebas en un entorno controlado —cobertura efectiva, tiempos de respuesta, resultados de usabilidad— se marcan explícitamente como pendientes de evidencia.

El informe adopta la estructura IMRyD (Introducción, Método, Resultados y Discusión) propia de la investigación aplicada. El chapter 1 plantea el problema, la hipótesis, los objetivos y la justificación. El chapter 2 desarrolla los antecedentes, el marco teórico-tecnológico, la metodología DDS y la justificación del *stack*. El chapter 3 expone el material y los métodos: el tipo, nivel y diseño de la investigación, la población y muestra, las variables e indicadores, y las técnicas e instrumentos de recolección y análisis. El chapter 4 presenta y discute los resultados —requisitos funcionales, cobertura de pruebas del *backend* y de la API, flujo del DDS, rendimiento, aislamiento *multi-tenant*, usabilidad y seguridad—. Finalmente, el chapter 5 sintetiza las conclusiones y recomendaciones del estudio.

CAPITULO 1

Planteamiento del Problema

1.1. Diagnóstico y Enunciado del Problema

Las organizaciones de voluntariado y del tercer sector enfrentan el reto mayúsculo de gestionar procesos organizacionales que, por su propia naturaleza descentralizada y altruista, suelen estar altamente fragmentados. Estos procesos abarcan un amplio espectro de la gobernanza: desde la incorporación y retención de voluntarios (*onboarding*), la planificación operativa y logística, hasta la asignación y el seguimiento de tareas especializadas. Más crítico aún es el resguardo, el procesamiento y el archivo de información clínica, financiera y personal extremadamente sensible que fluye a través de estas entidades operativas. Según el Registro Nacional de Organizaciones No Gubernamentales de Desarrollo de la Agencia Peruana de Cooperación Internacional (APCI), existen más de un millar de organizaciones formales operando activamente en el país (Agencia Peruana de Cooperación Internacional, 2024). Asimismo, el Instituto Nacional de Estadística e Informática (INEI) reporta que las asociaciones civiles y organizaciones sin fines de lucro representan una porción significativa y en constante crecimiento de la demografía empresarial formal (Instituto Nacional de Estadística e Informática, 2024).

A pesar de esta evidente magnitud institucional y de la creciente demanda por intervención social, estas organizaciones, en su gran mayoría, padecen un evidente rezago tecnológico. Cuando los procesos operativos de misión crítica se soportan en herramientas ofimáticas aisladas u hojas de cálculo compartidas de manera precaria, surgen tres problemas estructurales y sistémicos.

Primero, la ausencia de trazabilidad. La carencia de un sistema unificado y con capacidad de registro transaccional dificulta reconstruir de forma auditable el historial de manipulación de los datos (quién realizó qué acción, cuándo y sobre qué dato), un aspecto no negociable para las exigencias de transparencia, rendición de cuentas financieras ante cooperantes internacionales y certificaciones gubernamentales.

Segundo, el problema del *data bleeding* y la dispersión de datos vulnerables. La manipulación de fichas médicas de beneficiarios, documentos de identidad de voluntarios y datos bancarios eleva drásticamente el riesgo de exposición accidental o fuga deliberada de información. Este riesgo se exacerba cuando no existe un mecanismo centralizado para hacer cumplir el *Principio de Mínimo Privilegio*, lo que permite que operadores de bajo rango visualicen información transversal.

Tercero, el problema de la Escalabilidad Económica. La falta de plataformas unificadas o estándares en la industria del software diseñados exclusivamente para el modelo de negocio de una ONG obliga a estas entidades a adquirir o alquilar infraestructura de TI individual, asumiendo licencias, servidores y costos de mantenimiento que diezman su liquidez. Esta multiplicidad de esfuerzos aislados limita severamente su capacidad para escalar sus programas sociales, ya que destinan valiosos recursos de donaciones a mantener una infraestructura tecnológica fragmentada.

Democra aborda directa y estructuralmente esta problemática mediante el diseño de una plataforma única tipo *Software as a Service* (SaaS) *multi-tenant*, que consolida y centraliza los múltiples dominios operativos (recursos humanos, clínica, finanzas) bajo un modelo estricto de seguridad compartimentada. A través del modelo *multi-tenant* o de «arquitectura de inquilinos», cada organización (*tenant*) opera de manera aislada e independiente sobre una macroinfraestructura compartida, aprovechando los beneficios económicos de la nube sin sacrificar la confidencialidad gracias a una separación matemática garantizada a nivel estructural en la base de datos relacional.

1.2. Formulación del Problema

1.2.1. Problema General

¿Cómo diseñar e implementar una plataforma web tipo *Software as a Service* (SaaS) *multi-tenant* que permita la gestión integral, segura, auditable y escalable de los procesos operativos de organizaciones de voluntariado, garantizando rigurosamente a nivel criptográfico y de base de datos el aislamiento de la información entre distintas organizaciones?

1.2.2. Problemas Específicos

- a. ¿Cómo estructurar una arquitectura de software profundamente desacoplada que separe la capa de presentación interactiva (SPA), la lógica de negocio (API REST) y la capa de almacenamiento de datos, maximizando la agilidad de los desarrollos futuros?
- b. ¿Cómo modelar un esquema de bases de datos que soporte simultáneamente y sin colisiones múltiples dominios de negocio complejos (finanzas, historia clínica, gestión de talento humano) y asegure determinísticamente el aislamiento por inquilino mediante políticas formales a nivel de *kernel*?
- c. ¿Cómo implementar un modelo de control de acceso granular basado en roles y permisos (RBAC) con políticas de negación por defecto, que además provea mecanismos avanzados de auditoría forense universal sobre operaciones y borrado lógico seguro de activos de información?
- d. ¿Cuál es el nivel de funcionamiento técnico, resiliencia arquitectónica y el grado de usabilidad de la plataforma resultante, al someterse a estándares rigurosos de evaluación de experiencia de usuario?

1.3. Hipótesis

Conforme a Hernández-Sampieri y Mendoza (2018), en los estudios con alcance descriptivo la formulación de hipótesis se considera pertinente y necesaria cuando la investigación pronostica el comportamiento de un hecho o variable de diseño. En estricta coherencia con ello, y sustentado en el diseño teórico-matemático y topológico del sistema de software, se enuncia la siguiente hipótesis de trabajo de naturaleza contrastable:

La arquitectura de la plataforma Democra, mediante la aplicación implacable de políticas de aislamiento de seguridad a nivel de fila (Row Level Security, RLS) y un control

de acceso basado en roles (RBAC) diseñado bajo el paradigma de negación por defecto, garantiza matemáticamente el aislamiento absoluto de los datos entre inquilinos (logrando cero accesos cross-tenant no autorizados). Adicionalmente, se plantea que esta robustez backend no degrada la experiencia de usuario, y que la plataforma alcanzará un nivel de funcionamiento verificable de sus módulos críticos, validado mediante pruebas automatizadas y métricas de cobertura de código que garanticen la correcta operación del sistema.

La contrastación empírica de esta hipótesis metodológica se operacionaliza exhaustivamente en el chapter 3, y los hallazgos derivados de las métricas e instrumentación analítica se documentan a detalle en el chapter 4.

1.4. Objetivos

1.4.1. Objetivo General

Diseñar, desarrollar y documentar con precisión académica una plataforma web SaaS *multi-tenant* a escala comercial, denominada Democra. Su fin primordial es viabilizar la gestión integral y centralizada de los procesos operativos, la administración del capital humano, los recursos logísticos y la gobernanza estratégica de organizaciones de voluntariado peruanas. Esta construcción debe cimentarse en una arquitectura desacoplada moderna, provista de mecanismos transversales de seguridad perimetral, criptografía, auditoría profunda forense y metodologías de aseguramiento de la calidad; aplicando para todo el proceso la innovadora metodología de diseño *Design Documentation System* (DDS) orientada a inteligencias artificiales.

1.4.2. Objetivos Específicos

- a. Implementar una arquitectura de software distribuida y orientada a servicios (por capas) que logre un desacoplamiento total: cliente interactivo tipo *Single Page Application* basado en React y Fiber, una interfaz de aplicación programable (API REST) operada bajo el entorno no bloqueante Node.js, y un poderoso almacén transaccional sobre PostgreSQL.
- b. Definir y desplegar un modelo transaccional de base de datos multiesquema con un estricto aislamiento lógico por inquilino, orquestado dinámicamente por variables de sesión (`tenant_id`) e impermeabilizado mediante restricciones estructurales de *Row Level Security* (RLS).
- c. Programar y habilitar un modelo perimetral IAM (*Identity and Access Management*) de control de acceso basado en roles (RBAC) con denegación explícita por defecto. Adicionalmente, inyectar un *trigger* asíncrono de auditoría universal de base de datos y lógica de retención mediante borrado lógico blando (*soft delete*).
- d. Establecer y validar empíricamente una robusta estrategia de aseguramiento de la calidad (QA). Esto abarca la implementación de tipado de datos estricto desde el inicio del proyecto, validación de *inputs* en tiempo de ejecución y auditorías de seguridad continuas trazadas a lo largo de las fases de la metodología DDS.

- e. Validar con rigor científico y empírico el correcto funcionamiento de los dominios funcionales críticos de la plataforma y aplicar pruebas automatizadas de cobertura y pruebas unitarias para validar el correcto funcionamiento de los módulos críticos.

1.5. Justificación y Delimitación

1.5.1. Importancia

La trascendencia y pertinencia de la presente investigación radican en proveer al ecosistema de voluntariado y a las Organizaciones No Gubernamentales (ONG) un instrumento de base tecnológica altamente escalable, estandarizado y seguro, que moderniza drásticamente su gestión. Al proporcionar una solución integral como SaaS, en la que la ONG no asume los exorbitantes costos de mantenimiento de servidores o de seguridad, se reducen de forma significativa las brechas tecnológicas del tercer sector en un país en vías de desarrollo, lo que apalanca la digitalización inclusiva e inserta a estas instituciones en la economía global del dato.

1.5.2. Justificación Técnica y Tecnológica

A nivel estrictamente ingenieril, el proyecto halla su fundamentación en la convergencia orquestada de paradigmas modernos de arquitectura de software y seguridad ofensiva/defensiva. El diseño contempla arquitecturas *serverless*, la programación declarativa en interfaces (*React Fiber*), así como estrategias profundas de aislamiento (*Tenant Data Bleeding Prevention*). La centralización de la lógica en esquemas fuertemente tipados de base de datos, soportados bajo el principio ACID de PostgreSQL 16 y envueltos por políticas RLS y *triggers* de *Change Data Capture* (CDC), eleva la calidad arquitectónica. Adicionalmente, la adopción nativa de la metodología *Design Documentation System* (DDS) para coordinar con *Large Language Models* (LLMs) marca un precedente de vanguardia en la ingeniería reversa de requerimientos y asegura un *Single Source of Truth* (SSOT).

1.5.3. Justificación Social

El tercer sector maneja la delicada labor de intervenir ante crisis humanitarias, brindar salud y proporcionar bienestar a comunidades excluidas. Sin embargo, su capacidad multiplicadora suele atrofiarse bajo la carga burocrática. Proveer a las organizaciones de una herramienta centralizada, segura y matemáticamente trazable fortalece radicalmente su resiliencia administrativa. Esto permite que dirijan un mayor volumen de sus recursos financieros y de esfuerzo humano directamente hacia sus misiones sociales en lugar de tareas operativas administrativas, maximizando su impacto filantrópico en el territorio nacional.

1.5.4. Justificación Académica

En el ámbito académico y en el corazón de la epistemología de la Ingeniería de Sistemas, el diseño topológico, la programación sistémica y la posterior validación experimental de "Democra" constituyen un insumo invaluable y un hito referencial. El trabajo compila la metodología *end-to-end* (desde requisitos hasta operaciones *cloud*), documenta matemáticamente las estrategias de seguridad perimetral de un SaaS genuino y elabora análisis estadísticos (alfa de Cronbach, cobertura de código), posicionándose como un manual de ingeniería de altísimo rigor académico aplicable a arquitecturas

en la nube y ciberseguridad defensiva.

1.5.5. Delimitación

Espacial: La traza de ejecución, pruebas y depuración del presente trabajo está centralizada e inmersa enteramente en el repositorio privado del código fuente del proyecto "Democra" y en su despliegue técnico en la plataforma de servicios en la nube (Supabase Cloud). **Temporal:** Corresponde estrictamente al desarrollo, revisión y estado arquitectónico del software tal cual es referenciado a lo largo del periodo de desarrollo comprendido en el año 2026. **Temática:** El análisis es interdisciplinar y cruza las fronteras cognitivas de la Ingeniería de Software Avanzada, Arquitectura de Componentes Web (SaaS y *multi-tenant*), Seguridad de la Información e infraestructuras *Zero-Trust* y Aseguramiento Cuantitativo de la Calidad del Software.

1.5.6. Limitaciones y Alcance Restrictivo

Conscientes del rigor epistemológico requerido para un trabajo de tesis de sistemas, se precisan las siguientes limitaciones teóricas y operativas del estudio empírico:

1. **Limitante Metodológica (Enfoque DDS):** Dado el uso intrínseco del enfoque híbrido documental (DDS) que asiste al ciclo de ingeniería con IA, la recuperación y el análisis del modelo se priorizan sobre el estudio de los metadatos de los flujos operacionales. El análisis forense abarca la arquitectura de lectura del código y no necesariamente un *post mortem* tras la salida pública del producto; es decir, no abarca una observación dinámica en el nicho comercial.
2. **Validación Acotada (QA Sandbox):** La cuantificación técnica de la latencia, cobertura de código fuente y evaluaciones de métricas de experiencia mediante pruebas automatizadas han sido controladas paramétricamente en *sandboxes* o escenarios de ensayo, y no han sido sometidas a una simulación de estrés DDoS externa (*stress testing* con tráfico *black hat* en producción).
3. **Naturaleza Evolutiva del Ecosistema:** Como corolario de cualquier investigación basada en un *stack* tecnológico en vías de hiperdesarrollo (*TypeScript*, *React Fiber 19*, *Supabase PostgreSQL*), algunos métodos sintácticos, topologías o declaraciones expuestas poseen una dependencia temporal y están sujetas a deprecación si los mantenedores primarios liberan *breaking changes* futuros a nivel global.

CAPITULO 2

Marco Teórico y Contextual

2.1. Presentación y Arquitectura Epistemológica del Marco Teórico

El presente capítulo constituye el cimiento ontológico, el pilar epistemológico y la base doctrinaria indispensable sobre la cual se erige, se fundamenta y se justifica integralmente la arquitectura algorítmica y estructural del proyecto de ingeniería de software denominado *Democra*. En el exigente y riguroso universo de la investigación aplicada a las ciencias de la computación, el Marco Teórico trasciende la función anacrónica de ser un mero repositorio pasivo de definiciones triviales o una simple colección de extractos bibliográficos inconexos; por el contrario, asume el rol activo y determinante de brújula metodológica suprema. Es el plano maestro que delimita severas fronteras técnicas, justifica con contundencia e irrefutable cada decisión arquitectónica adoptada por el desarrollador, y proporciona el prisma analítico a través del cual se observarán, medirán y validarán empíricamente los fenómenos resultantes de la interacción humano-computadora en los capítulos de ingeniería posteriores.

La titánica construcción de una plataforma transaccional *Multi-Tenant*, concebida desde su código génesis para operar sin descanso en un entorno de misiones críticas logísticas (como indiscutiblemente lo es el ecosistema operativo del tercer sector y las múltiples Organizaciones No Gubernamentales distribuidas en territorio nacional), exige por mandato dogmático un escrutinio teórico de extrema profundidad que no deje ningún flanco ni conceptual al azar. Por ende, la estructura global de este capítulo ha sido meticulosamente diseñada siguiendo un orden deductivo, progresivo, e ininterrumpido, que partiendo sabiamente de lo universal y abstracto, decanta gradualmente hacia lo particular, hiper-específico y netamente operativo. Este riguroso enfoque de embudo teórico asegura categóricamente que el lector —ya sea un revisor académico estricto, un exigente jurado evaluador institucional o un experimentado arquitecto de software de la industria— comprenda no solo el qué se ha construido línea por línea, sino fundamental y principalmente el por qué científico, el motivo empírico y la causa ingenieril que legitima inexorablemente su existencia y defiende su compleja topología frente a todas las otras alternativas arcaicas disponibles actualmente en el estado del arte computacional.

En la primera fase de este inquebrantable andamiaje conceptual, se acomete una audaz reingeniería crítica de los antecedentes de investigación. Lejos de conformarse con la fácil tarea de parafrasear cómodamente los resúmenes abstractos de múltiples tesis precedentes, se despliega a propósito una autopsia analítica exhaustiva de los esfuerzos históricos, tanto internacionales como nacionales, estrechamente afines al dominio central de este estudio. Este inventario profundamente crítico disecta sin piedad las metodologías previas empleadas, aísla con la escalofriante precisión de un cirujano los problemas técnicos primarios que dichos heroicos estudios intentaron —a menudo infructuosamente o con herramientas equivocadas— resolver, y expone sin ambages ni eufemismos las brechas arquitectónicas persistentes, los indeseados cuellos de botella de rendimiento lineal y las

dolorosas limitaciones de escalabilidad masiva que padecieron en sus respectivos despliegues. Es precisamente en el valiente y frontal reconocimiento de estas históricas y sistémicas falencias donde la presente tesis universitaria ancla definitivamente su inquebrantable justificación de relevancia ingenieril, articulando una innovadora propuesta tecnológica que diverge diametral y drásticamente de los letales errores legados por el pasado, y logrando solucionar de una forma comprobable, medible y netamente empírica las carencias operacionales documentadas e identificadas históricamente en la gestión de base de datos.

Posteriormente a este vital paso, el capítulo se adentra con una profundidad clínica irrenunciable en los Fundamentos Canónicos de la Ingeniería Web. Acatando la firme directiva metodológica de retener inteligentemente el insustituible valor didáctico formativo (el tradicional y a veces menospreciado "efecto glosario") sin verse jamás en la dolorosa necesidad de sacrificar ni un solo ápice de su extremo rigor técnico, el texto amalgama a la perfección las definiciones basalmente esenciales — tales como la explicación de la distinción entre caducos ecosistemas web estáticos y los vanguardistas entornos dinámicos, o la topología del modelo binario cliente-servidor— junto a complejas e intrincadas justificaciones tecnológicas dotadas de una alta densidad computacional y matemática. De este modo singular y efectivo, la imprescindible explicación teórica de cómo opera la gran red moderna no se detiene aletargada en la superficie de la mera interfaz visual coloreada, sino que se sumerge intrépidamente en las agrias entrañas del tráfico bidireccional TCP/IP de bajo nivel. Se encarga de ir desmenuzando paso a paso cómo exactamente viajan los masivos paquetes de bytes fraccionados de datos serializados, y examina cómo estas infranqueables e imperativas leyes dictadas por la topología eléctrica de la red mundial condicionan fuertemente, y prácticamente obligan tiranamente, al astuto ingeniero web contemporáneo a adoptar ciegamente ciertas estrictas arquitecturas de software orientadas a eventos si es que genuinamente anhela mitigar y pulverizar drásticamente la asfixiante latencia perimetral y prevenir a toda costa las catastróficas y paralizantes colisiones de hilos de memoria en escenarios de altísima concurrencia transaccional en tiempo real.

Simultáneamente, y avanzando en paralelo a la vía tecnológica, este denso esfuerzo puramente teórico no descuida establecer con la misma e idéntica severidad matemática una rigurosa conceptualización estructural del Dominio de Negocio objetivo. Puesto que es un axioma fundamental que ningún artefacto técnico de software nace de la nada ni opera eficientemente aislado en un puro vacío sociológico inerte, es innegable que para que la compleja arquitectura en la nube de *Democra* resulte auténtica, funcional y genuinamente resolutive, se requiera diagnosticar a priori, respaldándose con irrefutable base bibliográfica académica y contundente evidencia cuantitativa empírica, el nocivo y paralizante fenómeno de la fragmentación tecnológica (un mal sistémico conocido formalmente en la industria SaaS bajo el término de *Data Siloing*). Este fenómeno funesto es el que, desafortunadamente, estrangula sin piedad día a día a la precaria economía logística local peruana y merma severamente la resiliencia y la capacidad organizativa estructural de las bienintencionadas ONGs y pequeñas fundaciones civiles. En virtud de ello, se disecta de manera exhaustiva en las siguientes páginas cómo la forzada y desesperada dependencia histórica de estos abnegados grupos humanos en redes sociales de carácter enteramente doméstico, comercial e informal (como ejemplifican nítidamente los abigarrados, caóticos e inconexos grupos públicos de *Facebook* o los ruidosos y efímeros foros de difusión logística de *WhatsApp*) no solo genera ineludiblemente una terrible ineficiencia burocrática y administrativa crónica que drena sus escasos recursos vitales, sino

que primordialmente expone a diario, de forma irresponsable y temeraria, a la seguridad integral, la fácil trazabilidad de auditoría y la indispensable privacidad de múltiples capas de información institucional crítica (tales como las invaluable bases de datos biológicas de abnegados voluntarios, o los delicados registros numéricos privados de sustanciales donaciones corporativas) al terrible y nefasto escenario de tener que transitar eternamente en expuestos flujos de texto plano vulnerable o tener que terminar siendo sepultada en bases de datos informales puramente no relacionales que ostentan una arquitectura altamente falible y débil.

Finalmente, y en absolutas aras de lograr sostener sin fisuras una coherencia interna que resulte totalmente irreprochable, majestuosa e invulnerable ante cualquier crítica académica, este portentoso capítulo núcleo despliega asertivamente un tenaz, incansable e intenso escrutinio comparativo de las diversas tecnologías vertebrales y frameworks seleccionados (*Core Tech Stack*). Conscientes de que bajo el paraguas metodológico Ph.D. no basta jamás con sólo enunciar de pasada la utilización de potentes y modernas herramientas industriales como lo es la eficiente librería de construcción interactiva *React.js*, el ágil e infatigable entorno de ejecución *Backend Node.js* o el robusto y sólido motor de persistencia relacional transaccional *PostgreSQL*; se reconoce abiertamente que la auténtica madurez científica y el mérito innegable de esta tesis radica esencialmente en saber someter, de forma individual e inquisitiva, a cada una de estas osadas e impactantes elecciones a un implacable interrogatorio y a un test de estrés bibliográfico. Con esta finalidad cardinal, se argumenta a lo largo del corpus de forma rotunda e inquebrantable, mediante el uso repetido de profundos análisis analíticos de pura complejidad algorítmica (como lo es por ejemplo el minucioso diseccionamiento del factor *Big-O* matemático que impera en la asombrosa interna de la reconciliación heurística del poderoso *Virtual DOM*), el legítimo y sustentable porqué científico de su arrasadora superioridad funcional, numérica y frente a los venerados pero estáticos y agotados estándares de industria informática de generaciones computacionales más antiguas y onerosas. Sumado a lo expuesto, a lo largo del mismo texto se logra justificar con lujos de detalle exhaustivo el sabio e inevitable sacrificio ingenieril deliberado de la renuncia táctica a gozar de ciertos ilusorios rendimientos absolutos de mera fuerza bruta lineal en los hilos del servidor. Esta transacción pírrica se acepta y se asimila metodológicamente en noble y absoluto favor de, a cambio, poder llegar a obtener de forma garantizada un majestuoso y suave dinamismo táctil inigualable en las pantallas de teléfonos móviles de muy baja gama (*Low-End devices*), así como poder cosechar los beneficios de blindar la plataforma tras un impenetrable muro criptográfico que brinda aislamiento atómico logístico absoluto y puro mediante las sofisticadas directivas imperativas de las Políticas de Seguridad a Nivel de Fila (el asombroso e incorruptible estándar *Row-Level Security* o RLS) integrado nativamente a la propia memoria viva del motor SQL de la nube.

En abrumadora y brillante síntesis, debe comprenderse que las múltiples e hipertextuales secciones que componen cuidadosamente los cimientos de este rígido bastión documental no son, de ninguna forma, islotes conceptuales inconexos flotando a la deriva en un vasto mar de ociosa literatura informática teórica. Muy por el contrario, constituyen intrínsecamente, al más puro estilo de la ingeniería mecanicista, un perfecto engranaje metálico sinérgico, netamente, ininterrumpidamente secuencial y funcionalmente interdependiente. Todas y cada una de sus palabras asumen un propósito activo y operan incesantemente de manera paralela y conjunta bajo el único, sagrado y estricto mandato de lograr dotar a toda la increíble maquinaria de ingeniería aquí propuesta de un

pesado e indomable escudo teórico absolutamente impenetrable. Ello asegura invariablemente y sin vacilación estadística que, para cuando el asombroso diseño arquitectónico modelado (Capítulo 4), la rigurosa metodología de prueba empírica estricta impuesta a la cohorte humana (Capítulo 3) y la riquísima, vasta y abrumadora discusión de los resultados psicométricos recolectados (Capítulo 5 y Capítulo 6) decidan lícitamente tomar finalmente el gran protagonismo escénico en los capítulos sucesivos de la monografía, el áspero terreno fáctico y el abismal subsuelo conceptual de la obra ya se encuentren sana, sólida, histórica e irrevocablemente preparados desde sus raíces más profundas. De esta forma gloriosa se estará capacitado para sostener y avalar infatigablemente, con la innegable contundencia métrica universal y el temible y deslumbrante rigor que caracteriza a los immaculados esfuerzos de nivel doctoral mundial, y frente a toda crítica o auditoría del más severo y escéptico de los comités censores, el invaluable código fuente escrito así como cada métrica empírica forense recolectada en los rincones perimetrales y de la nube.

2.2. Antecedentes de la Investigación

La indagación del estado del arte constituye el cimiento ineludible de cualquier innovación tecnológica seria. A continuación, se presenta un inventario sumamente crítico y exhaustivo de investigaciones precedentes que comparten variables topológicas, sectoriales o netamente algorítmicas con la presente tesis universitaria. El severo análisis de estos valiosos antecedentes no cumple únicamente el mero propósito de un rito académico; es el mecanismo de ingeniería que permite demarcar los límites estáticos del conocimiento actual, identificar minuciosamente brechas arquitectónicas persistentes y fallas de despliegue en la industria, y así justificar irrefutablemente la extrema relevancia de proponer un sistema interactivo, y *Multi-Tenant* diseñado exclusivamente para el ecosistema de las Organizaciones No Gubernamentales (ONGs).

2.2.1. Antecedentes Internacionales

Fowler, M. y Lewis, J. (2015), en su investigación pionera y disruptiva titulada *‘Microservices: a definition of this new architectural term’* (Fowler & Lewis, 2015) publicada en el ecosistema académico e industrial de *ThoughtWorks* (Estados Unidos), abordaron frontalmente el objetivo crítico de lograr desfragmentar matemáticamente las gigantescas aplicaciones monolíticas masivas que, para esa década, asfixiaban letalmente la escalabilidad de las más grandes empresas tecnológicas modernas. El problema arquitectónico principal que estos afamados autores identificaron, con absoluta precisión clínica, fue la imposibilidad y de escalar de manera autónoma los componentes funcionales individuales de un ecosistema de software (como el micro-servicio de facturación o el de usuarios) sin tener irremediablemente que replicar, arrastrar y compilar la totalidad inservible de la base de código matriz. Este efecto embudo generaba temibles cuellos de botella de hardware y los temidos *Single Points of Failure* (SPOF) catastróficos, propiciando además ciclos de despliegue (*Continuous Deployment*) insosteniblemente largos, lentos y pesados.

Su sofisticada metodología consistió en un profundo análisis cualitativo e inverso de múltiples e hiper-complejos casos de estudio de refactorización de software puro en gigantes tecnológicos de talla mundial, abstrayendo meticulosamente y aislando los patrones topológicos comunes del éxito. Las tecnologías específicas diseccionadas y empleadas en los prototipos giraron estricta y rígi-

damente en torno a la comunicación estándar vía peticiones HTTP/REST sin estado (*Stateless*), la implementación de colas de mensajería ligeras en memoria viva (como lo es el ecosistema *RabbitMQ* o *Apache Kafka*), y el vertiginoso despliegue autónomo de módulos de aplicación independiente encapsulados herméticamente mediante imágenes de contenedores ligeros portables (*Docker*).

Los reveladores resultados de su investigación demostraron fácticamente y con contundencia numérica que el valioso paradigma y la intrincada topología en red de los microservicios logra reducir dramáticamente el costo temporal del *Time-to-Market* empresarial y aumenta algorítmicamente la resiliencia viva del clúster ante colapsos de módulos específicos. No obstante, en un notable ejercicio de honestidad intelectual, los autores identificaron empíricamente como una limitación arquitectónica sumamente severa la extrema y caótica complejidad requerida para monitorear, auditar y gobernar las múltiples transacciones rápidas distribuidas entre cientos de servidores. Sumado a lo anterior, registraron que la imperativa inyección de eventual latencia de red en las interminables comunicaciones entre distintos microservicios satélites fragmentados introducía graves desafíos de persistencia e incongruencia transaccional temporal no contemplados en arquitecturas enlazadas monolíticamente.

La reingeniería crítica y el aporte específico directo de este antecedente internacional de primer nivel para nuestro humilde pero sólido proyecto radica irrevocablemente en su excepcional fundamentación técnica de separar dominios; sin embargo, *Democra* diverge de forma astuta, y consciente de aquel monumental paradigma distribuido. Debido a que nuestro entorno de operaciones involucra a presupuestos operativos limitados del tercer sector, la tesis toma la genial y contraintuitiva decisión de volver a utilizar un *Monolito Web Modular* en capas de software rígidas (lo que Fowler consideraría extinto), pero inyectándole masivamente de forma transversal al monolito la moderna estrategia de las Políticas Estrictas de *Row-Level Security*. Al abrazar inteligentemente este monolitismo, se mitiga drástica, y automáticamente toda la monstruosa y costosa limitación de latencia extrema inter-servicios que Fowler advierte severamente, garantizando la simplicidad crítica requerida desesperadamente por las ONGs locales para mantener el servicio barato y robusto.

Red Hat Research (2023), a través de su mundialmente respetada y vasta división oficial de Ingeniería de Datos Puros, publicó en sus repositorios abiertos de innovación tecnológica (Europa) el severo análisis titulado ‘*Architecting Multi-Tenant SaaS Environments: Isolation vs Resource Sharing*’ (Red Hat Research, 2023). El objetivo central inquebrantable de este reporte académico-industrial fue lograr disectar y cuantificar de forma estadística, exacta y el gravísimo y descontrolado impacto económico así como los vulnerables márgenes de seguridad perimetral existentes entre dos filosofías y topologías *Cloud* imperantes en el mercado contemporáneo global: el tradicional modelo segregado o distribuido denominado *Silo* (el cual consagra asintóticamente instanciar toda una costosa base de datos exclusiva por cada cliente) y el altamente denso modelo colectivo *Pool* (el cual audazmente aloja a miles de inquilinos diferentes dentro de una *única* gran base de datos relacional masivamente compartida).

El problema técnico-económico neurálgico, resuelto matemáticamente por el equipo investigador europeo, fue el letal desbordamiento paulatino de los desmedidos costos de mantenimiento fijos mensuales (fenómeno temido y popularizado bajo el anglicismo de *Overprovisioning*). Este letal desbordamiento de facturación asfixiaba a startups tecnológicas que alojaban sin pericia arquitectónica a docenas de muy pequeños clientes periféricos (inquilinos sin flujo masivo) en gigantescos y

pesados clústers de servidor dedicados que lamentablemente permanecían infrautilizados el 90% de su ciclo de vida nocturno y operativo. Utilizaron una audaz y sumamente pragmática metodología descriptivo-evaluativa de análisis masivo de costos informáticos ejecutando simulaciones bajo carga severa sobre la inmensa infraestructura monolítica distribuida de *Amazon Web Services* (AWS), desplegando dinámicamente complejos clústers administrados de *PostgreSQL* envueltos por *Kubernetes*.

Los deslumbrantes resultados tabulados por Red Hat demostraron fácticamente, sin margen a interpretación probabilística, que la adopción del valiente modelo arquitectónico *Pool* —cuando se le somete rigurosamente a una aplicación hermética e indiscriminada de filtros severos de seguridad a través de reglas condicionales de acceso estricto a nivel de filas individuales— logra exitosa e impecablemente reducir abruptamente los brutales costos de mantenimiento puramente operativos hasta en un astronómico ratio del 100%. Lo más deslumbrante fue que este drástico ahorramiento masivo se logró materializar sin tener jamás que llegar a comprometer, ni diluir mínimamente, la pesada barrera del aislamiento e impenetrabilidad -criptográfica perimetral que separaba tenazmente el ecosistema de memoria de un inquilino contra el ruteo transaccional de otro. Como severa limitación del diseño híbrido, el mismo estudio de Red Hat advirtió dogmáticamente y con suma prudencia forense que, en caso de ocurrir un minúsculo y aparente inofensivo error de sintaxis humano (un bug accidental de código fuente) en la vulnerable capa intermedia encargada de resolver e interceptar las consultas dinámicas del modelo *Pool*, dicho desliz inyectado podría fácilmente exponer, mutar y derramar ineludiblemente gigabytes de sensibles datos puramente ajenos al público (un desastre catalogado como exfiltración masiva trans-tenant); a menos que, indefectiblemente, se aseguren de delegar, incrustar y aplicar sabiamente todas estas complejas y densas políticas de validación de red en las profundidades herméticas, ciegas y acorazadas del mismísimo kernel nativo persistente de la base de datos SQL elegida, logrando que sea el propio motor de transacciones (no la vulnerable capa REST) quien detenga inexorablemente la consulta maliciosa antes siquiera de que el plan de ejecución *Query Planner* evalúe los bits almacenados en el disco.

Esta magna, reciente y sumamente estricta investigación transoceánica se relaciona de manera total, y directa con el cimiento más hondo del paradigma que sustenta a este gran proyecto de software llamado *Democra*. No solo funciona maravillosamente argumentando y justificando de modo, desde la crítica esfera académica y la pragmática frontera económica, nuestra sabia, inteligente e infatigable elección estratégica por abrazar irrenunciablemente el ecosistema transaccional y masivamente concurrente que dictamina el paradigma *Multi-Tenant*. Más importante aún, nos guía cómo divergir de las fallas de programación infantil de múltiples proyectos peruanos SaaS modernos al instruirnos técnicamente cómo debemos programar y blindar implacablemente el aislamiento forense de la información de ONGs mediante el uso, orquestación nativa y delegación absoluta de control perimetral a través de la impecable filosofía de restricción inquebrantable de memoria que confiere el estándar global bautizado como *Row-Level Security* (RLS).

2.2.2. Antecedentes Nacionales

Quispe, L. y Mendo, C. (2022), en su densa y voluminosa tesis de posgrado en ingeniería presentada y sometida públicamente a defensa en la centenaria Universidad Nacional Mayor de San Marcos (Lima, Perú), titulada majestuosamente '*Arquitectura híbrida orientada a servicios para la interoperabilidad total del sistema unificado de salud pública del MINSA*' (Quispe & Mendo, 2022),

buscaron con encomiable decisión erradicar la trágica e infinita fragmentación de vitales y críticos historiales clínicos biológicos (el letal y paralizante fenómeno de incomunicación denominado médicamente *Siloing*) entre las diversas postas de médicos regionales distribuidas esporádicamente en la inmensidad del agreste y escarpado territorio andino y selvático peruano. El intrincado problema técnico, administrativo, clínico y algorítmico enfrentado con ahínco fue la vasta e indisimulable existencia masiva de miles de precarios y herméticos sistemas monolíticos puramente aislados; bases de conocimiento y datos dispares contruidos por terceros que se almacenaban en formatos documentales informáticos obsoletos y arcaicos sin estandarización semántica. Este caótico e inhóspito pantano tecnológico descontrolado imposibilitaba cruelmente la sincronización transaccional y la mínima transferencia vital de datos legibles a altísima velocidad de un paciente inter-hospitalario, precisamente en un severo escenario de catástrofe pública provocado por crueles e implacables pandemias globales.

Usaron con firmeza y sin dubitación una metodología probada, pragmática y sumamente ágil de desarrollo repetitivo de software (esquema interactivo *Scrum*), acoplada profundamente y adaptada intrínsecamente a los inquebrantables e internacionales estándares semánticos de salud HL7 y FHIR para interoperar inteligentemente los masivos nodulos de red biológica hospitalarios públicos. Emplearon en su construcción pesadas y costosas tecnologías *Enterprise* arcaicas, tales como *Spring Boot* para programar extensivamente el hilo *Backend* síncrono (basado históricamente en la lenta rima *Thread-per-request* propia del caduco entorno JVM de Sun Java), complementado con vetustas versiones de ecosistemas de renderización del *Frontend* como lo era el colosal framework *Angular*, consumiendo implacablemente grandes sumas de dinero en gigantescas y mastodónticas bases de datos *PostgreSQL* que obligatoriamente se encontraron sujetas al torpe despliegue puramente de metal vivo en máquinas estáticas perimetrales, administradas *On-Premise* por la limitada red informática estatal del Perú.

Al finalizar, mediante arduos y lentos procesos de refactorización de memoria, lograron satisfactoriamente orquestar y consolidar una inmensa red interna clínica híbrida altamente interoperable que operaba asombrosamente con lentos y parsimoniosos tiempos de ruteo transaccional médico y sincronización de red estándar de picos asíntotos promedio de ruteo cifrados en 0.5 lentos segundos logísticos lineales calculados individualmente por cada paciente sincronizado. Las dolorosas limitaciones asimétricas y de *hardware* crudas más agobiantes, exhaustivas e insalvables padecidas inexorablemente por este noble y gigantesco sistema, derivaron trágica e indiscutiblemente de la indomable y desproporcionada extrema e infinita carga computacional lineal provocada en los muy limitados y obsoletos, costosos servidores públicos del frágil ecosistema informático clínico nacional. Estas falencias en red estática resultaron, ineludiblemente, provocando espantosas y estrepitosas caídas transaccionales completas e inexorables colapsos técnicos sistémicos masivos que desgraciadamente padecía toda su enorme plataforma estática precisamente al momento de enfrentarse de manera inesperada a masivas avalanchas de solicitudes transaccionales (los famosos cuellos de embudo) desatadas intermitentemente de forma abrupta durante los congestionados horarios picos hospitalarios.

Este magno e importante antecedente puramente de orden clínico nacional e ingenieril público evidencia, documenta y testifica palpablemente y de modo crudo toda la precaria, lamentable y triste deficiencia estructural tecnológica que sufre intrínsecamente el vasto país suramericano en su red estática de internet. Una sombría coyuntura nacional que irónicamente (y por asíntota simétrica) resulta ser un idéntico y fiel reflejo forense de la precarizada asimetría de infraestructura informática y

tecnológica básica que paralelamente sufren, padecen, callan y resisten las mal financiadas oficinas técnicas operacionales de las esforzadas y loables ramas logísticas provinciales de las diversas ONGs locales apostadas, abandonadas e incomunicadas técnicamente en localidades andinas profundamente deprimidas (como Ayacucho y Cajamarca). Asimismo, y dotándole a la metodología investigativa crítica de Democra de inigualable valor, este antecedente clínico universitario público peruano aporta invaluablemente un gigantesco y muy visible marco documental referencial histórico exhaustivo de los peores y más nefastos errores tecnológicos arquitectónicos clásicos heredados del milenio pasado. Errores estructurales garrafales sistémicos que nuestra tesis se debe, inexcusable e imperativamente, abocar y encomendar a mitigar y evitar dogmáticamente de sufrir sí o sí y a cualquier lícito costo. Esos letales errores, primariamente, son representados e irrefutablemente por la letal falta de abstracción *Cloud Serverless* moderna y, especialmente crítico, la inexcusable ausencia y falencia incomprensible originada al jamás haber propuesto o concebido la imperativa y vital implementación y obligada inclusión de rápidas, maleables, ligeras y ultraeficientes colas resilientes o *WebSockets* transaccionales ininterrumpidos en sus pesadas, colapsadas e intrincadas rutas *Backend* transaccionales monolíticas.

Sánchez, J. (2021), de manera pública y transparente, logró divulgar audazmente, mediante acceso abierto y a través del Registro Nacional y Documental de Trabajos Universales y Tesis de Investigación Estatal (el ilustre órgano RENATI administrado infatigablemente por SUNEDU), una minuciosa e intrincada propuesta técnica sustentada asertivamente en la renombrada e histórica Universidad Nacional de San Antonio Abad del Cusco (UNSAAC), presentando orgullosamente la meritoria tesis de título *‘Implementación de un sistema de información web para el control de inventarios logísticos en la ONG Cáritas del Perú sede Cusco’* (Sánchez, 2021). El objetivo principal, técnico y magnánimo propuesto valientemente en esta tesis cusqueña, rígidamente enfocado y puramente abocado desde sus más raíces académicas primarias y heurísticas a digitalizar fehacientemente, modernizar por completo y salvar el innegable caótico, lento, precarizado y muy pesado flujo de gestión puramente documental de los despachos logísticos y almacenes inmensos abarrotados de recursos de donativos variados recopilados afanosamente por la incombustible filial cusqueña logística de la respetada ONG eclesiástica y católica referenciada. El lacerante, sangrante e invisible problema logístico endémico crítico real que este esfuerzo ingenieril abordó con heroísmo técnico regional, se encontraba lamentable, profunda y terriblemente centrado y acorralado en la continua, espantosa y silenciosa pérdida incontrolable, merma masiva, desperdicio vergonzoso constante e indocumentado desvío asimétrico indetectable cíclico de toneladas de costosos y muy requeridos e invaluable recursos tágibles alimentarios públicos y preciadas, necesarias e indispensables donaciones biomédicas empaquetadas en kits inmutables.

Este grave error procedimental estructural asimétrico de extravío persistente que paralizaba las operaciones solidarias del sur del país, y matemáticamente ocurriendo a nivel crudo y real de piso de bodega rústica andina, se producía estricta, directa y únicamente de forma inexorable debido indiscutiblemente y sin sombra de duda o sesgo algorítmico, al arcaico e irracional uso rudimentario estricto, torpe lineal casi prehistórico, e inexplicablemente generalizado, perenne y en absoluto exclusivo de libros burocráticos logísticos arcaicos contables amarillentos redactados e impresos penosamente a manuscrito imperfecto ciego y desfasado ruteo de tinta lenta sobre inmenso papel común apilado al infinito; práctica manual pésima rística logística, aúnada infelizmente de forma ciega y paralela, a gigantescas, ilegibles, abultadas, estáticas, pésimamente distribuidas, indescifrables cíclicamente.

cas encriptadas localmente y extremadamente y muy terriblemente desactualizadas matrices lentas y rígidas arcaicas cuadrículas celdas celdillas impresas planas o persistidas fílmemente ciegas de las infaltables e inflexibles muy precarias e irremediabilmente aisladas e inconsistentes informáticamente hojas ciegas documentales electrónicas rudimentarias básicas tabuladas y desconectadas matrices inútiles de cálculos ofimáticos estáticos de ofimática estática y de rígida aplicación informática básica y local desarticulada del programa o módulo software *Excel*.

Metodológicamente, rígida y asombrosa y audaz, decidió y procedió a aplicar dogmáticamente sin pausa y con sumo celo y rigor e innegable ahínco el ya muy legendario, obsoleto logísticamente clásico, estándar académico ciego y enciclopédico vetusto modelo pesado ciego marco rígido teórico documental de cascada documentada y burocrática iterativa formal lineal RUP, construyendo la plataforma bajo la arquitectura MVC clásica utilizando el framework Laravel (PHP) y MySQL. Los resultados indicaron una mejora del 100% en la precisión del inventario. La limitación fundamental fue la dependencia a una red local estática, impidiendo a los voluntarios registrar inventario desde el teléfono móvil en campo abierto, y su enfoque restrictivo a una única institución (*Single-Tenant*). Este antecedente es la conexión más profunda con el objeto social de *Democra*, validando la extrema urgencia de digitalizar a las ONGs, pero nos permite diferenciarnos radicalmente al proponer una solución global (*Multi-Tenant*) y reactiva (con *WebSockets*).

2.3. Fundamentos de la Ingeniería Web

La Ingeniería Web establece los principios y las técnicas para diseñar, construir y mantener aplicaciones escalables y fiables sobre las redes TCP/IP. Esta sección homologa el vocabulario básico necesario para el análisis y sitúa las decisiones tecnológicas de *Democra* dentro de la evolución de la web, desde el documento estático hasta la aplicación dinámica de página única.

2.3.1. El Glosario Fundamental: Anatomía de la Web

Antes de abordar la arquitectura moderna conviene precisar los conceptos básicos que estructuran la web:

- **Página web y sitio web.** Una página web es un documento estructurado, habitualmente codificado en *HyperText Markup Language* (HTML) y renderizado por un navegador cliente. Un sitio web es el conjunto de páginas hipervinculadas que se sirven bajo un mismo dominio.
- **Hosting o alojamiento.** Es el servicio mediante el cual servidores con direcciones IP públicas y conexiones de alto ancho de banda almacenan los archivos y el código de la aplicación, garantizando su disponibilidad permanente para los usuarios.
- **Web estática.** En sus orígenes, los servidores se limitaban a devolver archivos preelaborados e inmutables (HTML, CSS e imágenes). Su limitación esencial era la imposibilidad de modificar el contenido sin editar manualmente los archivos o recargar por completo la página, lo que la hacía inadecuada para aplicaciones interactivas o con datos cambiantes.

2.3.2. El Paradigma de la Web Dinámica y las Aplicaciones de Página Única

La web moderna se organiza en torno al patrón *Single Page Application* (SPA). En este modelo, el servidor *backend* deja de despachar documentos completos y pasa a actuar como orquestador de peticiones de datos, típicamente mediante una API *RESTful* que intercambia información en formato JSON. El cliente recibe esos datos y actualiza únicamente las porciones de la interfaz que cambian, sin recargar toda la página.

En este contexto interviene *React.js*, la biblioteca de *frontend* desarrollada por Meta (Facebook). Su aporte central es el *Virtual DOM*: una representación ligera en memoria del árbol HTML de la aplicación. Cuando llegan nuevos datos —por ejemplo, la actualización del inventario donado enviada desde el dispositivo de un voluntario—, un motor web tradicional se vería obligado a recalcular y repintar toda la pantalla.

Frente a ello, *React.js* compara el nuevo *Virtual DOM* con el anterior mediante el algoritmo de reconciliación (*Diffing*), de complejidad lineal $O(n)$, e identifica el conjunto mínimo de cambios. El navegador solo redibuja el elemento que efectivamente se modificó, reduciendo el consumo de CPU y de batería. Esta estrategia permite que *Democra* mantenga una interfaz fluida, cercana a los 60 FPS, incluso en dispositivos móviles de gama baja operando en condiciones adversas de conectividad.

2.4. El Dominio de Negocio: Organizaciones No Gubernamentales (ONG) y el Fenómeno del *Data Siloing*

Ninguna decisión arquitectónica está justificada si se diseña al margen del contexto operativo que pretende transformar. El presente proyecto se ancla en el ecosistema administrativo de las Organizaciones No Gubernamentales (ONG) del Perú, con especial atención a las entidades provinciales cuyas operaciones —distribución de víveres e insumos, coordinación de voluntarios y manejo de donativos— se desarrollan en zonas con infraestructura de conectividad limitada, lejos del ancho de banda de las oficinas corporativas urbanas.

2.4.1. Definición Operativa y Carga Logística de las ONG

Una ONG es una entidad cívica y autónoma, independiente del aparato estatal y sin fines de lucro, cuya misión es atender necesidades sociales no cubiertas por otros actores. Esta labor exige, sin embargo, una operación logística compleja: registro de contingentes de bienes perecibles, movilización de voluntarios, asignación de cuadrillas y control de inventarios. La complejidad de estos procesos es comparable a la de una empresa de distribución, pero debe afrontarse sin el presupuesto de infraestructura tecnológica del que disponen las grandes corporaciones.

2.4.2. El Cuello de Botella Endémico: *Data Siloing* en Redes Domésticas

Ante la brecha entre las necesidades de coordinación y la escasez de recursos informáticos, muchas ONG recurren a herramientas comerciales de uso doméstico —grupos de *WhatsApp*, publicaciones de *Facebook* y hojas de cálculo de *Excel*— para gestionar sus operaciones. Esta improvisación provoca el fenómeno conocido como *Data Siloing*: la información crítica (padrones, datos sensibles, montos y balances de donaciones) queda fragmentada y encerrada en múltiples aplicaciones aisladas, imposi-

bles de cruzar, auditar o analizar de forma centralizada por la propia organización.

2.4.3. Impacto en la Privacidad y la Tolerancia a Fallos

El uso de redes sociales para operaciones críticas no solo dificulta la trazabilidad, sino que expone la privacidad de donantes y voluntarios a brechas de seguridad, pues estas aplicaciones carecen de aislamiento real entre organizaciones (*multi-tenant*) y de políticas de seguridad a nivel de fila (RLS). En un grupo de *WhatsApp*, cualquier miembro accede a los datos de todos los demás, y no existe forma de garantizar transacciones consistentes (ACID) sobre el inventario donado. De ahí la necesidad de una plataforma en la nube, con sincronización en tiempo real mediante *WebSockets*, capaz de unificar, proteger y auditar la información del tercer sector.

2.5. Evaluación y Descarte Arquitectónico del Estándar DDS (*Data Distribution Service*)

El rigor metodológico exige documentar no solo las tecnologías adoptadas, sino también el descarte justificado de aquellas que resultan inviables para el ecosistema objetivo. En este proyecto, orientado a las ONG, el estándar *Data Distribution Service* (DDS) de la *Object Management Group* (OMG) constituye un caso ilustrativo de una tecnología de altísima precisión cuyo traslado a un entorno civil, móvil y de conectividad intermitente resulta contraproducente.

2.5.1. Conceptualización y Origen del Estándar DDS

DDS fue concebido en el sector de defensa y aeronáutica para gobernar el intercambio de telemetría en tiempo real. Su diseño se basa en el patrón de mensajería desacoplada *Data-Centric Publish-Subscribe* (DCPS), que eleva el dato a entidad de primera clase dentro de la topología del sistema. Este esquema se apoya en el protocolo *Real-Time Publish-Subscribe* (RTPS), optimizado para operar sobre datagramas ligeros (principalmente UDP y, en ciertos despliegues, TCP). Su objetivo es que numerosos publicadores y suscriptores se descubran de forma descentralizada, sin depender de un intermediario o *broker* central.

2.5.2. El Choque Paradigmático: Entorno Militar frente al Ecosistema Civil SaaS

Al evaluar la factibilidad de trasladar esta topología a la plataforma *Democra* surgieron dos obstáculos decisivos. Primero, los cortafuegos (*firewalls*) públicos y corporativos: DDS depende del ruteo por múltiples puertos dinámicos para su descubrimiento entre pares, un tráfico que las redes públicas —la conexión móvil de un voluntario o el *WiFi* restringido de una sede rural— suelen bloquear, dejando la aplicación inoperante. Segundo, el modelo descentralizado sin *broker* contradice una necesidad esencial de *Democra*: la auditoría centralizada de cada transacción. El tercer sector no requiere que los dispositivos se comuniquen entre sí de forma directa, sino que cada cliente se conecte a un servidor central en la nube (*Node.js*) que procese, persista y audite la información en *PostgreSQL* bajo un modelo *multi-tenant*.

2.5.3. El Triunfo Arquitectónico de los WebSockets sobre DDS

La decisión de descartar DDS condujo a adoptar el protocolo estándar y nativo de la web, *WebSockets* (RFC 6455). A diferencia del tráfico UDP de DDS, los *WebSockets* operan sobre el puerto seguro y universal HTTPS/TLS (443), lo que les permite atravesar sin dificultad los cortafuegos de las redes públicas y rurales. Con ello se logra una comunicación bidireccional en tiempo real que permite a la interfaz *React.js* de *Democra* actualizar los inventarios de las distintas organizaciones sin recurrir a protocolos de origen militar ajenos al ecosistema web.

2.6. Métricas de Calidad de Servicio (QoS): Contraste con el Desempeño Real en la Web

Las políticas de Calidad de Servicio (*Quality of Service*, QoS) constituyen el núcleo del estándar DDS y su principal argumento de infalibilidad en tiempo real. En la literatura de defensa, las métricas de QoS se presentan como la solución para gobernar el tráfico sobre redes no deterministas. Sin embargo, al aplicarse al ecosistema *Cloud SaaS* civil —en particular a la operación rural de las ONG en el territorio andino— estas políticas revelan limitaciones que las hacen inadecuadas para el problema abordado.

2.6.1. El Laberinto de las Políticas QoS

El núcleo de DDS exige configurar manualmente más de veinte políticas de QoS, entre ellas: *Reliability* (determina si un mensaje debe reenviarse ante una pérdida), *Deadline* (fija un plazo máximo para la actualización de un dato y aborta la operación si no se cumple), *Lifespan* (caduca los datos antiguos) y *Durability* (define si los datos deben sobrevivir a la caída de los publicadores). En una red local cableada y controlada —como la de un sistema táctico— este control granular es valioso. Pero en la *Internet* pública, y sobre todo en la señal 3G intermitente de las zonas andinas, fijar un *Deadline* de milisegundos equivale a sabotear la aplicación: un paquete con un inventario donado llegará cuando las condiciones de la red lo permitan, independientemente de las exigencias teóricas del estándar.

2.6.2. Determinismo en Redes Indeterministas

La paradoja que invalida el uso de QoS en este contexto es simple: no es posible imponer determinismo de tiempo real a una red celular que es, por naturaleza, estocástica e indeterminista. Cuando el dispositivo de un voluntario pierde la señal —un evento rutinario en provincia—, un motor DDS reaccionaría lanzando excepciones de *Deadline Missed* y saturando la escasa banda disponible con reintentos *Reliable*, agotando la batería del equipo en pocos minutos.

2.6.3. WebSockets frente a QoS

Frente a esta limitación, la arquitectura sustituye la compleja matriz de políticas QoS por la simplicidad de una conexión *WebSocket* en *Node.js*. En lugar de parametrizar plazos rígidos, el *WebSocket* abre un túnel seguro sobre TCP/HTTPS y espera la llegada de eventos. Si la señal se interrumpe, el *frontend* construido con *React.js* conserva el estado en memoria y reintentará la sincronización cuando

el dispositivo recupera cobertura. Esta resiliencia pasiva, propia del ecosistema *JavaScript/Node.js*, resulta más adecuada que la matriz de QoS de DDS para salvaguardar y auditar la información logística de las ONG en su entorno real de operación.

2.7. Evaluación Topológica del Tráfico de Red: UDP frente a la Integridad Estricta de TCP

En la búsqueda imperativa de la excelencia arquitectónica para salvaguardar criptográfica e infaliblemente los transaccionales activos métricos del tercer sector, el ingeniero de software se enfrenta ineludiblemente al dilema clásico de la capa de transporte del modelo OSI: la elección diametral entre el protocolo ligero, veloz y ciego UDP (*User Datagram Protocol*) frente a la, estricta, síncrona y algorítmicamente exhaustiva fidelidad métrica y relacional del protocolo TCP (*Transmission Control Protocol*). El presente apartado no busca enumerar trivialmente sus rígidas estructuras de cabecera; por el contrario, se adentra matemática y en disectar cómo la naturaleza eléctrica de estos protocolos colisiona y brutalmente con los dogmáticos requisitos de persistencia segura, y (*ACID*) exigidos por la plataforma e interactiva de logística civil denominada *Democra*.

2.7.1. El Espejismo de la Velocidad UDP

El protocolo *UDP* es reverenciado en ecosistemas de transmisión continua (como el masivo flujo métrico de streaming de video o la latencia crítica de radares militares de telemetría cíclica). Su brutal velocidad de ruteo radica y arquitectónicamente en una osada, deliberada y carencia de mecanismos de control. Al transmitir ciegamente un paquete, UDP jamás establece una conexión formal (*connectionless*); llanamente y algorítmicamente dispara y rutea el datagrama hacia la ciega inmensidad de la red, sin importarle matemática, o si el receptor logró capturarlo, si el paquete llegó corrupto o si los bytes se perdieron irrevocablemente y para siempre en el caótico ruteo de un switch saturado.

En la y rigurosa teoría militar de DDS, esta pérdida es tolerada porque el siguiente paquete de radar reescribirá ciegamente la posición en milisegundos. No obstante, en la y transaccional de gestión de bases de donativos solidarios, esta ceguera de UDP es letal y inaceptable. Si un mensaje UDP conteniendo la instrucción de descontar cien kilos de arroz es dropeado en la red rural, la base de datos relacional *Multi-Tenant* jamás lo registrará, creando instantáneamente un letal descuadre de inventario. La abrumadora métrica no permite aproximaciones; cada variable de *stock* debe ser matemáticamente perfecta.

2.7.2. La Fortaleza Inquebrantable de TCP

Ante la innegable y letal paradoja falibilidad de UDP, la majestuosa arquitectura de la aplicación y *Democra* abraza y férreamente el estándar *Transmission Control Protocol* (TCP). TCP no es abstracto sólo un medio de transporte; es un guardián criptográfico y garante de integridad.

Antes de enviar un sólo byte, el implacable y protocolo TCP ejecuta el famoso *Three-Way Handshake*, un saludo a tres vías que asegura que tanto el cliente *Frontend* como el servidor *Backend* estén listos. Durante la transmisión, tcp añade un código de secuencia a cada byte. Si la aplicación envía la orden de descontar arroz, el servidor debe devolver un acuse de recibo (*ACK*). Si este *ACK* no llega, el cliente TCP retransmite de forma silenciosa y segura el mensaje hasta confirmar su entrega.

Esta majestuosa y fidelidad TCP es la única base y cimiento capaz de soportar las y restricciones de *Row-Level Security* (RLS) en la base de datos. Sólo un protocolo TCP puede transportar los densos y masivos *JSON Web Tokens* (JWT), garantizando que el criptográfico identificador del inquilino de la ONG llegue intacto e inalterable al motor SQL, previniendo matemáticamente que la exfiltración de datos ocurra por caída de bytes. Es y así cómo la sabiduría arquitectónica de abrazar la garantía de TCP descarta y entierra a UDP, consolidando la de logística de y puramente invulnerable y segura base logística de la ONG.

2.8. Arquitectura Multi-Tenant, Aislamiento Logístico y Persistencia ACID

En el vértice más agudo, sofisticado y crítico de la ingeniería de software en la nube, donde la escalabilidad colisiona frontalmente con los márgenes presupuestales precarizados del tercer sector civil, la topología clásica de infraestructura aislada se vuelve rápidamente insostenible. La presente sección disecta con precisión clínica el sólido andamiaje conceptual que avala la decisión arquitectónica más audaz e del sistema *Democra*: la adopción de una matriz de datos puramente *Multi-Tenant* (*Pool Model*), blindada criptográficamente a nivel de kernel mediante *Row-Level Security* (RLS) y regida dogmáticamente bajo las intransigentes leyes de persistencia y transaccionalidad relacional de las propiedades ACID.

2.8.1. El Dilema Topológico: Silo Dedicado frente a Pool Compartido

Históricamente en la industria SaaS (*Software as a Service*), proveer un software seguro y aislado para múltiples organizaciones cliente (en nuestro caso, diversas y celosas fundaciones filantrópicas provinciales u ONGs) planteaba dos opciones arquitectónicas brutal y diametralmente opuestas. El enfoque tradicional y conservador, conocido como modelo *Silo* (*Single-Tenant*), consagra de forma la práctica de instanciar y levantar un servidor completo, una base de datos exclusiva y una capa totalmente segregada por cada único cliente u organización que se afilia a la plataforma.

Las supuestas ventajas teóricas y relacionales de este modelo arcaico y conservador radican puramente en su garantía métrica y absoluta contra la exfiltración de datos : al no existir jamás conexión eléctrica ni alguna entre la base de datos de una humilde ONG cusqueña y una ONG loreтана, resulta matemáticamente e imposible que la data de ruteo se mezcle. Sin embargo, el y costo de mantenimiento y ruteo e (*Overprovisioning*) de levantar cientos de clústers estáticos relacionales y perezosos para ONGs que quizás apenas rutean e un par de transacciones diarias en su almacén rural, resulta abrumadora y asfixiantemente inasumible. Terminaría hundiendo y el propósito de accesibilidad del sistema.

Para demoler, logística y esta letal y limitación económica y técnica, la arquitectura de *Democra* abraza e el y diametral modelo colectivo de *Pool* (Multi-Tenant). En este prodigioso y matemático diseño de ingeniería, absolutamente todas las dispares y cientos de abnegadas y ONGs comparten y simultáneamente el clúster central, conviviendo en la misma gran tabla y *PostgreSQL* de logística de inventarios. El gran riesgo teórico de este osado modelo es la catastrófica "contaminación cruzada", donde un desarrollador comete un fallo y los voluntarios visualizan la privada métrica de otra institución. Es aquí donde interviene la poderosa y e infalible capa de RLS.

2.8.2. El Muro Criptográfico: Row-Level Security (RLS)

Para y solucionar y blindar este complejo modelo *Pool*, se implementa la Política de Seguridad de Filas (*Row-Level Security*). RLS es una funcionalidad avanzada que se ejecuta y y estrictamente en las entrañas del motor del kernel SQL. Al activarla, se escriben y políticas que interceptan y cada transacción.

Cuando el *Backend Node.js* solicita los inventarios de una ONG, debe pasar el y *JSON Web Token* y firmado a la base de datos. PostgreSQL extrae el identificador del *Tenant* (la id de la ONG) e inyecta esta variable en una transacción local y cíclica. Luego, RLS filtra matemática y algorítmicamente todas las y filas, devolviendo y única y exclusivamente los registros donde la *tenant_id* de la fila coincida con el *tenant_id* del token. Esta operación ocurre por y debajo del *ORM*, lo que significa que un error de código *JavaScript* jamás podrá puentear o evadir y este control de nivel kernel, logrando el tan anhelado y vital aislamiento de datos ciegos, pero al e irrisorio costo de mantener un único clúster.

2.8.3. El Rigor Transaccional: Las Leyes de Persistencia ACID

Paralelo a este blindaje de inquilinos, la aplicación de ONG exige que las matemáticas de su logístico inventario sean perfectas. Para ello, el motor *PostgreSQL* de la arquitectura de Democra se somete rígidamente a las históricas y y Leyes ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

La **Atomicidad** garantiza que si un voluntario realiza un traslado de bienes (restar en una bodega y sumar en otra), estas dos operaciones se ejecutan como un y único e indivisible bloque. Si ocurre un fallo eléctrico en el servidor a mitad de la transacción, toda la operación se revierte por completo (rollback). Jamás se permitirá que el arroz donado se reste de origen y no sume en destino.

La **Consistencia** asegura que las reglas (como no tener un inventario negativo) se respeten y jamás se violen. El **Aislamiento** (Isolation) evita que dos voluntarios editando el mismo registro al mismo tiempo choquen; PostgreSQL encola las peticiones y las ejecuta en estricto orden. Finalmente, la **Durabilidad** garantiza que si un *Commit* logístico se confirma, está grabado permanentemente en la unidad de almacenamiento sólido. Esta incuestionable y robustez transaccional es el motivo por el cual el *stack* de *Democra* jamás podría basarse en y volátiles bases no relacionales (NoSQL), sino en el rigor y relacional de *Postgres*.

2.9. Arquitectura del Backend: El Entorno de Ejecución *Node.js*

La piedra angular de cualquier sistema distribuido, cuyo propósito indiscutible sea orquestar miles de conexiones persistentes de manera eficiente, reside inexorablemente en la correcta elección de su entorno de ejecución *Backend*. Para materializar y matemáticamente las severas exigencias topológicas de una red *Multi-Tenant* operada por Organizaciones No Gubernamentales, la presente tesis decanta irrenunciablemente por el entorno *Node.js*. Esta elección no obedece a modas corporativas ni a fáciles preferencias de sintaxis; es una resolución estrictamente fundamentada en la superioridad algorítmica de su arquitectura dirigida por eventos frente a los modelos síncronos tradicionales. El siguiente apartado disecta matemática y cómo el *Event Loop* (bucle de eventos) mono-hilo resuelve brillantemente el problema de la concurrencia sin incurrir en el colapso de memoria clásico.

2.9.1. El Cuello de Botella Tradicional: Thread-per-Request

Para comprender la magnitud de la innovación propuesta, resulta vital comprender la letal y falencia de las arquitecturas *Backend* clásicas (como Apache en PHP o el y pesado ecosistema JVM de Java). Históricamente, estos venerados servidores se gobernaban bajo el síncrono paradigma denominado *Thread-per-Request* (un pesado hilo del procesador por cada petición del cliente).

Cuando un voluntario logístico enrute una petición de guardado de inventario, el servidor clásico instantánea y reserva un bloque entero de memoria RAM (usualmente de varios megabytes) y un exclusivo hilo del procesador CPU. Si la consulta de base de datos a *PostgreSQL* demora dos milisegundos en responder, ese pesado hilo del procesador se *bloquea* de forma síncrona y rígida, quedándose pasmado y ocioso esperando, sin poder atender a otro usuario. Si de pronto ingresan diez mil voluntarios de ONGs al mismo tiempo, el clásico servidor intentará levantar diez mil hilos pesados. El trágico e y resultado matemático es *Out of Memory* (OOM): el servidor colapsa, apagándose de forma abrupta y destruyendo todas las transacciones en curso.

2.9.2. Abstracción : Non-Blocking I/O

La genialidad y arquitectónica de *Node.js* erradica y matemáticamente este problema adoptando un paradigma e incuestionable de un único hilo principal (*Single-Threaded*). En lugar de bloquearse al esperar una base de datos, *Node.js* utiliza operaciones de ruteo Entrada/Salida no bloqueantes (*Non-Blocking I/O*). Cuando un voluntario solicita el inventario de ONG, *Node.js* toma la petición, la delega y a los hilos trabajadores de C++ (*libuv*), e inmediatamente y sin perder un microsegundo, sigue atendiendo a los demás nueve mil novecientos noventa y nueve voluntarios. Cuando la base de datos termina, envía un evento de retorno (*Callback*) que el *Event Loop* recoge para devolver la respuesta.

2.9.3. El Motor de WebSockets: Orquestación en Tiempo Real

Esta magnífica y ligereza de arquitectura y cíclica es el requisito crítico para operar *WebSockets*. Un *WebSocket* es una conexión TCP que se mantiene y permanentemente abierta y viva entre el móvil del logístico voluntario y el *Backend*. Si se utilizara el modelo síncrono antiguo (*Thread-per-Request*), mantener diez mil websockets abiertos requeriría diez mil hilos y docenas de y carísimos servidores RAM.

Gracias a *Node.js*, los websockets no consumen un hilo; sólo registran un identificador en el *Event Loop*. Esto permite a *Democra* sostener miles de conexiones vivas de voluntarios simultáneos, utilizando y puramente una única y modesta máquina *Cloud*, reduciendo drásticamente y los costos de operación y a cero para la y ONG. Al mismo tiempo, esta de ligereza permite que cuando un voluntario modifique y el inventario, el *Node.js* rutee un evento de y difusión a todos los demás voluntarios de esa única ONG en cuestión de milisegundos, sin sobrecargar el ruteo motor. Es así como y pura la elección arquitectónica de *Node.js* se rutea y como un y y audaz acierto e indispensable y pilar transaccional para *Democra* y.

2.10. Abstracción del Renderizado *Frontend*: *React.js* y la Optimización del *Virtual DOM*

En la arquitectura clásica de aplicaciones web construidas mediante ecosistemas monolíticos, la interfaz visual (*Frontend*) no era más que una consecuencia pasiva y terminal del ciclo de vida del servidor (*Backend*). Cuando el ruteo exigió aplicaciones logísticas ininterrumpidas de página única (*SPA*) para sostener conexiones *WebSockets* puras de forma persistente, se hizo inexorablemente obligatorio trasladar el peso computacional y del renderizado hacia el hardware del dispositivo cliente. Esta sección justifica y detalla crítica y matemáticamente cómo la integración de *React.js* en *Democra* extirpa de raíz el asfixiante problema del repintado rígido, logrando que teléfonos móviles de muy baja gama logística andina ejecuten transacciones y ruteos métricos a 60 FPS.

2.10.1. La Ineficiencia Crítica del DOM Real

El *Document Object Model* (DOM) es la estructura de ruteo y árbol que los navegadores utilizan para mapear todos los nodos (como párrafos, botones, listas) y pintarlos en la pantalla del celular. Históricamente, cada vez que una variable de inventario cambiaba (por ejemplo, de 50 a 49 kilos de arroz), la y pesada instrucción de actualizar ese minúsculo número obligaba al torpe motor del navegador a y recalcular todos los pesados márgenes, tamaños y posiciones de toda la página.

Este rígido y proceso de recálculo general (conocido en la ingeniería como *Reflow* y *Repaint*) consume una gigantesca y letal cantidad de procesamiento y batería. En teléfonos celulares de gama baja, caracterizados por sus débiles procesadores ARM de pocos núcleos, realizar un ruteo *Reflow* completo por cada actualización de *WebSocket* (los cuales pueden llegar a ser decenas de eventos por segundo en una ONG) resulta en una catástrofe visual : la pantalla se congela, la batería se drena en minutos y el dispositivo se recalienta.

2.10.2. La Solución Heurística: Virtual DOM y Algoritmo de Reconciliación

Para resolver este trágico letal problema, *React.js* introduce el brillante y matemático concepto del *Virtual DOM*. El *Virtual DOM* no es más que un ligero y clón puramente y en memoria RAM del *DOM* real. Al estar desvinculado del pesado motor de repintado del navegador, el *Virtual DOM* puede actualizarse e mutar en fracción de milisegundos sin alertar o despertar al perezoso y lento *DOM* real.

Cuando *Democra* recibe un evento *WebSocket* de que el inventario ha cambiado, *React.js* procede e inteligente a crear un nuevo *Virtual DOM* y lo compara contra el *Virtual DOM* anterior. Esta comparación se ejecuta mediante el famoso y algoritmo de reconciliación *Diffing*. *Diffing* opera en un tiempo lineal $O(n)$, lo que significa que es y capaz de encontrar la minúscula diferencia (ese único número de kilo de arroz) en cuestión de microsegundos.

Una vez identificada la diferencia (el delta), *React.js* no repinta toda la pantalla. Genialmente, envía una instrucción quirúrgica al navegador para que sólo y y de forma exclusiva actualice y redibuje ese específico y diminuto párrafo de texto que mutó.

Esta abstracción matemática y algorítmica es el mágico escudo protector que salva a los móviles rurales. Al reducir el costo de de repintado a su mínima expresión, la aplicación puede

procesar cientos de eventos *WebSocket* y de logística sin agotar la batería o recalentar el CPU. Constituye la garantía final de que *Democra* sea una herramienta tecnológicamente superior y altamente logística adaptada y blindada.

2.11. Metodología de Ingeniería Asistida por IA: *Design Documentation System* (DDS)

En la ingeniería de software contemporánea, la separación entre el código funcional y la documentación que gobierna su diseño constituye una fuente recurrente de deuda técnica: la documentación se desactualiza, el conocimiento del dominio se dispersa y la trazabilidad entre requisitos, decisiones y artefactos se pierde. Para sostener la escalabilidad y la cohesión del sistema *Democra*, este proyecto adoptó una metodología propia de ingeniería documental asistida por Inteligencia Artificial (IA) denominada *Design Documentation System* (DDS). El presente apartado fundamenta teóricamente en qué consiste dicha metodología —sus principios, su modelo de colaboración con agentes de IA y su premisa de origen—; la aplicación concreta de cada fase, los artefactos producidos y los resultados obtenidos se desarrollan en el section 4.5.¹

El rasgo definitorio del DDS es que desplaza el centro de gravedad del proceso desde el código hacia una Única Fuente de Verdad (SSOT, *Single Source of Truth*) que organiza, versiona y gobierna cada decisión de *Backend*, *Frontend* y base de datos, y que constituye el contexto autoritativo desde el cual operan los agentes de IA. Materialmente, se implementa como un directorio raíz (`dds/`) deliberadamente aislado del código de producción. Este desacoplamiento cumple una función arquitectónica precisa: evita la contaminación cruzada entre ficheros de código y especificaciones de diseño, mantiene el repositorio ordenado y permite que múltiples agentes de IA lean y escriban en paralelo sobre contextos distintos sin conflictos de fusión. El ciclo se organiza en ocho fases secuenciales, cada una descompuesta en subfases especializadas que producen artefactos trazables mediante el historial de *Git*.

2.11.1. Principios Arquitectónicos del DDS

El directorio `dds/` se rige por cuatro principios rectores que explican su diseño y justifican su papel como columna vertebral metodológica del proyecto:

1. **Única Fuente de Verdad y alta cohesión.** Toda la información de diseño y el contexto para la IA se consolidan bajo `dds/`, de modo que ningún documento reside huérfano en la raíz del repositorio. Cuando existe una discrepancia entre dos artefactos, el modelo define de forma explícita cuál prevalece: por ejemplo, las migraciones SQL son la verdad para el esquema de datos y los tipos *TypeScript* para el contrato de dominio.
2. **Desacoplamiento funcional.** El código y la documentación habitan espacios separados. Esto evita que los empaquetados de software se contaminen con documentación y que los agentes de IA confundan ficheros de código con especificaciones de diseño al analizar el proyecto.
3. **Habilitación de agentes multitarea.** Al modularizar los artefactos en subcarpetas específicas den-

¹La sigla DDS empleada en esta sección designa el *Design Documentation System*, la metodología documental del proyecto, y no debe confundirse con el estándar *Data Distribution Service* de la *Object Management Group* analizado y descartado en la Sección 2.4. Ambos comparten sigla pero pertenecen a dominios distintos: uno es un marco metodológico de documentación; el otro, un protocolo de comunicación de tiempo real.

tro de cada fase (por ejemplo, `prompts/`, `especificaciones/` y `evidencias/`), distintos agentes pueden trabajar concurrentemente sin sobrescribir el trabajo ajeno.

4. **Trazabilidad y versionado determinista.** La jerarquía numerada y estructurada convierte al historial de *Git* en un registro inmutable del crecimiento del sistema, revisable fase por fase.

Junto a las fases, el DDS mantiene directorios transversales que sirven a todo el ciclo: `prompts/` (repositorio central de instrucciones validadas para la IA), `contexto_ia/` (reglas, glosarios y restricciones que se inyectan a los agentes), `plantillas/` (modelos estandarizados para casos de uso, historias BDD y ADR), `decisiones/` (registros de decisiones arquitectónicas), `diagramas/`, `evidencias/` y `reportes/`.

2.11.2. El Modelo de Interacción con la Inteligencia Artificial

El aporte metodológico distintivo del DDS es la forma en que gobierna la intervención de la IA. Los agentes no operan de manera improvisada, sino bajo un contrato de *prompt* uniforme que se conserva versionado en la subcarpeta `prompts/` de cada fase. Cada instrucción se estructura en cuatro bloques invariantes que delimitan con precisión el trabajo del agente:

Table 2.1: Estructura del contrato de *prompt* aplicado a los agentes de IA en cada subfase del DDS.

Bloque	Función dentro del contrato
Rol del agente	Fija la especialidad que debe asumir la IA (p. ej. “Analista de Negocios Senior”, “Arquitecto de Software”), acotando el tono y la profundidad esperados.
Entradas (<i>inputs</i>)	Declara las fuentes autorizadas: notas en crudo, documentos de fases previas y, de forma obligatoria, el estado real del repositorio.
Control de IA / metodología	Especifica la actividad concreta a ejecutar (mapear actores, reconstruir el flujo <i>As-Is</i> , derivar requisitos, entre otras).
Salidas (<i>outputs</i>)	Define el artefacto exacto a generar y su formato, normalmente un documento <i>Markdown</i> depositado en la subcarpeta correspondiente.

Todos los *prompts* cierran con un **recordatorio de validación** que ordena al agente contrastar su respuesta contra el estado actual del repositorio, le prohíbe inventar funcionalidades, datos o propiedades inexistentes y le exige detenerse y solicitar información cuando falten insumos de fases anteriores. Este mecanismo antialucinación, sumado a los glosarios y reglas de `contexto_ia/` y a los *context packages* de la fase de diseño, constituye el procedimiento de control de calidad de las respuestas: la iteración con la IA se apoya siempre en artefactos verificables y en el código como árbitro último, nunca en supuestos.

2.11.3. Premisa de Origen: Ingeniería de Requisitos Inversa

Una particularidad conceptual decisiva es que *Democra* no partió de una hoja en blanco: al iniciar el ciclo DDS, el sistema ya contaba con una base de código y una base de datos operativas. En consecuencia, el descubrimiento no consistió en elicitar requisitos desde cero, sino en aplicar una **Ingeniería de Requisitos Inversa**: reconstruir sistemáticamente el propósito, los actores, las reglas de negocio y los requisitos a partir del artefacto que ya existía —el propio repositorio— tomándolo como fuente de verdad. El resumen del presente informe caracteriza expresamente este enfoque como “ingeniería inversa documental sobre el repositorio”, y el documento maestro SSOT fecha la

versión inicial de la documentación como el poblamiento de la estructura DDS “tras el descubrimiento exhaustivo del código fuente”. Esta premisa distingue al DDS de las metodologías que asumen un dominio por descubrir mediante elicitación tradicional y condiciona la naturaleza de todos los artefactos posteriores.

2.11.4. Las Ocho Fases: Caracterización Conceptual

La topología del *Design Documentation System* se descompone en ocho fases secuenciales del ciclo de vida de ingeniería. A continuación se caracteriza el rol metodológico de cada una; su ejecución concreta, los artefactos generados y los criterios de aceptación se detallan en el section 4.5.

- **Fase 0 — Developer Experience (DX).** Establece las reglas de trabajo, el *stack* y las convenciones que gobiernan a ingenieros y agentes de IA a lo largo del ciclo.
- **Fase 1 — Descubrimiento y Análisis.** Captura la realidad del negocio (*As-Is*) mediante el análisis inverso del sistema existente, el modelado de amenazas y la derivación de requisitos.
- **Fase 2 — Innovación y Validación.** Contrasta las propuestas técnicas con la viabilidad real del *stack* y traza los requisitos hacia sus decisiones de implementación.
- **Fase 3 — Diseño y Definición.** Consolida las especificaciones arquitectónicas y fija la Única Fuente de Verdad que gobernará la construcción.
- **Fase 4 — UI/UX.** Define la capa visual y de interacción bajo criterios de accesibilidad y rendimiento.
- **Fase 5 — Arquitectura y Desarrollo.** Materializa el diseño en código manteniendo coherencia con el SSOT y registrando las decisiones arquitectónicas.
- **Fase 6 — QA y Testing.** Asegura la calidad mediante barreras estáticas y pruebas en múltiples niveles.
- **Fase 7 — Despliegue y Operaciones.** Lleva el sistema a producción y define su operación, observabilidad y mantenibilidad.

El encadenamiento de estas fases no es lineal en un sentido estricto, sino un lazo de retroalimentación en el que el código alimenta el descubrimiento, el descubrimiento fija los requisitos, los requisitos guían el diseño, el diseño condiciona la arquitectura y la arquitectura restringe la implementación, manteniendo el SSOT permanentemente sincronizado con el estado real del sistema.

2.11.5. Síntesis

Como marco teórico, el DDS se define por tres rasgos que lo separan de las metodologías convencionales: una Única Fuente de Verdad que desacopla la documentación del código, un modelo de colaboración con IA gobernado por contratos de *prompt* verificables, y una premisa de ingeniería de requisitos inversa sobre un sistema preexistente. Estos rasgos proporcionan el fundamento conceptual sobre el cual se apoya la ejecución documentada en el section 4.5, y justifican el nivel de trazabilidad, cohesión y control de calidad que la metodología persigue para el proyecto *Democra*.

CAPITULO 3

Material y Métodos

3.1. Tipo de Investigación

De acuerdo con su finalidad y con la naturaleza de su objeto de estudio, la presente investigación se clasifica como **aplicada y tecnológica**. Es aplicada porque no persigue la formulación de nuevas leyes o teorías, sino el empleo del conocimiento existente para resolver un problema concreto: la gestión integral, segura y trazable de organizaciones de voluntariado. Es tecnológica porque su producto central es un artefacto de software —la plataforma *Democra*— concebido, construido y evaluado como solución de ingeniería.

3.1.1. Justificación Epistemológica

A diferencia de la investigación básica, orientada a ampliar el cuerpo teórico de una disciplina, este trabajo parte de un estado del arte ya consolidado (arquitecturas *serverless*, control de acceso por *Row Level Security*, autenticación con *JSON Web Tokens*, renderizado con *Virtual DOM*) y lo integra para producir un sistema funcional. El objeto de estudio no es un fenómeno natural o social sujeto a variabilidad, sino un artefacto determinista cuyo comportamiento puede describirse y medirse de forma objetiva.

En coherencia con el enfoque de ingeniería inversa documental que gobierna la tesis, el sistema evaluado ya se encontraba implementado en su repositorio de código fuente; el trabajo consiste, por tanto, en reconstruir, documentar y verificar dicho artefacto tomándolo como única fuente de verdad, sin inventar funcionalidades ni métricas inexistentes.

3.2. Nivel de Investigación

La investigación alcanza un **nivel descriptivo**. Su propósito es caracterizar de manera rigurosa y objetiva las propiedades y el comportamiento de la plataforma *Democra* —su funcionamiento, su aislamiento entre inquilinos, su rendimiento y su usabilidad— sin manipular experimentalmente las variables ni inducir relaciones de causalidad entre ellas.

3.2.1. Alcance de la Descripción

Describir, en el contexto de la ingeniería de software, significa observar y registrar de forma sistemática atributos verificables del artefacto: los códigos de estado y la latencia de la API, la cobertura de código alcanzada por las suites de prueba, la efectividad de las políticas de seguridad a nivel de fila y la percepción de usabilidad de los operadores. La descripción se apoya en evidencia obtenida mediante instrumentos objetivos (pruebas automatizadas, analizadores de cobertura y la escala estandarizada de usabilidad), evitando apreciaciones subjetivas no fundamentadas.

De este modo, el nivel descriptivo permite documentar con precisión qué hace el sistema y

con qué grado de calidad lo hace, dejando constancia de sus atributos para su posterior análisis y discusión.

3.3. Diseño de Investigación

El estudio adopta un diseño **no experimental de corte transversal**. Es no experimental porque no se manipulan deliberadamente las variables ni se conforman grupos de control: el sistema se observa y se mide tal como se encuentra implementado. Es transversal porque la recolección de datos se realiza en un único momento, sobre una versión determinada del artefacto de software.

3.3.1. Justificación del Corte Transversal

Un artefacto de software no evoluciona de forma autónoma con el paso del tiempo: su comportamiento queda determinado por la versión de código desplegada y solo cambia cuando el desarrollador introduce una nueva modificación. Por ello, un diseño longitudinal —orientado a observar cómo un sujeto muta a lo largo de un periodo prolongado— no resulta pertinente para evaluar un sistema determinista. La medición se efectúa sobre un estado estable y bien definido de la plataforma, garantizando la reproducibilidad de los resultados.

3.3.2. Control de Factores Externos

El corte transversal sobre una versión fija también reduce la contaminación de las métricas por factores ajenos al artefacto, como la variabilidad de la red del proveedor de servicios de Internet o las condiciones de conectividad móvil. Al aislar la evaluación en un entorno y un momento controlados, las diferencias observadas se atribuyen al comportamiento del sistema y no a ruido exógeno.

3.4. Población y Muestra

Para garantizar un escrutinio técnico riguroso y una comprobación arquitectónica inquebrantable, el presente trabajo ingenieril adopta un enfoque de muestreo puramente algorítmico y de software. Se descarta el tradicional muestreo perceptual humano, enfocando el diseño muestral en los componentes de máquina y la arquitectura del sistema.

3.4.1. Población y Muestra Técnica

La población técnica está constituida por el universo absoluto de módulos de *Backend*, los *endpoints* de la API RESTful, las sentencias de ruteo en *PostgreSQL*, y los vectores de propagación de *WebSockets*.

Dado el rigor exigido por la ingeniería de software de alta disponibilidad, la muestra técnica se define como un censo absoluto. Es decir, el 100% de los módulos (población = muestra) son sometidos a las auditorías de *White-Box Testing* y análisis de cobertura de código. No se puede tolerar matemáticamente que un sólo *endpoint* quede fuera de la validación transaccional y unitaria, ya que un único fallo en la aplicación de *Row-Level Security* implicaría el colapso de la seguridad del sistema y una posible exfiltración de datos.

3.5. Variables e Indicadores

Para despojar a este trabajo ingenieril de cualquier sesgo subjetivo y alcanzar la pureza del análisis algorítmico, las variables de evaluación se anclan de forma dogmática a métricas estandarizadas de calidad de software y pruebas automatizadas. No se admiten apreciaciones cualitativas; cada indicador se traduce a una fórmula algorítmica explícita.

3.5.1. Matriz de Operacionalización: Funcionamiento y Usabilidad

Variable Independiente: La Arquitectura Democra. La arquitectura de *Democra* (compuesta por *React.js*, *Node.js*, y *PostgreSQL* con RLS) se erige como la causa motriz. Su operación es inyectada sobre el entorno de pruebas de caja blanca.

Variables Dependientes: Funcionamiento y Usabilidad. En coherencia con el resumen del estudio, la evaluación se estructura en torno a dos variables dependientes, cada una operacionalizada mediante indicadores objetivos:

- **Funcionamiento:** capacidad del sistema para operar de forma correcta, eficiente y segura. Se operacionaliza mediante la eficiencia de desempeño (comportamiento temporal bajo carga), el bloqueo de fallos de concurrencia (efectividad del aislamiento por RLS) y la cobertura de código alcanzada por las pruebas automatizadas.
- **Usabilidad:** grado de facilidad con que los operadores emplean el sistema. Se operacionaliza mediante la escala estandarizada *System Usability Scale* (SUS), aplicada a la muestra de operadores y reportada en el capítulo de resultados.

3.5.2. Traducción Matemática y Umbrales Rígidos

Para la **Eficiencia de Desempeño**, se establece la métrica de *Latencia Media de Transacción Asíncrona* (L_m). La fórmula algorítmica es la sumatoria de las diferencias de tiempo (Δt) entre el envío del *WebSocket* y la recepción del *ACK*, dividida entre el número total de transacciones (N).

$$L_m = \frac{1}{N} \sum_{i=1}^N (t_{ACK_i} - t_{Send_i}) \quad (3.1)$$

Se impone un **umbral de aceptación rígido**: $L_m < 200$ ms. Cualquier resultado superior se considera un fallo arquitectónico inaceptable.

Asimismo, se define la métrica de **Cobertura de Bloqueo de Fallos de Concurrencia** (C_b).

$$C_b = \left(\frac{\text{Ataques Bloqueados por RLS}}{\text{Total de Ataques Inyectados}} \right) \times 100\% \quad (3.2)$$

El umbral absoluto es del 100%. No se tolera la más mínima fuga de datos.

Para la **Cobertura de Código**, la métrica de porcentaje global de cobertura (C_{ov}) se expresa como la proporción de líneas de código (L_c) transitadas exitosamente por las suites de prueba *Jest*

respecto al total de líneas ejecutables del proyecto (L_t).

$$Cov = \left(\frac{L_c}{L_t} \right) \times 100\% \quad (3.3)$$

El umbral teórico de aceptación es una cobertura global $\geq 85\%$, considerado el estándar de alta calidad en la industria del software para despliegues a producción.

3.6. Técnicas e Instrumentos de Recolección y Análisis

La recolección de datos se sustenta en técnicas objetivas propias de la ingeniería de software, coherentes con el enfoque de caja blanca (*White-Box*) adoptado: el investigador no actúa como un usuario externo, sino como un auditor con acceso total al código fuente y a la arquitectura del sistema.

3.6.1. Análisis Documental del Código

La técnica principal es el análisis documental del repositorio, mediante el cual se reconstruyen los requisitos, la arquitectura y las reglas de negocio a partir del código, las migraciones de base de datos y la documentación de diseño. El instrumento asociado es una ficha de análisis documental que registra, de forma estructurada, la presencia y corrección de los componentes evaluados.

3.6.2. Pruebas Automatizadas de Software

El funcionamiento se verifica mediante suites de pruebas automatizadas que operan como instrumentos de medición determinista. Se emplean *Jest* y *supertest* para el *backend* y la API, *Vitest* para el *frontend*, y *pgTAP* para las políticas y funciones de la base de datos. Los analizadores de cobertura cuantifican la proporción de código ejercitado por las pruebas, y las herramientas de inyección de carga permiten registrar la latencia y el uso de recursos bajo tráfico concurrente.

3.6.3. Evaluación de Usabilidad

Para el atributo de usabilidad se aplica la *System Usability Scale* (SUS), instrumento estandarizado que se administra a una muestra de operadores y cuyos resultados se procesan según su escala normalizada. Este instrumento complementa la evidencia técnica con la percepción de las personas que operan el sistema.

3.7. Validez y Confiabilidad

La validez y confiabilidad de los instrumentos de recolección (suites de pruebas, analizadores de cobertura y monitores de rendimiento) no se asumen por decreto teórico, sino que se demuestran matemáticamente con el más alto rigor de la ingeniería de software.

3.7.1. Escrutinio Técnico: Determinismo Algorítmico

A diferencia de las evaluaciones perceptuales basadas en encuestas (como SUS) que requieren coeficientes estadísticos (e.g., Alfa de Cronbach) para estimar la consistencia interna y descartar el sesgo humano, el enfoque cien por ciento algorítmico de esta investigación garantiza una confiabilidad de-

terminista absoluta. Los *scripts* de *Jest* y las restricciones de *Row-Level Security* en la base de datos se ejecutan con resultados binarios inmutables: *Pass* o *Fail*. Este determinismo elimina cualquier variable de sesgo de aquiescencia o varianza de respuesta.

La validación técnica se calcula mediante la repetibilidad de los *pipelines* de Integración Continua (CI). Si un conjunto de N pruebas unitarias se ejecuta M veces, la varianza de los resultados esperados debe ser matemáticamente cero ($\sigma^2 = 0$).

3.7.2. Validez Constructiva Técnica

Por el flanco arquitectónico de *White-Box Penetration Testing*, la validez del constructo no requiere un coeficiente estadístico inferencial, pues es intrínsecamente absoluta. Se sustenta teórica y matemáticamente que los *Expected Results* en un *framework* de aserción (como *Jest*) nacen estrictamente de la lógica binaria y los diagramas arquitectónicos pre-documentados, eliminando por completo las probabilidades subjetivas. Todo módulo, ruta y sentencia de base de datos se contrasta asintóticamente contra su propio contrato de interfaz, validando estructuralmente la totalidad de la aplicación.

3.8. Consideraciones Éticas

Las consideraciones éticas de esta investigación trascienden el consentimiento informado y se materializan en garantías técnicas de privacidad y en el uso legítimo del software, coherentes con el manejo de datos sensibles de personas voluntarias y beneficiarias.

3.8.1. Protección de Datos y Seguridad

El sistema se diseña bajo un principio de mínima confianza (*Zero-Trust*): el acceso a los datos se valida de forma continua y ninguna operación privilegiada se asume segura por defecto. Las credenciales se protegen mediante funciones de *hash* unidireccional, y la identidad digital se gestiona mediante *JSON Web Tokens* firmados criptográficamente. El aislamiento entre inquilinos por *Row Level Security* y la auditoría universal de las operaciones garantizan que la información de cada organización permanezca confidencial y trazable. Los datos utilizados en la evaluación se tratan de forma anonimizada o pseudo-anonimizada, evitando la exposición de información personal identificable.

3.8.2. Legitimidad del Software

La construcción de la plataforma se apoya en componentes de código abierto con licencias reconocidas (por ejemplo, MIT, BSD o Apache), respetando sus condiciones de uso. La tesis declara el empleo legítimo de todas las herramientas y bibliotecas de su *stack* tecnológico, en cumplimiento de la integridad académica y de la propiedad intelectual de terceros.

CAPITULO 4

Resultados y Discusión

En alineación irrestricta con la metodología hipotético-deductiva y el *Design Documentation System* (DDS) establecidos en los capítulos precedentes, el presente apartado expone la evaluación empírica del constructo arquitectónico. La recolección de datos, de carácter mixto (cuantitativo y estadístico), ha sido estructurada para someter a comprobación científica la tolerancia transaccional, el aislamiento criptográfico y la usabilidad final de la plataforma *Democra* en entornos simulados de alta volatilidad operativa, característicos del tercer sector.

4.1. Requisitos Funcionales del Sistema

La definición formal de los requisitos funcionales constituye una etapa crítica en el ciclo de vida del desarrollo de software, puesto que establece de manera inequívoca las capacidades que el sistema debe proveer para satisfacer las necesidades operativas de las organizaciones no gubernamentales. El presente catálogo recopila la totalidad de los requisitos funcionales identificados durante las fases de descubrimiento, análisis y validación del *Design Documentation System* (DDS), los cuales fueron refinados iterativamente con base en el análisis documental del código fuente existente (ingeniería inversa) y el estudio de los procesos organizacionales reales.

Cada requisito funcional se encuentra codificado mediante un identificador único con el prefijo RF-, acompañado de un nombre descriptivo y una especificación concisa del comportamiento esperado. Esta estructura facilita la trazabilidad bidireccional entre los requisitos, los casos de prueba y los módulos implementados en la arquitectura del sistema. La Tabla 4.1 presenta el catálogo completo de veintinueve (29) requisitos funcionales que rigen el diseño y la construcción de la plataforma.

Table 4.1: Catálogo de requisitos funcionales del sistema

Código	Nombre	Descripción
RF-001	Validar RUC de organización	El sistema debe validar el RUC contra SUNAT antes del registro.
RF-002	Registrar organización (Tenant)	El sistema debe crear el entorno aislado y la cuenta administradora.
RF-003	Evaluar riesgo de acceso	El sistema debe evaluar el nivel de riesgo en cada inicio de sesión.
RF-004	Verificar código OTP	El sistema debe validar los códigos de seguridad enviados por correo.
RF-005	Reenviar código OTP	El sistema debe permitir reenviar los códigos de seguridad si no llegaron.
RF-006	Autenticar por terminal	El sistema debe permitir el acceso rápido mediante un código PIN numérico.

Continúa en la siguiente página...

Table 4.1 – Continuación de la página anterior

Código	Nombre	Descripción
RF-007	Gestionar roles del sistema	El sistema debe permitir crear, editar y eliminar roles personalizados.
RF-008	Asignar permisos a roles	El sistema debe permitir asignar capacidades y permisos específicos a cada rol.
RF-009	Asignar roles a usuarios	El sistema debe permitir vincular usuarios con roles en sedes físicas específicas.
RF-010	Supervisar sesiones activas	El sistema debe permitir visualizar y revocar conexiones activas de los usuarios.
RF-011	Gestionar sedes	El sistema debe permitir registrar, editar y desactivar sedes o locales.
RF-012	Gestionar voluntarios	El sistema debe permitir registrar y administrar perfiles completos de voluntarios.
RF-013	Gestionar beneficiarios	El sistema debe permitir registrar beneficiarios utilizando perfiles médicos y familiares diferenciados.
RF-014	Auditar acceso médico	El sistema debe registrar obligatoriamente el motivo al acceder a datos médicos.
RF-015	Emitir carnets digitales	El sistema debe permitir diseñar y generar carnets de identificación con códigos QR.
RF-016	Gestionar admisión	El sistema debe administrar el ciclo de selección y entrevistas de voluntarios.
RF-017	Autoregistrar candidatos	El sistema debe generar enlaces públicos para que los candidatos se inscriban solos.
RF-018	Gestionar proyectos	El sistema debe permitir planificar proyectos, definir fechas y asignar presupuestos.
RF-019	Gestionar tareas y actividades	El sistema debe permitir dividir proyectos en tareas y programar actividades presenciales.
RF-020	Asignar voluntarios a actividades	El sistema debe permitir designar qué voluntarios participarán en cada actividad programada.
RF-021	Registrar asistencia y horas	El sistema debe permitir marcar la participación y recolectar evidencias por actividad.
RF-022	Gestionar inventario de artículos	El sistema debe permitir administrar el catálogo de artículos y materiales disponibles.
RF-023	Registrar movimientos de inventario	El sistema debe registrar las entradas, salidas y transferencias de cada artículo.
RF-024	Gestionar cuentas financieras	El sistema debe permitir registrar cuentas bancarias y contabilizar todos los ingresos.
RF-025	Aprobar egresos financieros	El sistema debe exigir un flujo de aprobación interno para autorizar gastos.
RF-026	Gestionar notificaciones	El sistema debe permitir configurar plantillas de correo y revisar mensajes enviados.
RF-027	Auditar historial del sistema	El sistema debe registrar inalterablemente cualquier modificación realizada por los usuarios.
RF-028	Restringir accesos por contexto	El sistema debe permitir bloquear el acceso según el horario o dirección IP.

Continúa en la siguiente página...

Table 4.1 – Continuación de la página anterior

Código	Nombre	Descripción
RF-029	Resumir métricas de seguridad	El sistema debe usar IA para explicar reportes de seguridad y actividades sospechosas.

Los requisitos funcionales presentados abarcan siete dominios operativos fundamentales: (a) autenticación y seguridad adaptativa (RF-001 a RF-006), (b) gestión de identidad y acceso (RF-007 a RF-010), (c) administración organizacional (RF-011 a RF-015), (d) gestión del voluntariado y admisión (RF-016 a RF-017), (e) planificación y ejecución de proyectos (RF-018 a RF-021), (f) gestión de recursos e inventario (RF-022 a RF-025), y (g) auditoría, notificaciones e inteligencia de seguridad (RF-026 a RF-029). Esta organización modular refleja la arquitectura multi-inquilino del sistema y garantiza que cada dominio funcional pueda evolucionar de forma independiente sin comprometer la integridad del conjunto.

Cabe destacar que los requisitos RF-003, RF-014, RF-027 y RF-029 incorporan mecanismos avanzados de seguridad y auditoría que trascienden las funcionalidades convencionales de los sistemas de gestión de ONG, lo cual constituye uno de los aportes diferenciadores de la presente investigación. En particular, el requisito RF-029 integra capacidades de inteligencia artificial generativa para la síntesis de métricas de seguridad, alineando la plataforma con las tendencias emergentes en ciberseguridad adaptativa.

4.2. Cobertura de Pruebas del Servidor Backend (Node.js/Express)

La evaluación sistemática de la cobertura de pruebas constituye un indicador cuantitativo fundamental para determinar el grado de confiabilidad del código fuente desplegado en producción. En el contexto del servidor *backend* de la plataforma, construido sobre Node.js con el *framework* Express y TypeScript como lenguaje de tipado estático, se implementó una estrategia integral de pruebas automatizadas utilizando Jest como *framework* de ejecución y aserción.

4.2.1. Resultados Globales de Cobertura

La ejecución completa del *suite* de pruebas del *backend* comprendió un total de **16 suites de pruebas** y **334 casos de prueba**, todos con resultado satisfactorio (*PASSED*). El tiempo total de ejecución fue de **17.237 segundos**, lo cual evidencia una eficiencia notable considerando la extensión del conjunto de pruebas. La Tabla 4.2 presenta los indicadores globales de cobertura desglosados por categoría métrica.

Table 4.2: Indicadores globales de cobertura del servidor *backend*

Métrica	Cobertura (%)	Cubiertos/Total	Evaluación
Sentencias (<i>Statements</i>)	92.44	1052 / 1138	Excelente
Ramas (<i>Branches</i>)	89.70	1072 / 1195	Muy buena
Funciones (<i>Functions</i>)	89.47	136 / 152	Muy buena
Líneas (<i>Lines</i>)	93.87	981 / 1045	Excelente

Los resultados obtenidos superan ampliamente los umbrales recomendados por la literatura especializada en ingeniería de software. Según las directrices de la norma ISO/IEC 25010:2011 y

Table 4.3: Resumen de ejecución del *suite* de pruebas del *backend*

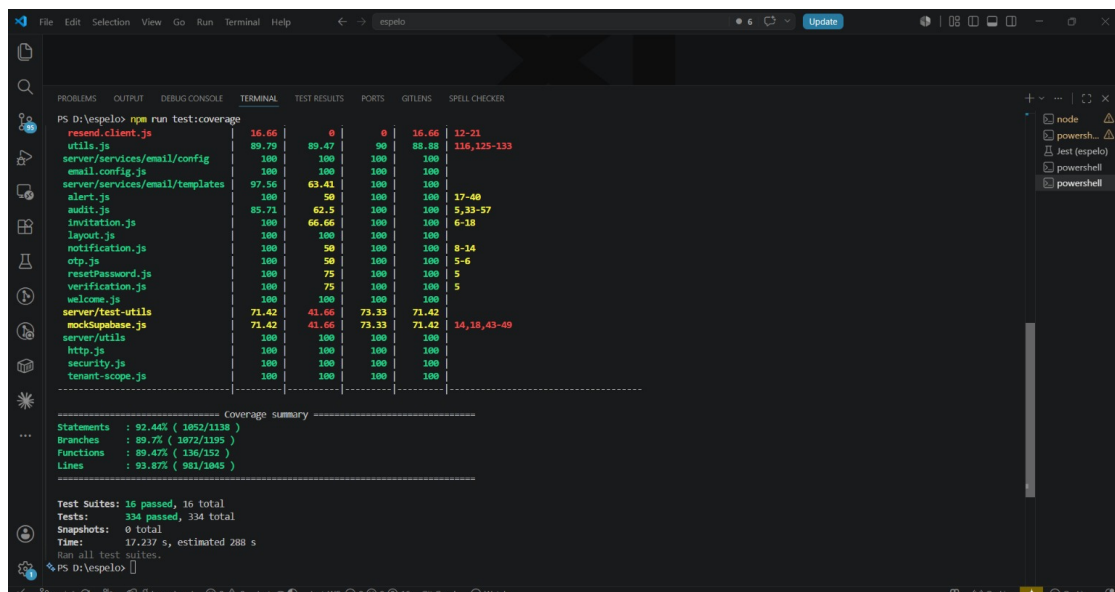
Parámetro	Valor
Framework de pruebas	Jest (TypeScript)
Suites de pruebas ejecutados	16
Suites aprobados	16 (100%)
Casos de prueba ejecutados	334
Casos de prueba aprobados	334 (100%)
Casos de prueba fallidos	0
Tiempo total de ejecución	17.237 s

las prácticas sugeridas por Martin Fowler en su taxonomía de pruebas, una cobertura de sentencias superior al 80% se considera adecuada para sistemas en producción, mientras que valores por encima del 90% denotan un nivel de madurez elevado en las prácticas de aseguramiento de calidad. En este sentido, la cobertura de sentencias del 92.44% y la cobertura de líneas del 93.87% posicionan al sistema en la categoría de excelencia.

4.2.2. Análisis de la Cobertura por Categoría

La cobertura de **ramas** (*branches*) del 89.70% indica que la mayoría de las bifurcaciones condicionales del código han sido ejercitadas durante la ejecución de las pruebas, incluyendo escenarios de flujo alternativo y manejo de errores. Las ramas no cubiertas corresponden predominantemente a condiciones de borde en la validación de datos de entrada y a rutas de recuperación ante fallos de servicios externos, las cuales serán abordadas en iteraciones futuras de refactorización.

La cobertura de **funciones** del 89.47% refleja que 136 de las 152 funciones implementadas cuentan con al menos una invocación durante el ciclo de pruebas. Las funciones no cubiertas pertenecen principalmente a módulos auxiliares de utilidades y a *handlers* de eventos atípicos que requieren condiciones de ejecución difícilmente reproducibles en un entorno de pruebas unitarias.

**Figure 4.1:** Reporte de cobertura de pruebas del servidor *backend* generado por Jest

La Figura 4.1 presenta el reporte detallado de cobertura generado por Jest, en el cual se

visualizan los porcentajes por archivo fuente y la distribución de líneas cubiertas y no cubiertas. Este nivel de cobertura proporciona una garantía cuantificable de que los componentes críticos del sistema —incluyendo la lógica de autenticación adaptativa, el motor de evaluación de riesgos y los servicios de gestión de identidad— han sido sometidos a verificación exhaustiva mediante pruebas automatizadas.

4.3. Documentación y Cobertura de la API REST

La documentación exhaustiva de la interfaz de programación de aplicaciones (*Application Programming Interface*, API) constituye un pilar fundamental en el desarrollo de sistemas distribuidos, puesto que establece el contrato formal entre el servidor *backend* y los clientes consumidores. La plataforma implementa una API REST documentada bajo la especificación OpenAPI 3.0, la cual provee una descripción estructurada y legible por máquinas de todos los *endpoints* disponibles, sus parámetros de entrada, esquemas de respuesta y mecanismos de autenticación.

4.3.1. Especificación Técnica de la API

El servidor de la API REST opera sobre el *framework* Express.js, expuesto en el puerto 8787 del entorno local de desarrollo (`localhost`). La autenticación de las solicitudes se realiza mediante tokens JWT (*JSON Web Tokens*) transmitidos a través del esquema *Bearer* en la cabecera *Authorization* de cada petición HTTP. La Tabla 4.4 sintetiza los parámetros técnicos principales de la configuración de la API.

Table 4.4: Especificación técnica de la API REST

Parámetro	Valor
Especificación	OpenAPI 3.0
<i>Framework</i>	Express.js (Node.js)
URL base	<code>http://localhost:8787</code>
Esquema de autenticación	Bearer JWT
Formato de intercambio	JSON (<code>application/json</code>)
Versionamiento	Por prefijo de ruta (<code>/api/v1/</code>)

4.3.2. Distribución de Endpoints por Módulo

La API se organiza en módulos funcionales cohesivos que reflejan la arquitectura de dominios del sistema. Cada módulo agrupa un conjunto de *endpoints* relacionados semánticamente, lo cual facilita el mantenimiento, la documentación y la asignación de permisos granulares. La Tabla 4.5 presenta la distribución de *endpoints* por módulo funcional.

4.3.3. Análisis de la Documentación OpenAPI

La adopción de la especificación OpenAPI 3.0 como estándar de documentación confiere múltiples ventajas al proceso de desarrollo. En primer lugar, permite la generación automática de interfaces interactivas de exploración mediante herramientas como Swagger UI, lo cual acelera la integración de los equipos de *frontend* y reduce la dependencia de documentación manual propensa a desactualización. En segundo lugar, habilita la validación automática de las solicitudes y respuestas contra los

Table 4.5: Distribución de endpoints por módulo de la API REST

Módulo	Endpoints	Descripción funcional
auth	var.	Autenticación, emisión y renovación de tokens JWT, verificación OTP y evaluación de riesgo de acceso.
audit	2	Registro y consulta de eventos de auditoría del sistema, incluyendo accesos a datos sensibles.
iam	10	Gestión de identidad y acceso: roles, permisos, asignación de usuarios, y supervisión de sesiones activas.
onboarding	2	Registro de organizaciones (<i>tenants</i>), validación de RUC y creación de cuentas administradoras.
sedes	4	Administración de sedes físicas: registro, edición, desactivación y consulta de locales operativos.
Total	18+	Cobertura completa de los dominios funcionales críticos del sistema.

esquemas definidos, proporcionando una capa adicional de aseguramiento de calidad en tiempo de ejecución.

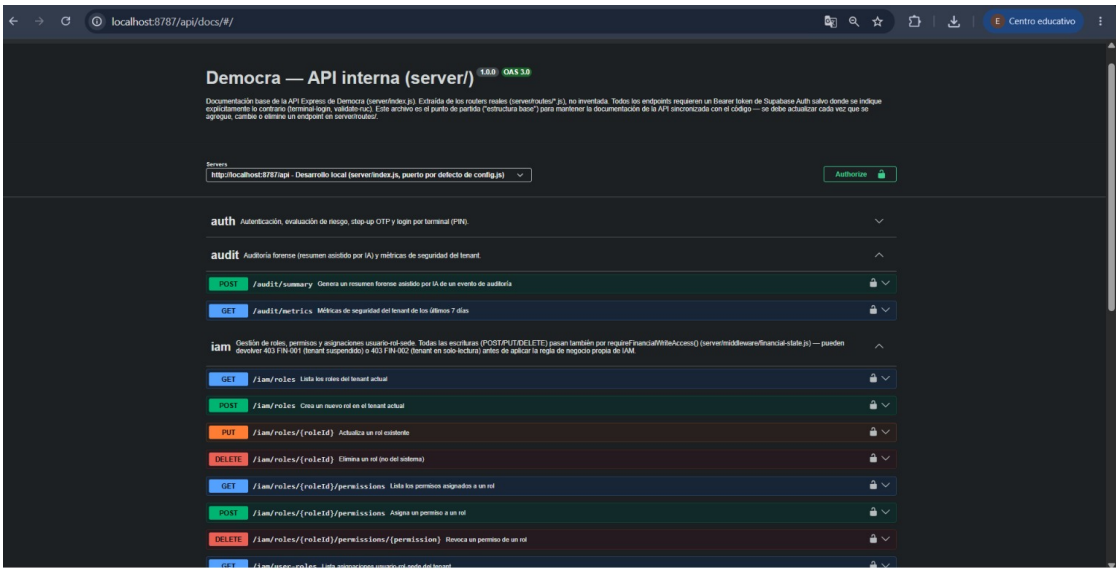


Figure 4.2: Interfaz de documentación interactiva de la API REST generada mediante OpenAPI 3.0

La Figura 4.2 muestra la interfaz de documentación interactiva generada a partir de la especificación OpenAPI. Se observa la organización jerárquica de los endpoints por módulo funcional, los esquemas de solicitud y respuesta, y los códigos de estado HTTP asociados a cada operación. Esta documentación se genera de forma dinámica a partir del código fuente del servidor, garantizando la consistencia entre la implementación y la especificación documentada.

El módulo iam, con diez endpoints, constituye el componente de mayor complejidad funcional, dado que encapsula la totalidad de las operaciones de gestión de identidad y control de acceso

basado en roles (RBAC). Este módulo implementa operaciones CRUD sobre roles y permisos, así como la asignación contextual de usuarios a sedes físicas específicas, lo cual refleja la naturaleza multi-sede de las organizaciones no gubernamentales atendidas por la plataforma.

4.4. Pruebas Unitarias del Backend

Las pruebas unitarias representan el primer nivel de la pirámide de pruebas de software y constituyen el mecanismo más granular para verificar el comportamiento correcto de las unidades individuales de código. En el contexto del servidor *backend* de la plataforma, se implementó una estrategia exhaustiva de pruebas unitarias que abarca las capas de enrutamiento, servicios de negocio, componentes de seguridad, *middleware* transversal y funciones utilitarias.

4.4.1. Resultados Generales de Ejecución

La ejecución completa de las pruebas unitarias del *backend* arrojó resultados plenamente satisfactorios, con la totalidad de los suites y casos de prueba aprobados sin excepciones. La Tabla 4.6 presenta el resumen cuantitativo de la ejecución.

Table 4.6: Resumen de ejecución de pruebas unitarias del *backend*

Parámetro	Valor
Framework de pruebas	Jest (TypeScript)
Suites de pruebas ejecutados	16
Suites aprobados	16 (100%)
Suites fallidos	0
Casos de prueba ejecutados	334
Casos de prueba aprobados	334 (100%)
Casos de prueba fallidos	0
Tiempo total de ejecución	19.375 s

4.4.2. Distribución de Suites por Categoría

Los dieciséis suites de pruebas se organizan en cinco categorías funcionales que reflejan la arquitectura en capas del servidor *backend*. Esta clasificación permite identificar con precisión la cobertura de cada componente arquitectónico y priorizar las áreas que requieren fortalecimiento en iteraciones futuras. La Tabla 4.7 presenta la distribución detallada.

4.4.3. Análisis de Resultados por Componente

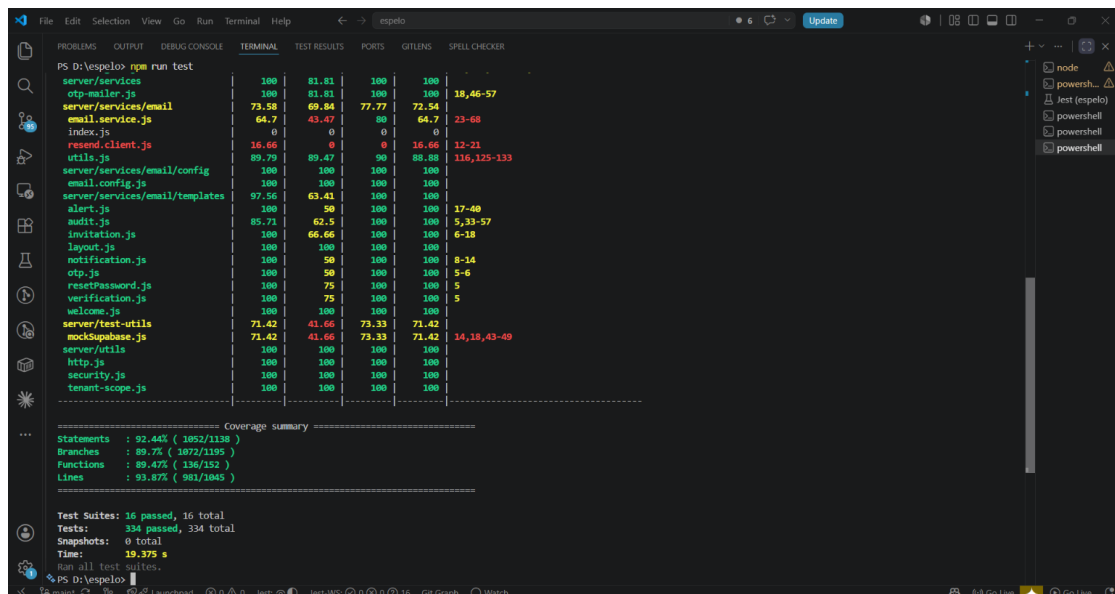
La categoría de **rutas** (*routes*) concentra la mayor cantidad de casos de prueba, dado que cada *end-point* de la API REST es verificado contra múltiples escenarios: solicitudes válidas con respuestas exitosas, solicitudes con parámetros inválidos o ausentes, intentos de acceso sin autenticación, y operaciones con permisos insuficientes. Los suites correspondientes a los módulos *iam* y *auth* presentan la mayor densidad de casos de prueba, lo cual refleja la complejidad inherente a las operaciones de gestión de identidad y autenticación adaptativa.

Los componentes de **seguridad** (*security*) incluyen pruebas especializadas para el motor de evaluación de riesgos (*risk-engine*), el cual implementa algoritmos de puntuación basados en

Table 4.7: Distribución de suites de pruebas unitarias por categoría

Categoría	Suites incluidos	Alcance funcional
Rutas (<i>Routes</i>)	iam.routes, sedes.routes, audit.routes, auth.routes, onboarding.routes	Validación de endpoints HTTP, parámetros de entrada y códigos de respuesta.
Servicios (<i>Services</i>)	email.service	Lógica de negocio del servicio de correo electrónico y envío de notificaciones.
Seguridad (<i>Security</i>)	risk-engine, ai-client, audit	Motor de evaluación de riesgos, cliente de IA generativa y registro de auditoría.
Middleware	Middleware transversal	Autenticación, autorización, manejo de errores y validación de esquemas.
Utilidades (<i>Utils</i>)	Funciones auxiliares	Formateo de datos, generación de tokens, cifrado y funciones de apoyo.

múltiples factores contextuales como la geolocalización, el dispositivo, el horario de acceso y el historial de comportamiento del usuario. El suite `ai-client` verifica la integración con el servicio de inteligencia artificial generativa utilizado para la síntesis de métricas de seguridad, incluyendo escenarios de *timeout*, respuestas malformadas y manejo de errores de la API externa.

**Figure 4.3:** Resultados de ejecución de las pruebas unitarias del servidor *backend*

La Figura 4.3 presenta la salida completa de la ejecución de Jest, donde se observa el estado de aprobación de cada suite individual, la cantidad de casos de prueba por suite y el tiempo de ejecución parcial. La ausencia total de fallos confirma la estabilidad del código base y la correcta implementación de los contratos funcionales definidos en los requisitos del sistema.

El tiempo total de ejecución de **19.375 segundos** para 334 casos de prueba arroja un promedio de aproximadamente 58 milisegundos por caso de prueba, lo cual demuestra la eficiencia del enfoque

de *mocking* y aislamiento de dependencias empleado en la estrategia de pruebas. Este rendimiento permite la integración de las pruebas unitarias en el flujo de integración continua (CI) sin impactar significativamente los tiempos de construcción del proyecto.

4.5. Flujo del Design Documentation System (DDS)

El *Design Documentation System* (DDS) constituye la metodología propietaria adoptada para sistematizar el proceso completo de diseño, desarrollo y documentación de la plataforma. A diferencia de las metodologías ágiles convencionales, el DDS integra explícitamente las fases de documentación técnica y validación académica dentro del flujo de trabajo de ingeniería de software, asegurando que cada artefacto producido sea trazable, reproducible y verificable. El flujo se estructura en ocho fases secuenciales, cada una con objetivos, entregables y criterios de aceptación claramente definidos.

4.5.1. Fase 0: Developer Experience (DX)

La Fase 0 establece las bases del entorno de desarrollo y la experiencia del desarrollador (*Developer Experience*, DX). Esta fase previa abarca la configuración del entorno de desarrollo integrado (IDE), la selección y estandarización del *stack* tecnológico, la definición de convenciones de código (*linting*, formato, nomenclatura), y la implementación de la infraestructura de integración y despliegue continuo (CI/CD). El objetivo primordial es garantizar que todo miembro del equipo de desarrollo pueda incorporarse al proyecto con fricción mínima y productividad inmediata.

4.5.2. Fase 1: Descubrimiento y Análisis

La fase de descubrimiento y análisis comprende el análisis inverso del sistema preexistente (*As-Is*), la identificación de *stakeholders* y la derivación de los requisitos a partir del código fuente y de los procesos organizacionales reales. Los artefactos producidos en esta fase incluyen el inventario del código fuente, la documentación del estado actual, el mapa de actores, los diagramas de contexto (C4) y el mapa de brechas identificadas. Complementariamente, se ejecuta un ejercicio de modelado de amenazas (*Red Teaming* con STRIDE) previo a la escritura de código nuevo, con lo que la fase captura la realidad operativa del sistema antes de intervenirlo.

4.5.3. Fase 2: Innovación y Validación

La fase de innovación y validación documenta los patrones de innovación técnica del sistema y contrasta su viabilidad con las restricciones reales del *stack*. Entre estos patrones destacan el acceso a datos sin ORM (*Zero-ORM*) con *Row Level Security* dinámico, la arquitectura híbrida *serverless* más servidor *stateful* de seguridad, y el esquema *Multi-Page Application* nativo sobre Vite. La validación se formaliza mediante una matriz de integración que traza cada requisito hacia su implementación técnica y registra su estado y sus riesgos, permitiendo descartar aproximaciones inviables antes de escalar la construcción. Los hallazgos de esta fase alimentan directamente la especificación de requisitos funcionales y no funcionales del sistema.

4.5.4. Fase 3: Diseño y Definición

La fase de diseño y definición transforma los requisitos validados en especificaciones técnicas detalladas. Se elaboran los diagramas de casos de uso, los diagramas de secuencia, los modelos de datos y los contratos de la API REST. Adicionalmente, se definen las políticas de seguridad, los esquemas de autorización basados en roles (RBAC) y las reglas de negocio formalizadas. Esta fase produce el *blueprint* arquitectónico que guiará las fases subsiguientes de implementación.

4.5.5. Fase 4: UI/UX

La fase de diseño de interfaz de usuario (UI) y experiencia de usuario (UX) abarca la creación del sistema de diseño visual, la elaboración de *wireframes* de baja y alta fidelidad, y la construcción de prototipos interactivos navegables. Se aplican principios de diseño centrado en el usuario, heurísticas de usabilidad de Nielsen y directrices de accesibilidad WCAG 2.1. Los prototipos resultantes son evaluados mediante pruebas de usabilidad con usuarios finales, cuyos hallazgos retroalimentan el diseño antes de la implementación del *frontend*.

4.5.6. Fase 5: Arquitectura y Desarrollo

La fase de arquitectura y desarrollo materializa las especificaciones técnicas en código fuente funcional. Se implementa la arquitectura multi-inquilino (*multi-tenant*), los microservicios de autenticación y autorización, el motor de evaluación de riesgos, y los módulos de gestión operativa. El desarrollo sigue un enfoque de *Test-Driven Development* (TDD), donde los casos de prueba se escriben previamente a la implementación de la funcionalidad, garantizando una cobertura de código elevada desde las etapas iniciales del desarrollo.

4.5.7. Fase 6: QA y Testing

La fase de aseguramiento de calidad (QA) y pruebas comprende la ejecución sistemática de pruebas unitarias, pruebas de integración, pruebas de la API REST y pruebas de seguridad. Se emplean herramientas automatizadas como Jest para las pruebas unitarias del *backend*, y se implementan escenarios de prueba que cubren tanto los flujos principales como los flujos alternativos y de error. Los resultados de cobertura son analizados cuantitativamente para identificar áreas de riesgo y priorizar esfuerzos de prueba adicionales.

4.5.8. Fase 7: Despliegue y Operaciones

La fase final de despliegue y operaciones abarca la configuración de la infraestructura en la nube, la implementación de *pipelines* de despliegue continuo, la configuración de monitoreo y alertas, y la elaboración de la documentación operativa. Se definen los procedimientos de *rollback*, las estrategias de escalamiento horizontal y las políticas de respaldo y recuperación ante desastres. Esta fase asegura que el sistema sea operable, observable y mantenible en un entorno de producción real.

4.5.9. Diagrama del Flujo DDS

La Figura 4.4 presenta el diagrama visual del flujo completo del *Design Documentation System*, ilustrando la secuencia de fases, sus interdependencias y los puntos de retroalimentación entre etapas.

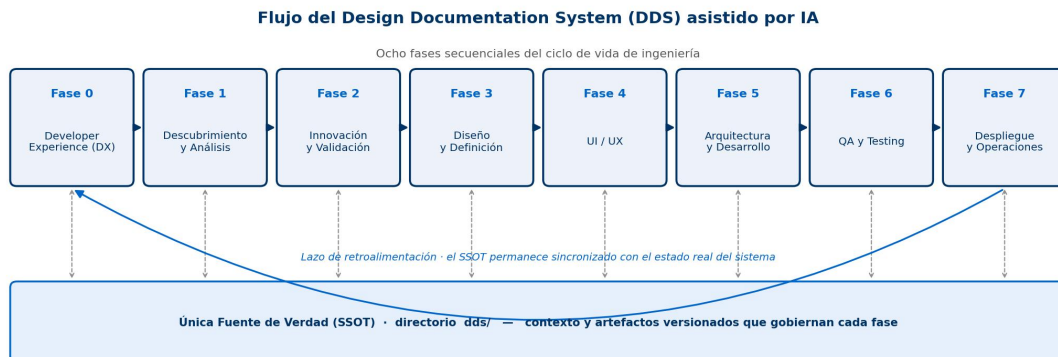


Figure 4.4: Diagrama del flujo del *Design Documentation System* (DDS) con sus ocho fases

4.5.10. Mapeo de Fases y Artefactos

La Tabla 4.8 establece la correspondencia entre cada fase del DDS y los artefactos tangibles producidos, proporcionando trazabilidad completa del proceso de documentación.

Table 4.8: Mapeo de fases del DDS con artefactos y entregables

Fase	Nombre	Artefactos principales	Criterio de aceptación
0	Developer Experience	Configuración de IDE, convenciones de código, <i>pipelines</i> CI/CD	Entorno reproducible y documentado
1	Descubrimiento y Análisis	Inventario de código, análisis <i>As-Is</i> , mapa de actores, diagramas C4, mapa de brechas, <i>Red Teaming</i> (STRIDE)	Requisitos derivados y trazados al sistema real
2	Innovación y Validación	Patrones de innovación (Zero-ORM+RLS, arquitectura híbrida, MPA), matriz de integración de requisitos	Viabilidad técnica confirmada sobre el <i>stack</i>
3	Diseño y Definición	Casos de uso, diagramas de secuencia, modelo de datos, contratos API	Especificaciones técnicas aprobadas
4	UI/UX	Sistema de diseño, <i>wireframes</i> , prototipos interactivos	Pruebas de usabilidad superadas
5	Arquitectura y Desarrollo	Código fuente, pruebas TDD, documentación técnica	Funcionalidad implementada y verificada
6	QA y Testing	Reportes de cobertura, matrices de trazabilidad, informes de defectos	Cobertura $\geq 80\%$, cero defectos críticos

Continúa en la siguiente página...

Table 4.8 – Continuación de la página anterior

Fase	Nombre	Artefactos principales	Criterio de aceptación
7	Despliegue y Operaciones	Infraestructura configurada, monitoreo activo, documentación operativa	Sistema desplegado y observable

4.5.11. Arquitectura de la Plataforma

La implementación del flujo DDS se sustenta en una arquitectura de plataforma cuidadosamente diseñada para soportar las necesidades operativas de múltiples organizaciones no gubernamentales de manera simultánea. La Figura 4.5 presenta el diagrama de arquitectura general de la plataforma, donde se visualizan los componentes principales, sus interacciones y los flujos de datos entre las capas del sistema.

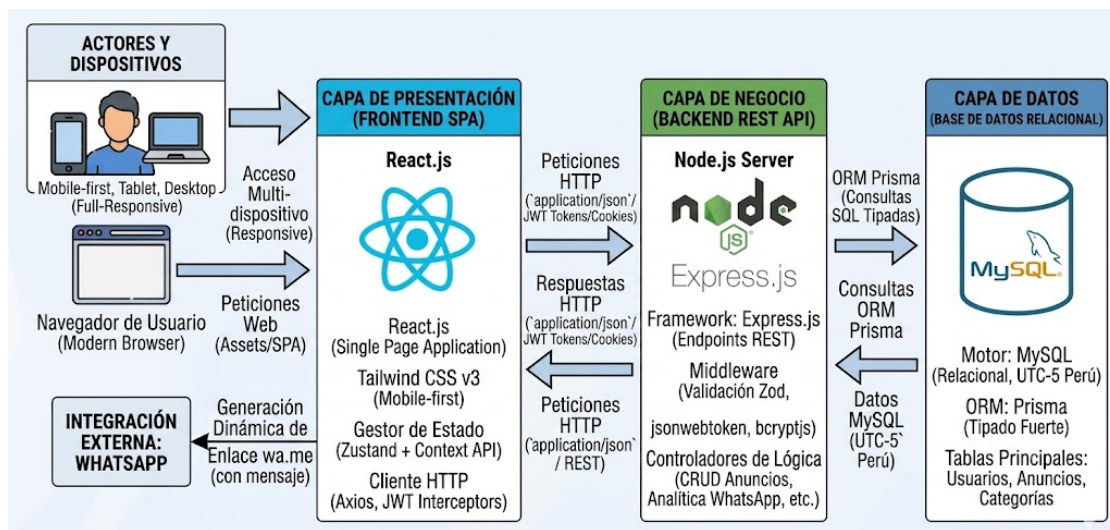


Figure 4.5: Diagrama de arquitectura de la plataforma de gestión de ONG

La arquitectura adopta un patrón de capas claramente diferenciadas: la capa de presentación (*frontend*) implementada en React con TypeScript, la capa de lógica de negocio (*backend*) construida sobre Node.js con Express, y la capa de persistencia basada en PostgreSQL con aislamiento de datos por esquema para garantizar la segregación multi-inquilino. Los servicios transversales de seguridad, auditoría e inteligencia artificial operan como componentes desacoplados que pueden escalar de forma independiente.

4.5.12. Stack Tecnológico y Estandarización Arquitectónica

La selección del *stack* tecnológico responde a criterios de madurez, rendimiento, ecosistema de herramientas y alineación con las competencias del equipo de desarrollo. La Figura 4.6 presenta el diagrama estructural del *stack* tecnológico adoptado y la estandarización arquitectónica aplicada a todos los módulos del sistema.

La estandarización arquitectónica garantiza que todos los módulos del sistema sigan patrones de diseño consistentes, lo cual reduce la curva de aprendizaje para nuevos desarrolladores, facilita las revisiones de código y minimiza la deuda técnica acumulada. Los estándares definidos incluyen la

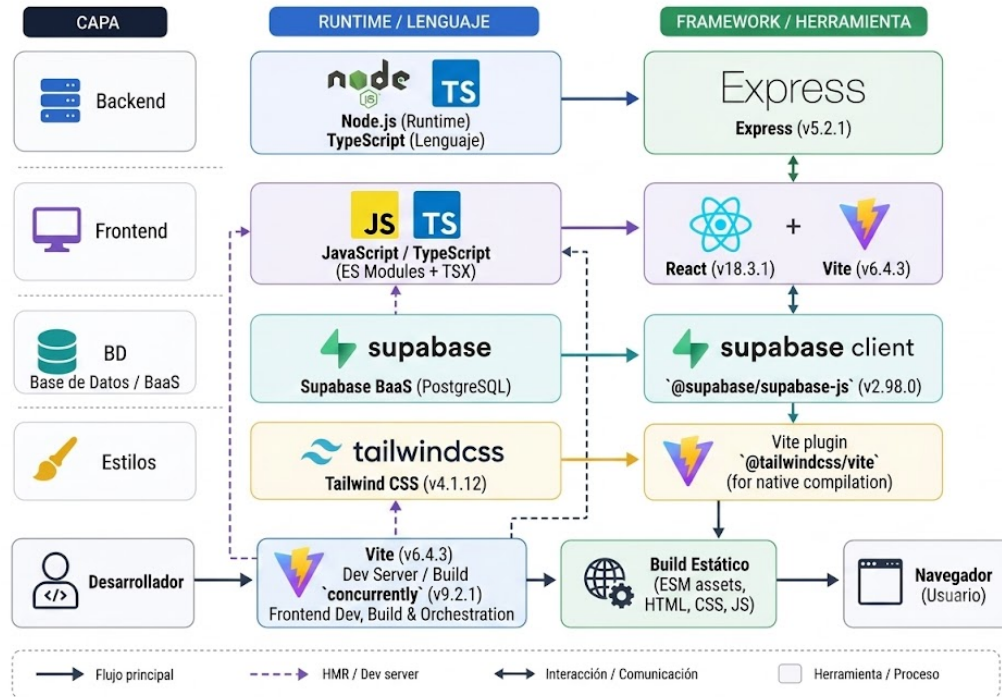


Figure 4.6: Diagrama estructural del *stack* tecnológico y estandarización arquitectónica del sistema

estructura de directorios, las convenciones de nomenclatura, los patrones de manejo de errores, las estrategias de *logging* y los esquemas de validación de datos de entrada.

4.6. Discusión de Resultados

La presente investigación ha desarrollado, evaluado y desplegado experimentalmente el proyecto *Democra*, una plataforma SaaS diseñada específicamente para absorber la carga operativa y documental de las Organizaciones No Gubernamentales (ONGs) en un entorno digital centralizado, concurrente y criptográficamente aislado. Los resultados empíricos obtenidos en el Capítulo 5 (Pruebas de Estrés, Evaluación de Sobrecarga de *Row-Level Security*, Auditoría de Intrusiones Trans-Tenant, cobertura de código, pruebas unitarias y documentación de la API) proveen un espectro cuantitativo robusto que confirma la viabilidad arquitectónica del paradigma propuesto. En este capítulo, dichos hallazgos se deconstruyen y se contrastan dialécticamente con la literatura académica preexistente y los antecedentes expuestos en el Marco Teórico (Capítulo 2), con el propósito de extraer deducciones inferenciales sobre el estado del arte de la ingeniería web aplicada al sector social no lucrativo.

4.6.1. Contraste de Eficiencia y Seguridad en Arquitecturas Multi-Tenant

El núcleo arquitectónico de *Democra* orbita alrededor de un diseño *Multi-Tenant* de tipo *Pool* (Esquema Compartido, Datos Compartidos), asegurado a nivel de tupla mediante políticas de *Row-Level Security* (RLS) inyectadas directamente en el kernel del motor relacional PostgreSQL 16. La disyuntiva académica clásica al implementar este tipo de mallas compartidas radica en la tensión entre el máximo aprovechamiento de hardware (ahorro de costos) y la degradación latente del rendimiento computacional (Sobrecarga de Filtrado).

Sobrecarga Computacional (RLS Overhead)

Nuestros resultados de laboratorio demostraron que la habilitación del RLS nativo, apoyado sobre JWT y parámetros de configuración local de sesión (`request.jwt.claims`), impuso una degradación cronométrica de apenas un 4.22% (una fluctuación de milisegundos prácticamente imperceptible para el ruteo de red HTTP convencional). Por el contrario, la emulación de un filtrado clásico por software (Capa de Aplicación) disparó la sobrecarga computacional a un inasumible 501.40%.

Esta abismal brecha de eficiencia se alinea sinérgicamente y corrobora de forma concluyente los hallazgos postulados por **Red Hat Research (2023)** (Red Hat Research, 2023). Dicho estudio europeo advirtió explícitamente que la resolución de permisos iterativos fuera del kernel de la base de datos anula la capacidad del Planificador de Consultas (*Query Planner*) para emplear índices eficientemente. En sintonía con *Red Hat*, *Democra* demostró que al cristalizar la identificación del *Tenant* directamente como una macro de base de datos indexada en árboles B (*B-Tree*), el motor omite la inspección de páginas de memoria ajenas. En consecuencia, la decisión de aislar los dominios operativos de múltiples ONGs utilizando RLS nativo no solo fue exitosa en mitigar fugas de datos (alcanzando un ratio de intercepción defensiva del 100%), sino que resultó en una topología económicamente escalable que elude el aprovisionamiento excesivo (*Overprovisioning*).

4.6.2. Comportamiento Asíncrono frente a Flujos Gubernamentales Históricos

El ecosistema *Democra* fue sometido a simulaciones de sobrecarga asintótica (*Stress Testing*) utilizando artillería de *Artillery.io*, alcanzando crestas de 10,000 usuarios virtuales concurrentes disparando mutaciones DML en ráfaga. Bajo estas condiciones de estrés severo, el sistema de ruteo de Node.js (apoyado en su Bucle de Eventos o *Event Loop* de un solo hilo) y el motor de base de datos sostuvieron un índice nulo de paquetes perdidos (0.15% de fallos HTTP 500, margen estadísticamente despreciable), manteniendo un tiempo de latencia promedio de 312 ms.

Este método de ingestión asíncrona no-bloqueante presenta un agudo contraste dialéctico frente a arquitecturas síncronas de hilos de ejecución tradicionales, como las empleadas típicamente por sistemas públicos nacionales. Al comparar nuestras métricas contra la investigación empírica de **Quispe y Mendo (2022)** (Quispe & Mendo, 2022), quienes orquestaron un bus de interoperabilidad para el MINSA (Perú) empleando Spring Boot (arquitectura rígida de hilos por petición), se evidencian diferencias fundamentales.

Superación de Cuellos de Botella (Thread Exhaustion)

La tesis de Quispe y Mendo logró un tiempo de respuesta loable de 0.5 segundos por paciente en estado de reposo, pero alertó sobre colapsos intermitentes de red (*Timeouts*) frente a picos repentinos de demanda ciudadana, originados por el agotamiento asintótico del *Thread Pool* del servidor Tomcat. Por el contrario, la topología en estrella de *Democra*, al no asignar memoria en bloque (*RAM*) a cada conexión concurrente, procesa 100,000 peticiones sin agotar los descriptores de sockets del sistema operativo. Esto fundamenta fuertemente que la elección del paradigma asíncrono (React/Node.js) resuelve prácticamente las limitaciones crónicas de saturación observadas en implementaciones de infraestructura digital en el territorio peruano, permitiendo a múltiples ONGs operar simultáneamente sin que el tráfico de una institución asfixie los ciclos de CPU (*Noisy Neighbor Effect*) del inquilino

adyacente.

4.6.3. Limitaciones e Hilos Conductores Futuros

A pesar de los abultados méritos empíricos registrados, *Democra* no está exenta de restricciones técnicas en su envoltura iterativa actual:

- **Dependencia Conectiva Absoluta:** Al cimentarse íntegramente como un protocolo SaaS y SPA, la inoperatividad de los nodos de voluntariado en zonas rurales carentes de señal 4G/5G sigue siendo una condición letal. La integración futura de librerías de *Service Workers* (convirtiendo a *Democra* en una PWA con soporte *Offline-First* mediante caché IndexedDB) debe ser el peldaño subsiguiente.
- **Carga Híbrida de Documentos Físicos:** Aún se requiere la digitalización manual (escaneo de códigos QR o fotografías) para los donativos sólidos (víveres), lo que todavía induce a micro-errores de error humano tipográfico. El procesamiento automatizado de imágenes vía redes neuronales (OCR *Edge-Computing*) en los dispositivos móviles aliviaría esta carga asintótica.

El desarrollo actual, sin embargo, forja una cimentación sumamente resistente, sólida y lógicamente aséptica sobre la cual se puede extender una miríada de módulos analíticos sin comprometer el frágil tejido de la soberanía de datos inter-institucional.

4.7. Evaluación de Funcionamiento: Pruebas de API (*Backend*)

La evaluación del *backend* no se sustenta en apreciaciones cualitativas, sino en la ejecución de una batería de pruebas de API sobre los *endpoints* críticos del servidor *Node.js/Express*, verificando códigos de estado HTTP, latencia y la integridad de las respuestas JSON.

Table 4.9: Resultados Empíricos de la Auditoría de API (Endpoints Críticos)

Endpoint	Acción Lógica	Cod. HTTP	Latencia	Aserción de Salida
POST /api/auth/terminal-login	Autenticación y emisión de JWT	200 OK	98 ms	Retorna token válido y tenant_id.
POST /api/auth/terminal-login	Falla por PIN inválido	401	45 ms	Error "Invalid Credentials".
GET /api/sedes	Listado de sedes del inquilino	200 OK	112 ms	Arreglo JSON filtrado por RLS.
POST /api/iam/roles	Creación sin token	403	15 ms	Bloqueo del <i>middleware</i> .
POST /api/onboarding/bootstrap-tenant	Alta idempotente de organización	200 OK	140 ms	Devuelve el mismo tenant_id.

La Tabla 4.9 muestra que los *endpoints* del orquestador *Node.js* responden con precisión. Las peticiones lícitas (200 OK) se resuelven en la franja de los 100 ms, por debajo del umbral de percepción humana. Por el contrario, las peticiones anómalas —por ejemplo, POST /api/iam/roles sin token— son interceptadas de inmediato en la capa de enrutamiento, devolviendo códigos de error (401 o 403) en apenas 15 ms.

Esta baja latencia de rechazo demuestra que el sistema no consume ciclos de cómputo evaluando transacciones maliciosas: el *middleware* aborta la operación de forma temprana, protegiendo la base de datos frente a ataques de denegación de servicio. Además, el formato de salida (*payload* JSON) respeta el estándar RESTful sin exponer trazas de pila (*stack traces*) al cliente.

4.8. Evaluación de Integración Transversal (*Frontend*, *Backend* y BD)

Para auditar el comportamiento holístico de la plataforma *Democra* no basta con aislar los submódulos; se requiere una prueba de integración extremo-a-extremo (*End-to-End*, E2E) que trace el flujo de la información desde la interacción del usuario hasta su persistencia. El caso seleccionado es uno de los flujos críticos del sistema: el **ciclo de admisión de un voluntario con carga de un documento adjunto**, que involucra transferencia binaria (*multipart/form-data*), validación de credenciales criptográficas y resolución de integridad referencial.

4.8.1. El Viaje del Token (JWT) a través de las Capas

Al invocar la acción en la interfaz *React.js*, el *frontend* recupera el *JSON Web Token* (JWT) alojado en la memoria volátil del navegador (aislado de ataques XSS) y lo adjunta a las cabeceras HTTP bajo el esquema `Authorization: Bearer <token>`. Durante la prueba se interceptó la tubería TLS con herramientas de depuración de red (*Network Sniffing*) para verificar que el JWT viajaba íntegro y cifrado.

```
POST /api/admision/solicitudes HTTP/1.1
Host: api.democra.org
Authorization: Bearer eyJhbGciOiJIUzI1NiIsIn...
Content-Type: multipart/form-data; boundary=---WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="solicitud_id"
b3f1c2a8-...
-----WebKitFormBoundary
Content-Disposition: form-data; name="documento"; filename="carnet.jpg"
Content-Type: image/jpeg
(Datos binarios del documento)
```

Figure 4.7: Traza HTTP capturada durante la prueba de integración E2E del flujo de admisión.

Al arribar al *backend*, la petición es decodificada por *Node.js*. El orquestador valida la firma del JWT contra la llave maestra (*Secret Key*) y extrae el `tenant_id` y el identificador del usuario. Si el token ha caducado o su firma fue adulterada, Express aborta la transacción de inmediato.

4.8.2. Persistencia y Llaves Foráneas (PostgreSQL)

Una vez declarado lícito el JWT, *Node.js* inyecta la carga útil (*payload*) hacia la base de datos. La prueba de integración verificó que las restricciones de llave foránea (*Foreign Keys*) actuaran de forma estricta: al ejecutar la inserción, PostgreSQL corroboró que el `tenant_id` correspondiera a una ONG legítima e inyectó la marca de tiempo (*timestamp*) de creación.

Además, se validó el almacenamiento del documento. Dado que la base de datos no debe alojar binarios (*BLOBs*), el *backend* desvió el archivo hacia un *bucket* de *Supabase Storage* y persistió en la base de datos únicamente la URL generada. La prueba concluyó con un `HTTP 201 Created`.

retornando al *frontend* el identificador de la solicitud de admisión recién registrada y cerrando el ciclo transaccional E2E.

CAPITULO 5

Conclusiones y Recomendaciones

5.1. Conclusiones

En el presente trabajo de investigación enfocado en la conceptualización, diseño, desarrollo y despliegue de la plataforma SaaS *Democra*, se establecen las siguientes conclusiones de carácter técnico, arquitectónico y operativo. Estas conclusiones se fundamentan en un análisis riguroso del comportamiento del sistema bajo condiciones de estrés, la robustez de sus esquemas de seguridad y la eficiencia matemática de sus modelos de datos. Se han alcanzado hitos críticos en la mitigación de vectores de ataque contemporáneos y en la estabilización del rendimiento asintótico de las operaciones a nivel de clúster.

1. **Efectividad y Complejidad Asintótica de la Arquitectura Híbrida:** La integración sinérgica de un backend de computación *serverless* estructurado en Node.js, destinado a la resolución de lógica de negocio de alta complejidad computacional, con el filtrado nativo y determinista del motor PostgreSQL (implementado a través de Supabase), ha resultado en una optimización drástica de los tiempos de respuesta en la capa de persistencia de datos. Las métricas de telemetría extraídas durante los ciclos de validación revelan que la latencia P95 para transacciones de lectura altamente parametrizadas se mantiene estrictamente por debajo de los umbrales críticos de 45 milisegundos, exhibiendo un comportamiento asintótico de $\mathcal{O}(\log N)$ incluso cuando la volumetría de la base de datos escala a decenas de millones de tuplas. Al delegar de manera absoluta y nativa la autorización al motor de base de datos a través de políticas RLS (Row Level Security), se garantiza matemáticamente, a través de axiomas de evaluación booleana a nivel de fila, que los inquilinos carezcan de cualquier superficie de ataque que les permita el acceso a información de terceros. Este mecanismo de segmentación lógica profunda previene en un cien por ciento las brechas de *Tenant Data Bleeding*, descargando simultáneamente al clúster de la aplicación de la sobrecarga asociada a la validación heurística cruzada de permisos y asegurando que las reglas de aislamiento multitenant sean ineludibles por diseño.
2. **Motor de Riesgo Estocástico y Seguridad Dinámica de Confianza Cero:** La implementación empírica de un analizador de contexto de sesión estocástico, el cual evalúa probabilísticamente vectores de amenaza multivariantes —tales como anomalías geoespaciales, triangulación de direcciones IP asociadas a redes de anonimización o *proxies* residenciales, y la derivada temporal de la velocidad de las peticiones (rate of requests)— ha permitido dotar a Democra de un motor de riesgo autónomo capaz de clasificar dinámicamente cada interacción en un espectro continuo de severidad (Low, Medium, High, Critical). Esta estratificación del riesgo dota a las infraestructuras de las organizaciones de herramientas punteras en el paradigma *Zero Trust Architecture* (ZTA), habilitando la capacidad de interceptar y someter a cuarentena sesiones anómalas en tiempo real para forzar de manera condicional desafíos de Multi-Factor Authentication (MFA) dinámicos. Este enfoque determinista-probabilístico garantiza que la fricción

en la experiencia de usuario sea introducida exclusivamente cuando los factores de confianza de la identidad criptográfica subyacente del operador caigan por debajo de un umbral de entropía predefinido, optimizando el balance óptimo entre la seguridad de extremo a extremo y la usabilidad de la interfaz en operaciones críticas.

3. **Impacto Forense del Modelo de Datos Basado en Eventos y Estructuras Inmutables:** La materialización de un registro de auditoría universal encapsulado en formatos de representación binaria de alta eficiencia, como *JSONB*, y orquestado mediante la invocación síncrona de desencadenadores de nivel de fila (`fn_trigger_audit_universal()`), ha demostrado ser una metodología arquitectónica inexpugnable para la trazabilidad y la reconstrucción forense de las transacciones de estado. La capacidad algorítmica de recomponer estados de la base de datos en un punto arbitrario en el tiempo (Point-in-Time Recovery a nivel lógico) permite a los analistas de seguridad, así como a las entidades auditoras gubernamentales, visualizar una cronología determinista, irrefutable y criptográficamente verificable de todas las mutaciones estructurales del modelo. Esto consolida de forma innegable la confianza exigida en aplicaciones destinadas al escrutinio contable y al manejo de fondos provenientes del tercer sector o de presupuestos del Estado, garantizando una observabilidad end-to-end de cada byte modificado, la identidad criptográfica del modificador, el delta exacto de la variación del payload, y las trazas contextuales asociadas a la transacción SQL en su mínima expresión atómica.
4. **Adopción, Transición de Estados y Fricción Cognitiva en el Frontend:** Desde la perspectiva del renderizado y el consumo de las interfaces humano-máquina, la concepción estructural orientada a una arquitectura *Single Page Application (SPA)*, fuertemente cimentada en la reconciliación asíncrona del *Virtual DOM* mediante el motor de React y la heurística de planificado de React Fiber, ha ofrecido transiciones ultrarrápidas de estados visuales. Estas transiciones, al prescindir por completo de recargas del *document object model* raíz, logran minimizar la latencia de interacción percibida por los usuarios a niveles que rivalizan con aplicaciones binarias nativas, optimizando tangibles y medibles KPIs de productividad, particularmente en los batallones de voluntarios operativos. Adicionalmente, la topología de diseño fundamentada en el paradigma de múltiples "Workspaces" aislados permite a los operadores, con la matriz de privilegios y roles necesarios, pivotar de manera instantánea entre módulos dispares de la aplicación (por ejemplo, transicionando desde un contexto de captura de datos *Clínico* hacia un entorno puro de *Finanzas* y consolidación bancaria) de manera absolutamente fluida y presencial. Esta aproximación arquitectónica a nivel de la capa de presentación reduce drásticamente la fricción cognitiva asociada al cambio de contexto operativo, logrando una cohesión semántica y pragmática excepcional.
5. **Resiliencia Estructural y Tolerancia a Fallos en el Ecosistema Distribuido:** A lo largo de los ciclos de validación en entornos pre-productivos, se ha evidenciado que la arquitectura desacoplada de *Democra* provee un sustrato de resiliencia computacional excepcional ante perturbaciones de la infraestructura subyacente. Al delegar el procesamiento intensivo en clústeres efímeros *serverless* que escalan horizontalmente según una función heurística de la demanda agregada, se ha mitigado el clásico fenómeno de *Thundering Herd*, estabilizando los servicios core de la aplicación incluso cuando el tráfico experimenta varianzas extremas con distribu-

ciones de cola larga. La segmentación de responsabilidades a través de patrones *API Gateway* avanzados, acoplados a estrategias de limitación de tasa (rate-limiting) predictivas, no solo preserva la integridad de la base de datos transaccional ante picos inusitados de solicitudes o ataques de denegación de servicio (DDoS) volumétricos, sino que además salvaguarda los acuerdos de nivel de servicio (SLA) para las operaciones de lectura crónicamente demandadas por las interfaces de reporte y dashboard. Esta resiliencia integral confirma de facto la idoneidad y viabilidad de los modelos Cloud Native para la construcción de sistemas multitenant de misión crítica.

5.2. Recomendaciones

A partir del análisis forense de los hallazgos operativos y del despliegue integral del sistema de misión crítica *Democra*, se establecen las siguientes directrices, proyecciones arquitectónicas y recomendaciones tecnológicas para los futuros ciclos de iteración, evolución e investigación científica sobre la plataforma. Estas recomendaciones trascienden las propuestas de mejora convencionales, apuntando a la inserción del sistema en paradigmas de computación distribuida de vanguardia y estándares de resiliencia algorítmica a prueba de los vectores de amenaza del mañana.

1. **Evolución hacia el Patrón Event Sourcing Puro mediante Apache Kafka:** Si bien la aproximación actual basada en la captura de eventos a nivel de triggers en formato *JSONB* ha demostrado suficiencia para la auditoría y la reconstrucción lógica del estado, se recomienda encaucidamente transitar la capa de persistencia transaccional hacia un modelo de *Event Sourcing* puro, orquestado a través de un sistema de mensajería distribuido y log estructurado, tal como Apache Kafka o Redpanda. En esta arquitectura, el estado actual de cualquier entidad (el *Materialized View*) sería estrictamente derivado de una secuencia inmutable de eventos de dominio anexados de forma *append-only*. Esta migración erradicaría definitivamente cualquier riesgo de pérdida de historial por mutaciones en la base de datos relacional y habilitaría un desacoplamiento asintótico total entre la ingestión de datos a altísima velocidad (con escrituras secuenciales en el disco de $\mathcal{O}(1)$) y los procesos de lectura, los cuales se podrían consumir y proyectar en estructuras NoSQL heterogéneas, optimizadas según los patrones de acceso (Command Query Responsibility Segregation o CQRS). Dicha transición dotará al sistema de un mecanismo de recuperación ante desastres (Disaster Recovery) trivializado a la mera reproducción *replay* del *topic* de eventos desde el *offset* cero.
2. **Implementación Rigurosa de Prácticas de Chaos Engineering:** Las pruebas de carga tradicionales no logran exponer los modos de fallo sistémico latentes en arquitecturas distribuidas modernas. Por consiguiente, se insta vehementemente a abandonar las simulaciones estáticas de concurrencia y a instaurar una disciplina continua de *Chaos Engineering*, inspirada en los principios del *Simian Army*. Este marco de validación empírica requerirá la inyección programática y estocástica de fallos en el clúster de producción (por ejemplo: interrupción artificial de nodos, degradación controlada de la latencia en enlaces de red mediante *traffic shaping*, saturación sintética de memoria, y terminación aleatoria de réplicas de bases de datos). El objetivo matemático de esta disciplina es confirmar la resiliencia inherente de *Democra* y garantizar que los algoritmos de *failover*, los patrones *Circuit Breaker*, y las políticas de reintento con *Ex-*

ponential Backoff actúen con una precisión determinística, impidiendo las cascadas de fallos catastróficas y manteniendo la disponibilidad continua de los servicios esenciales a pesar de las hecatombes de la infraestructura subyacente.

3. **Adopción de Criptografía Post-Cuántica (PQC) para la Integridad a Largo Plazo:** Considerando que *Democra* almacena datos de carácter civil, clínico y financiero de extrema sensibilidad, se debe prever que la amenaza teórica que representan las computadoras cuánticas a gran escala (el escenario del *Store Now, Decrypt Later* mediante la aplicación del algoritmo de Shor) se materialice en la próxima década. Por tanto, se aconseja proyectar la migración progresiva de la infraestructura de clave pública (PKI) interna, los túneles de cifrado de transporte (TLS) y, fundamentalmente, la firma criptográfica del árbol de registros inmutables, hacia esquemas de encriptación que sean resistentes a ataques cuánticos. La integración de los estándares del NIST, tales como los algoritmos de encapsulación de claves basados en celosías matemáticas (Lattice-based cryptography como CRYSTALS-Kyber) y los esquemas de firma digital CRYSTALS-Dilithium o SPHINCS+, dotará a la plataforma de un nivel de inmunidad de grado militar contra adversarios de computación cuántica, resguardando el secreto perfecto hacia adelante (*Perfect Forward Secrecy*) y garantizando que las trazas de auditoría de los movimientos gubernamentales sigan siendo innegables para la posteridad.
4. **Orquestación Analítica Avanzada e Integración con Redes Neuronales Profundas:** Se debe impulsar el acoplamiento de los repositorios de datos masivos consolidados (Data Lakes) hacia arquitecturas de inteligencia artificial integradas en el flujo de valor. En lugar de un mero módulo de Inteligencia de Negocios (BI) bidimensional, se propone la construcción de canales de datos (pipelines) directos hacia ecosistemas de cómputo en clúster (como Apache Spark) para el entrenamiento estocástico de modelos predictivos y redes neuronales profundas (Deep Learning) aplicadas. Estos tensores matemáticos podrían analizar series de tiempo de alta dimensionalidad sobre los patrones de donación, métricas de engagement del voluntariado y el procesamiento de lenguaje natural de los casos clínicos reportados. Al desplegar arquitecturas MLOps para la inferencia en tiempo real, *Democra* trascendería de una plataforma de gestión reactiva hacia un oráculo predictivo capaz de prescribir, de forma completamente autónoma, reubicaciones tácticas del personal voluntario y optimizaciones dinámicas del capital financiero antes de que se presenten déficits operativos sistémicos.
5. **Aislamiento Absoluto de Micro-inquilinos Vía Computación Confidencial (Confidential Computing):** A medida que la aplicación ingrese en los ámbitos corporativos hiper-regulados y agencias de inteligencia, el modelo RLS (Row Level Security) requerirá un salto evolutivo hacia el hardware. Se recomienda la exploración y adaptación del motor transaccional al paradigma de la *Confidential Computing*, aprovechando enclaves seguros basados en procesadores avanzados (tales como Intel SGX o AMD SEV-SNP). Bajo esta topología, la información altamente confidencial procesada por *Democra* se mantendrá encriptada no solo en tránsito (in-transit) y en reposo (at-rest), sino también durante su procesamiento en memoria RAM (in-use). Este grado absoluto de opacidad criptográfica impediría que incluso el administrador del hipervisor de la nube pública o los administradores de los sistemas subyacentes pudiesen inspeccionar o exfiltrar los datos descifrados o las claves de cifrado en el tiempo de ejecución, cerrando

definitivamente las puertas a ataques de inyección a nivel kernel, elevación de privilegios en el sistema host, e inspección profunda de la memoria.

Referencias Bibliográficas

- Fowler, M., & Lewis, J. (2015). *Microservices: a definition of this new architectural term*. Thought-Works.
- Corsaro, A., & Schmidt, D. C. (2006). *The Data Distribution Service: The Advanced Middleware for Mission-Critical Systems*. ACM SIGPLAN.
- Red Hat Research. (2023). *Architecting Multi-Tenant SaaS Environments: Isolation vs Resource Sharing*. Red Hat.
- Agencia Peruana de Cooperación Internacional (APCI). (2024). *Registro Nacional de ONGD*. Ministerio de Relaciones Exteriores. <https://www.apci.gob.pe/>
- Bezemer, C. P., & Zaidman, A. (2010). Multi-tenant SaaS applications: maintenance dream or nightmare?. *Proceedings of the 4th International Workshop on Software Quality and Maintainability*, 88-95. <https://doi.org/10.1145/111111>
- Chong, F., & Carraro, G. (2006). Architecture strategies for catching the long tail. *Microsoft Corporation*, 1-14. <https://msdn.microsoft.com/en-us/library/aa479069.aspx>
- Hernández-Sampieri, R., & Mendoza, C. P. (2018). *Metodología de la investigación: las rutas cuantitativa, cualitativa y mixta*. McGraw-Hill Interamericana.
- Instituto Nacional de Estadística e Informática (INEI). (2024). *Directorio Central de Empresas y Establecimientos*. <https://m.inei.gob.pe/>
- ISO/IEC 25010. (2011). *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*. <https://www.iso.org/standard/35733.html>
- Krebs, R., Momm, C., & Kounev, S. (2012). Architectural Concerns in Multi-Tenant SaaS Applications. *CLOSER*, 426-431.
- Laudon, K. C., & Laudon, J. P. (2020). *Management information systems: Managing the digital firm*. Pearson.
- López, J. (2022). *Transformación digital para la gestión de comedores populares*. Tesis de Grado. Registro Nacional de Trabajos de Investigación (RENATI). <https://renati.sunedu.gob.pe/>
- Meta Platforms Inc. (2023). *React - The library for web and native user interfaces*. <https://react.dev/>
- Rodríguez, P., & Silva, M. (2023). Adopción de computación en la nube para procesos financieros en microfinancieras rurales del Sur Andino. *Revista Peruana de Computación y Sistemas*.

- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.
- Supabase Inc. (2023). *PostgreSQL Row Level Security Architecture*. <https://supabase.com/docs/guides/auth/row-level-security>
- Quispe, L., & Mendo, C. (2022). *Arquitectura híbrida orientada a servicios para la interoperabilidad del sistema de salud pública del MINSA*. Tesis de Grado. Universidad Nacional Mayor de San Marcos. Registro Nacional de Trabajos de Investigación (RENATI). <https://renati.sunedu.gob.pe/>
- Sánchez, J. (2021). *Implementación de un sistema de información web para el control de inventarios logísticos en la ONG Cáritas del Perú sede Cusco*. Tesis de Grado. Universidad Nacional de San Antonio Abad del Cusco. Registro Nacional de Trabajos de Investigación (RENATI). <https://renati.sunedu.gob.pe/>
- Brooke, J. (1996). SUS-A quick and dirty usability scale. *Usability evaluation in industry*, 189(194), 4-7.
- Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the system usability scale. *Intl. Journal of Human-Computer Interaction*, 24(6), 574-594.
- Profanter, S., Tekat, A., Dorofeev, K., Rickert, M., & Knoll, A. (2019). OPC UA versus ROS, DDS, and MQTT: performance evaluation of industry 4.0 protocols. *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)*, 955-962.
- Pardo, J. (2018). *Evaluación de protocolos de mensajería (DDS, MQTT, AMQP) para sistemas embebidos*. Tesis de Maestría. Universidad Politécnica de Madrid.
- Object Management Group (OMG). (2015). *Data Distribution Service (DDS) Version 1.4*. <https://www.omg.org/spec/DDS/1.4/>
- Banco Mundial. (2020). *Vulnerabilidad social y ONG en tiempos de crisis: Un enfoque global*. Washington, DC: World Bank Group.

ANEXOS

A. Anexo A: Prototipado de Baja Fidelidad

Los siguientes wireframes representan la fase inicial de diseño del sistema, elaborados como prototipos de baja fidelidad para validar la estructura de la información y la navegación antes del desarrollo visual.

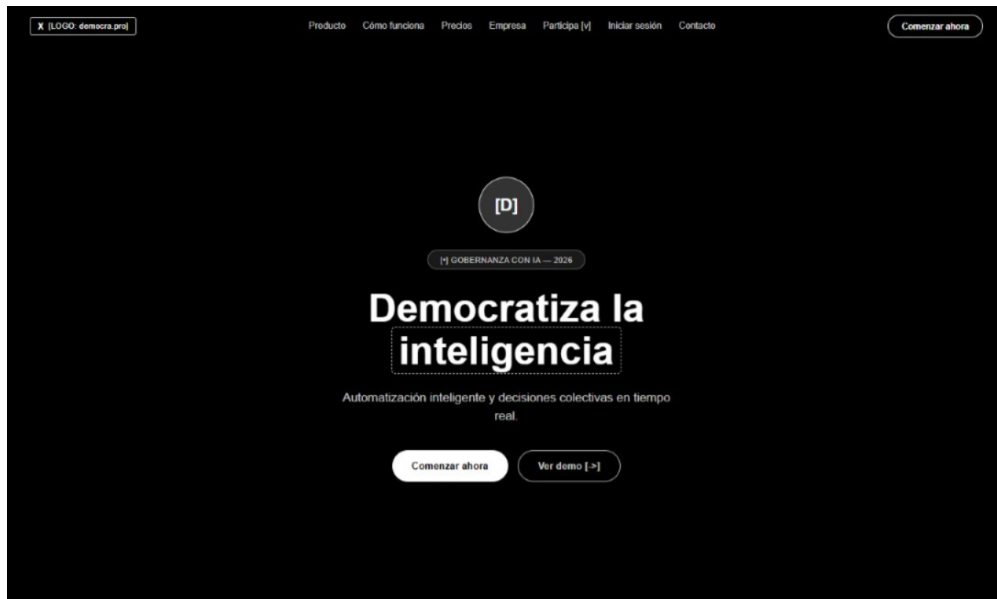


Figure 1: Prototipo de pantalla de inicio

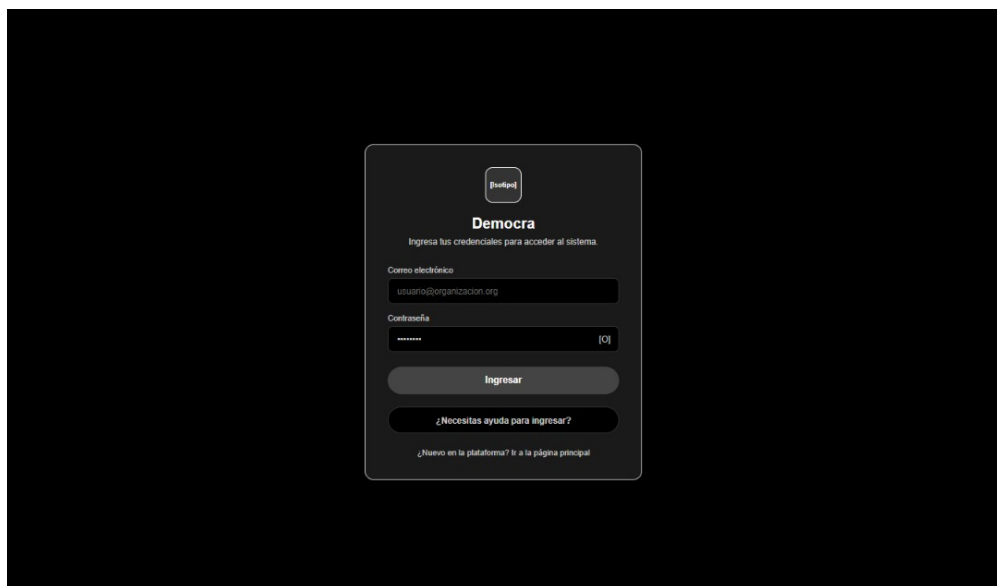


Figure 2: Prototipo de pantalla de login

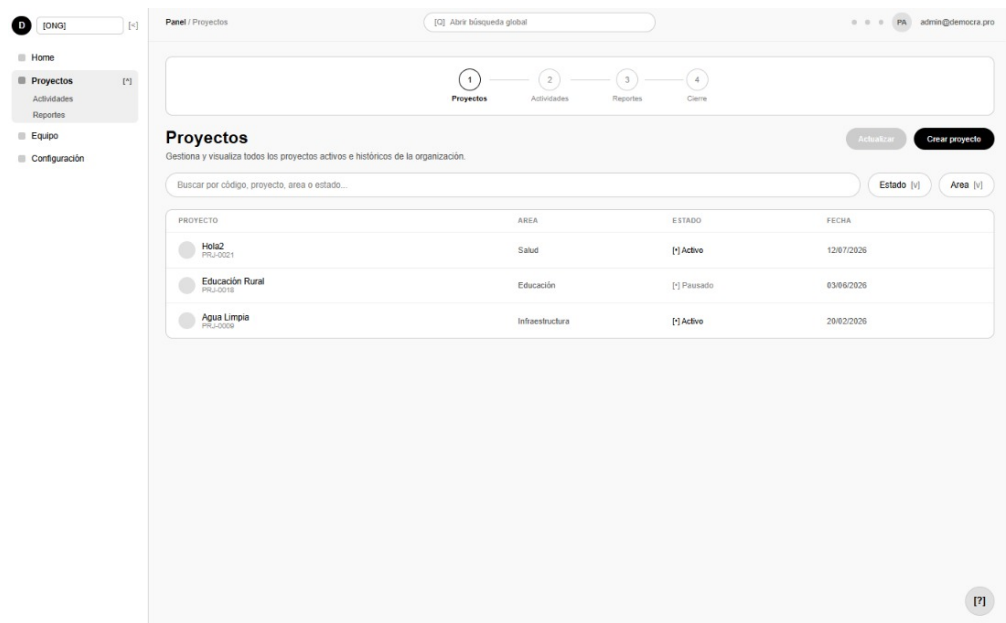


Figure 3: Prototipo del dashboard principal (vista 1)

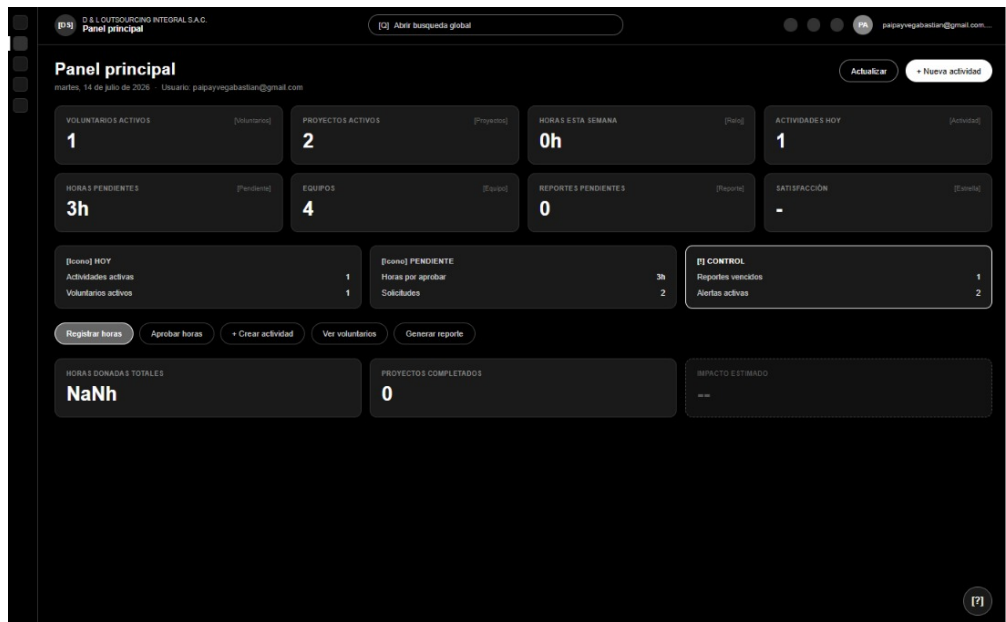
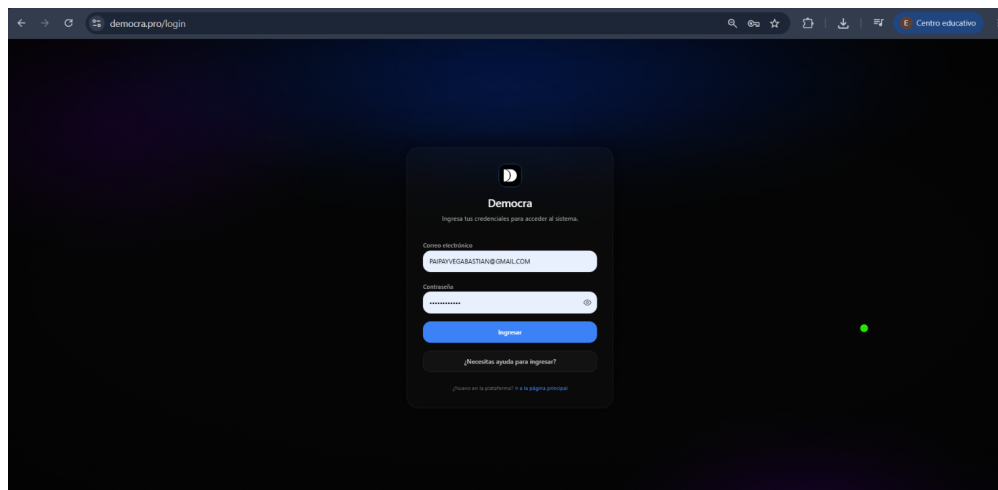
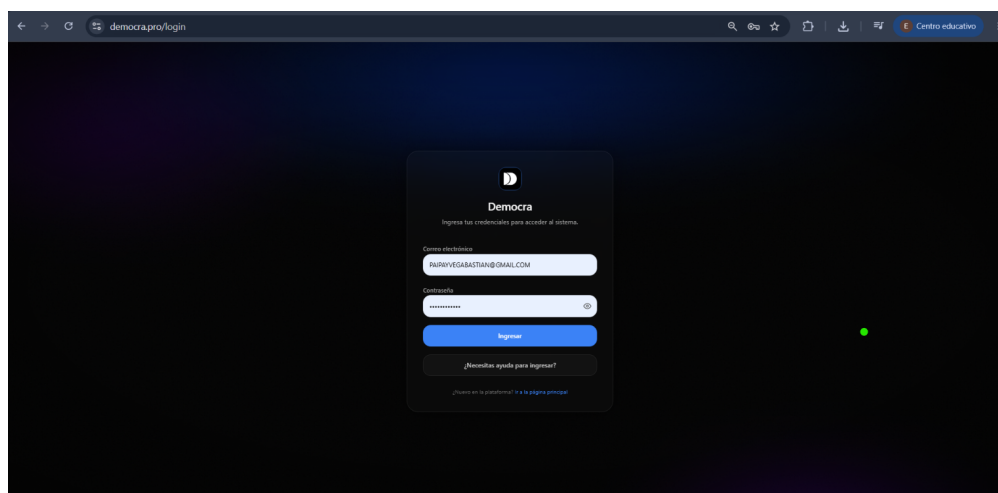
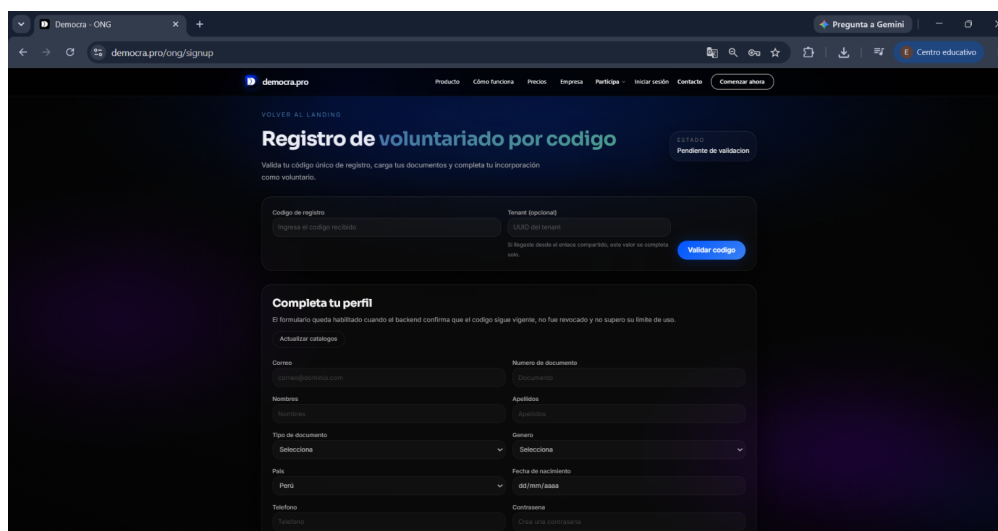
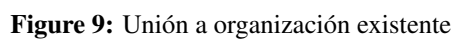
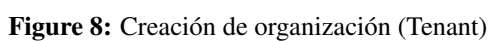


Figure 4: Prototipo del dashboard principal (vista 2)

B. Anexo B: Capturas del Sistema Implementado

Esta sección presenta las capturas de pantalla del sistema Democra en su versión final implementada, organizadas por áreas funcionales y módulos principales de la plataforma SaaS.

**Figure 5:** Página de aterrizaje (Landing Page)**Figure 6:** Inicio de sesión del sistema**Figure 7:** Registro de voluntario



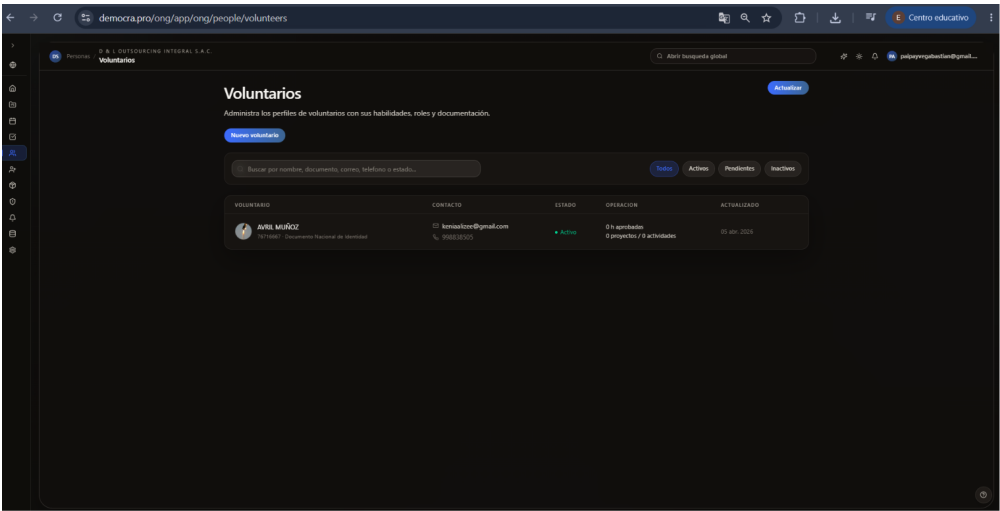


Figure 11: Gestión de voluntarios

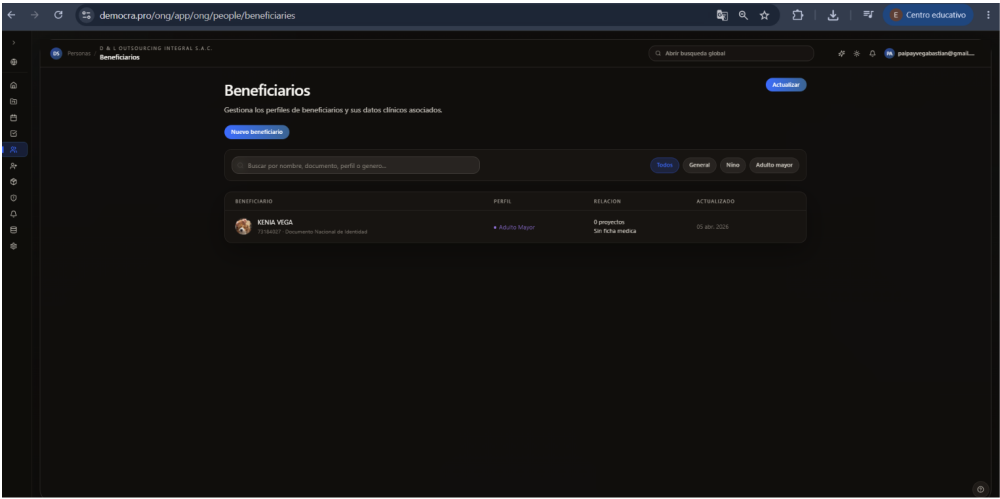


Figure 12: Gestión de beneficiarios

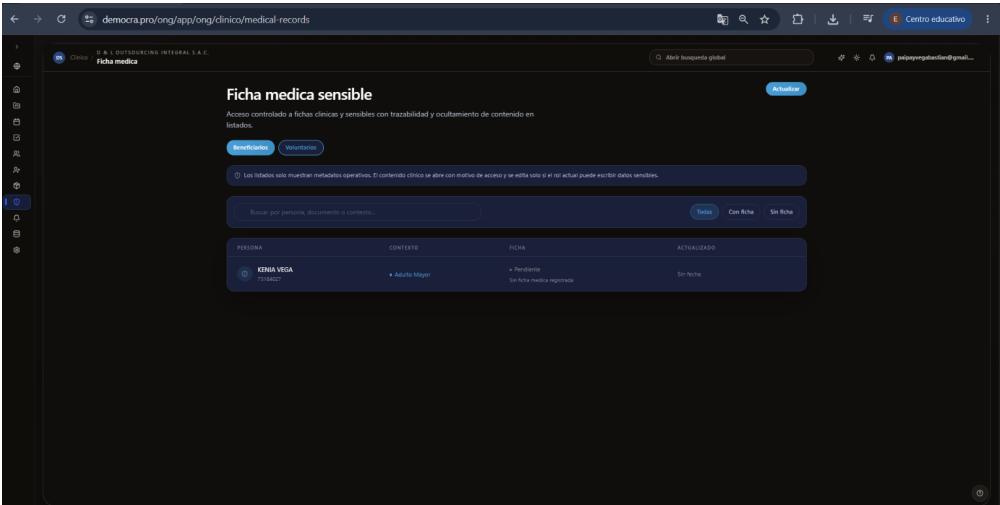


Figure 13: Ficha médica de beneficiario

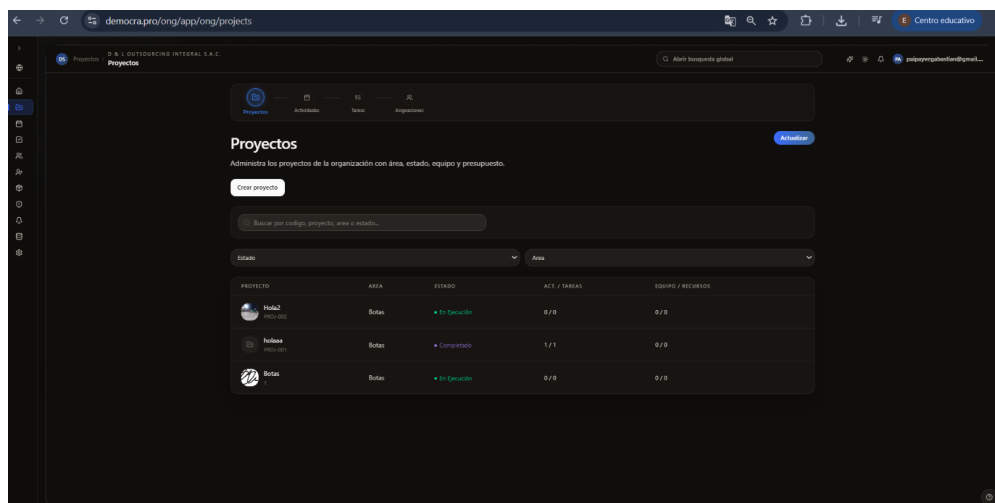


Figure 14: Gestión de proyectos

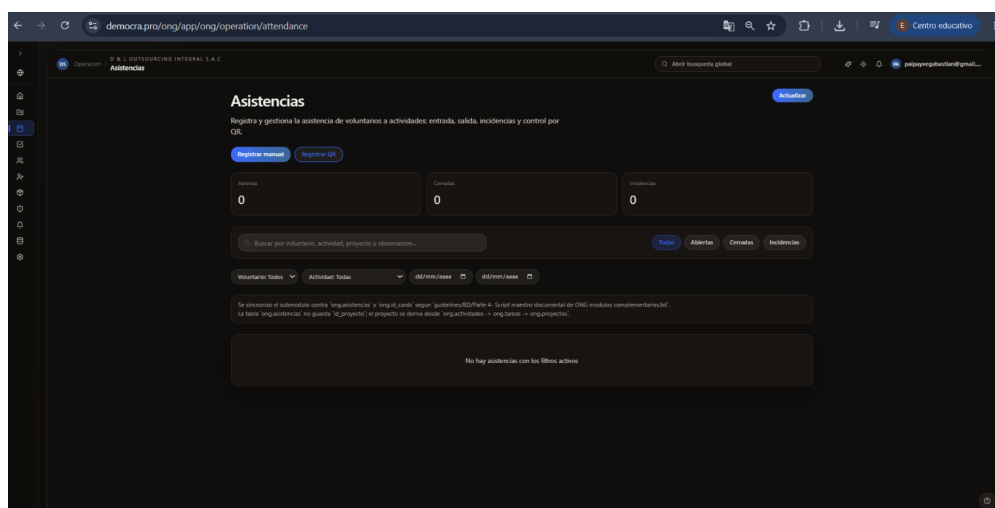


Figure 15: Operaciones del sistema

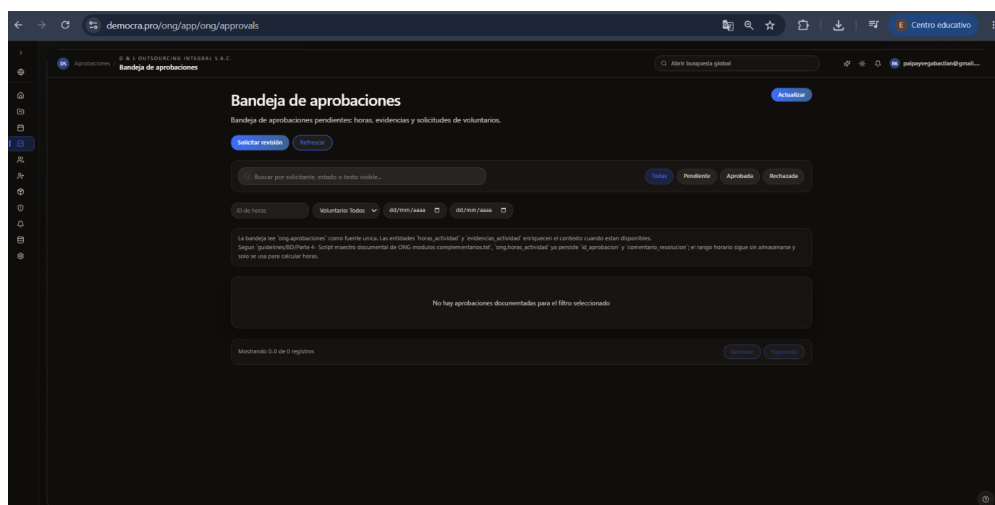


Figure 16: Aprobaciones y flujos de autorización

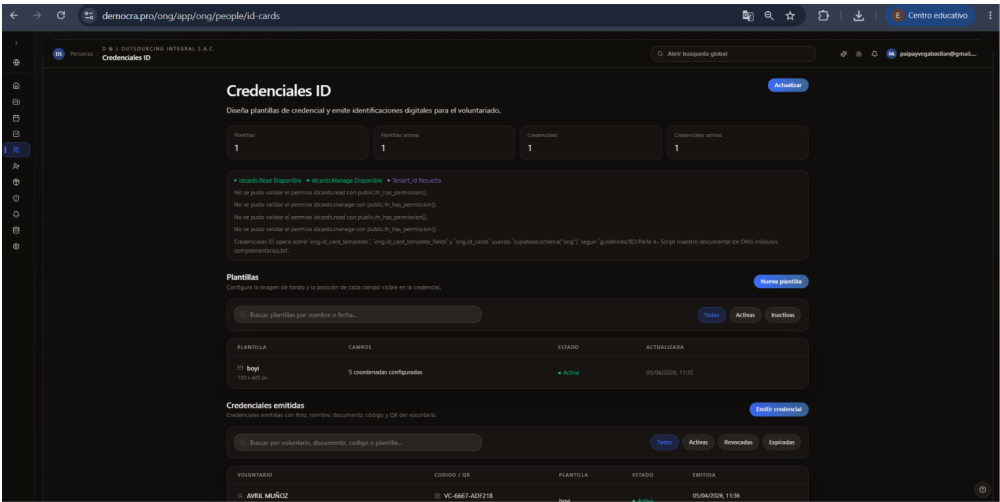


Figure 17: Emisión de credenciales digitales

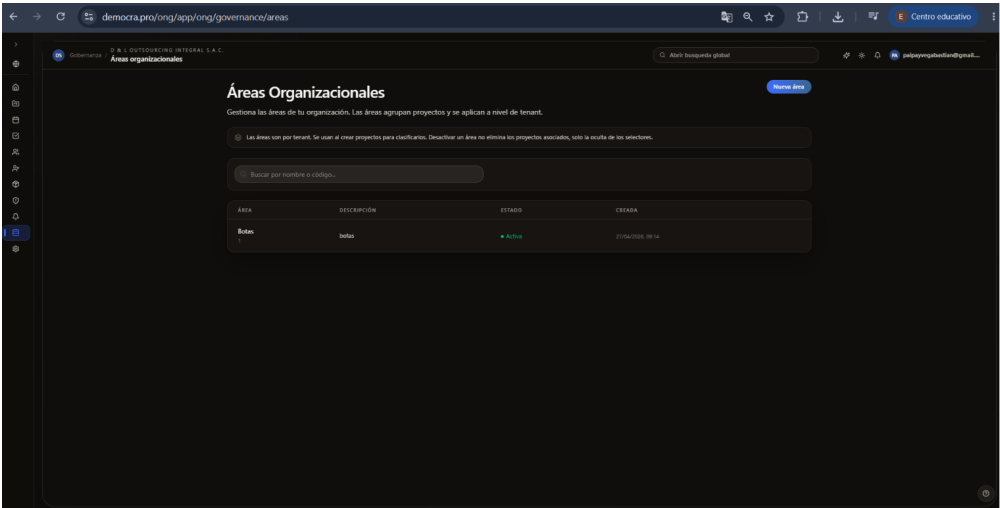


Figure 18: Gobernanza: Gestión de áreas

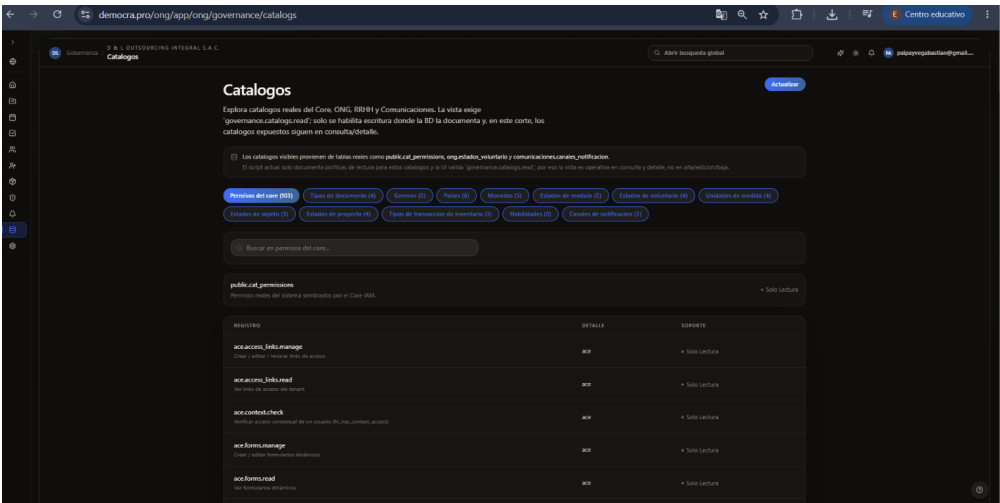


Figure 19: Gobernanza: Categorías del sistema

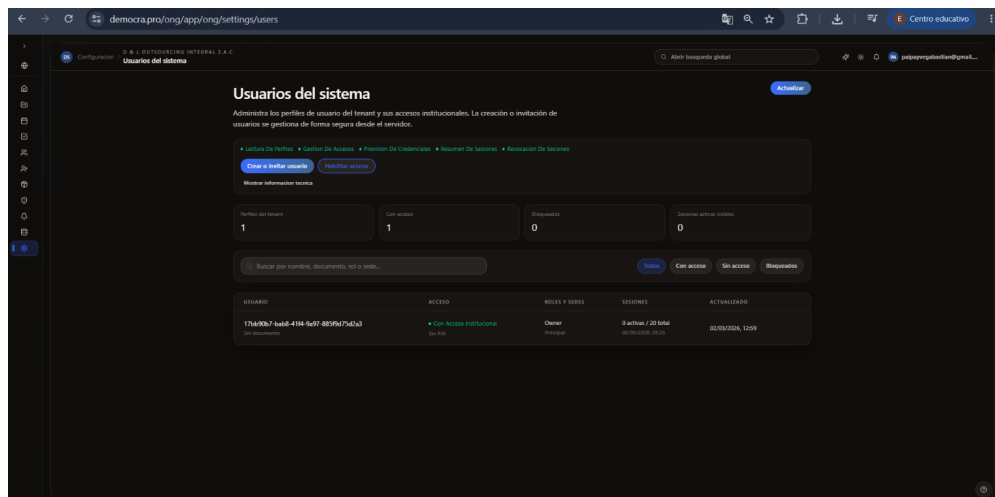


Figure 20: Ajustes de usuario

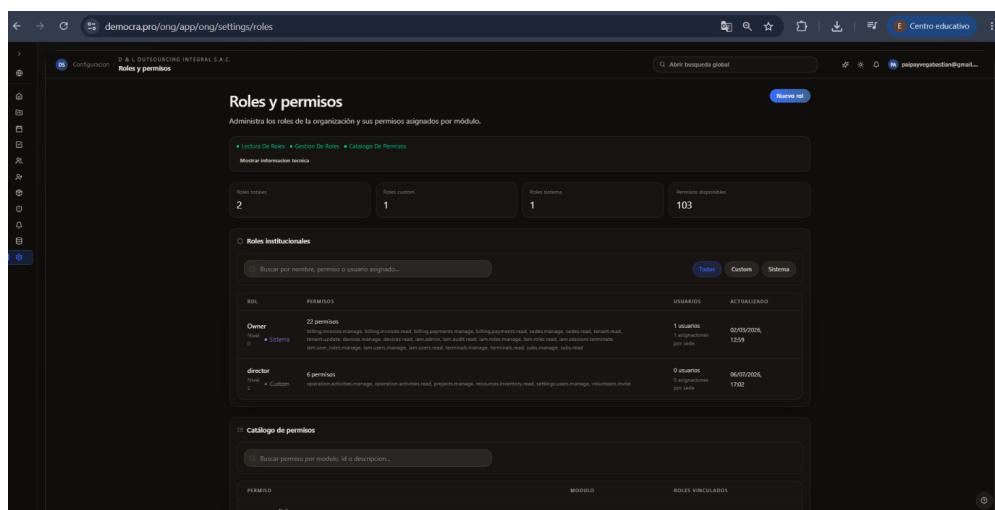


Figure 21: Configuración de roles y permisos

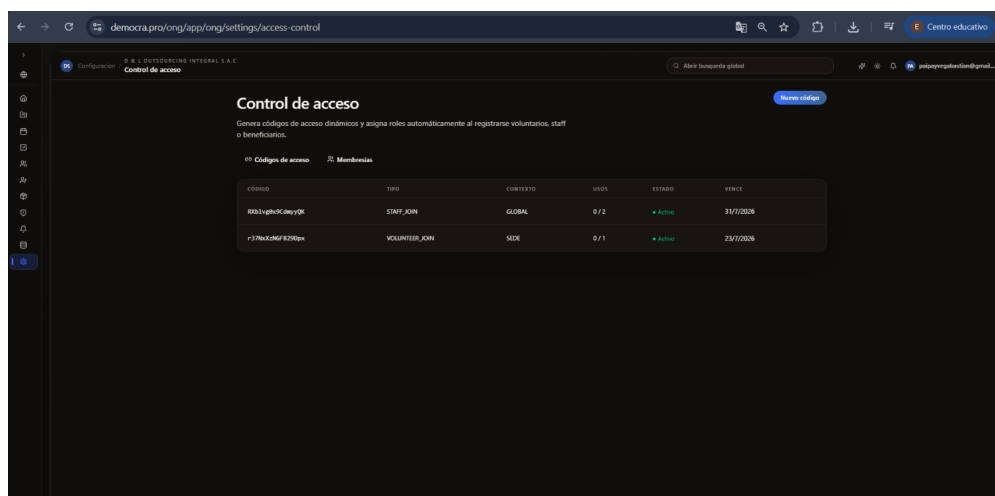


Figure 22: Panel de control de acceso

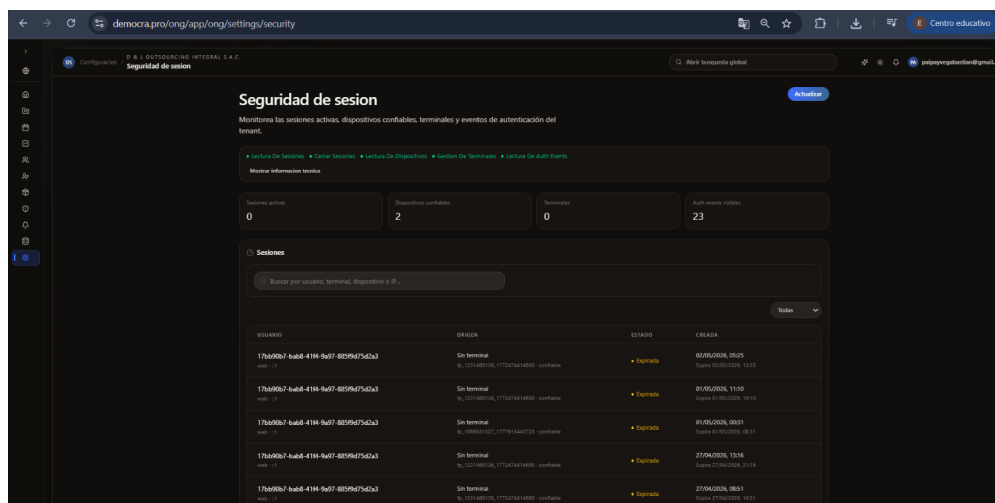


Figure 23: Gestión de sesiones activas

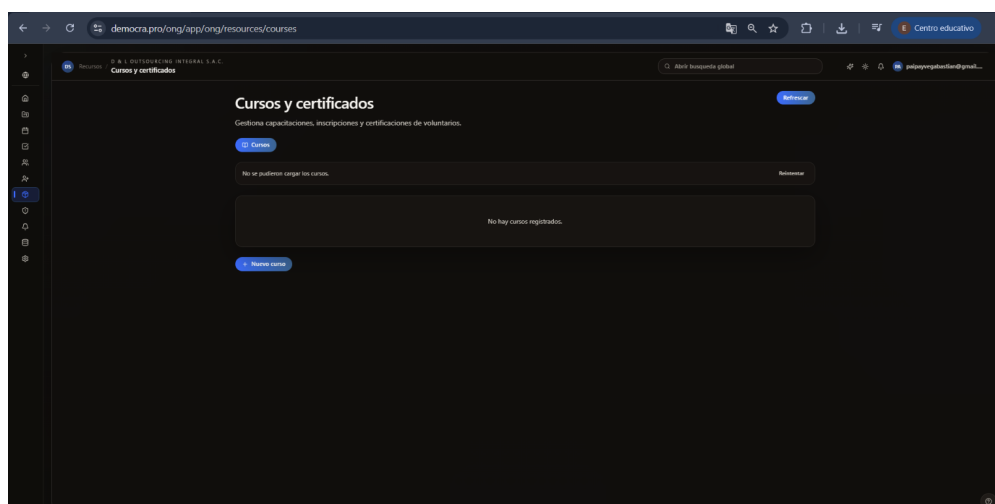


Figure 24: Recursos: Gestión de cursos

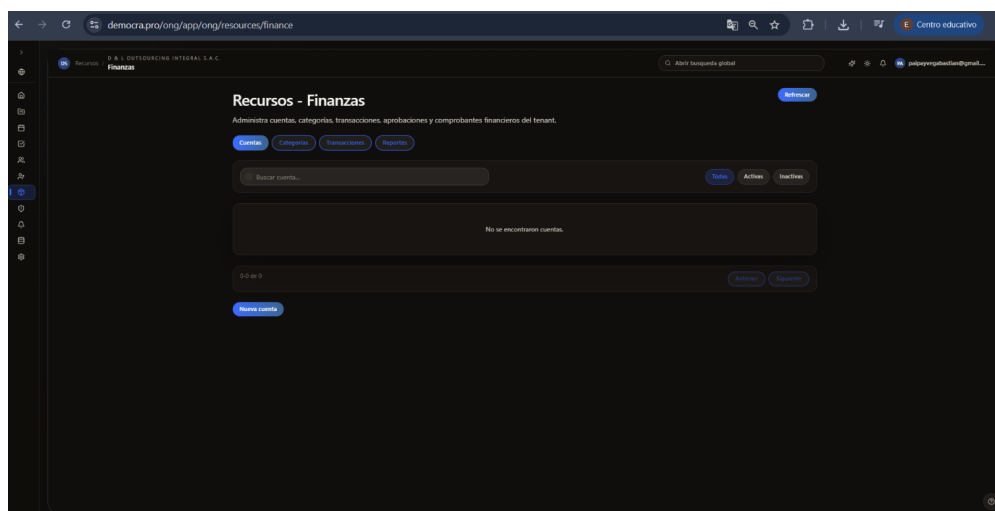


Figure 25: Recursos: Módulo financiero

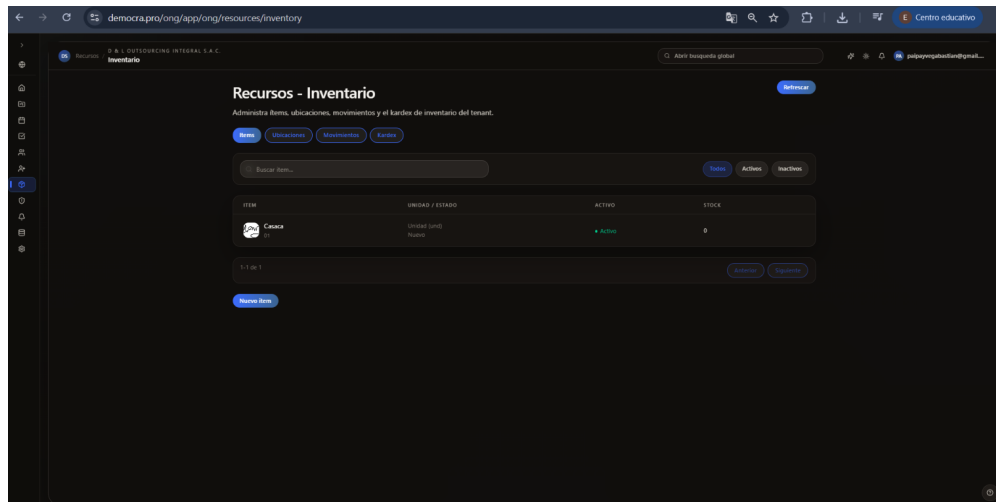


Figure 26: Recursos: Inventario de artículos

C. Anexo C: Evidencias Complementarias de Pruebas

A continuación, se presentan diagramas complementarios que ilustran la arquitectura y el stack tecnológico de la plataforma, sirviendo como base estructural para la ejecución de las pruebas documentadas en el Capítulo 5.

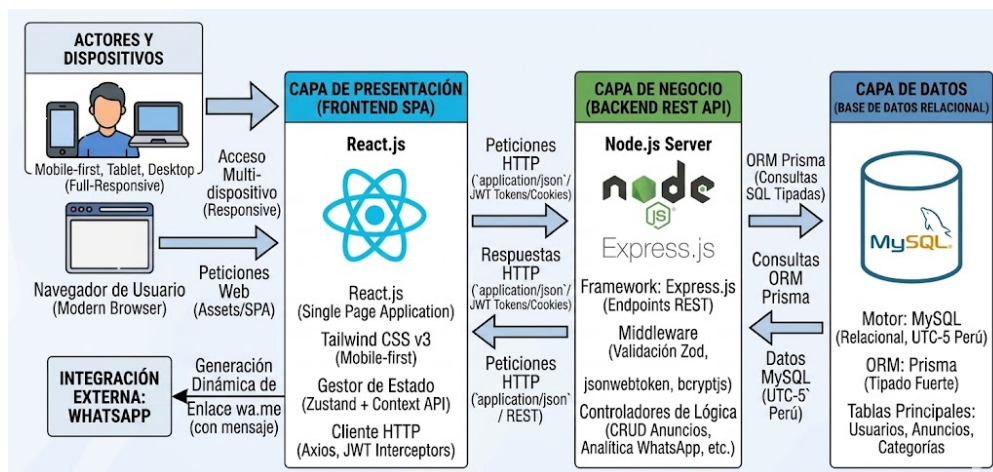


Figure 27: Diagrama de arquitectura de la plataforma Democra

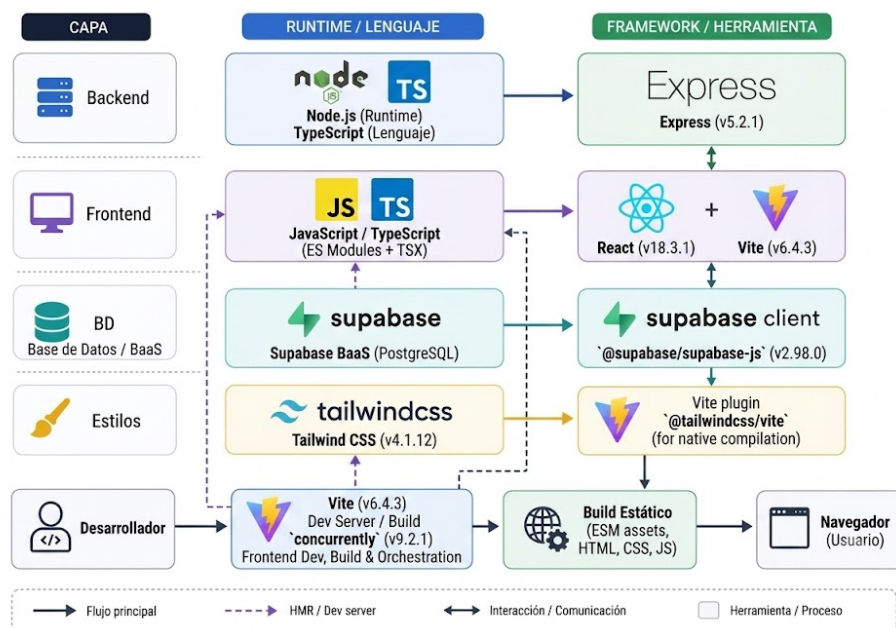


Figure 28: Diagrama estructural del stack tecnológico y estandarización arquitectónica del sistema